

Resumen

El presente Trabajo Final de Grado tiene como objetivo final, el desarrollo y evaluación de modelos de predicción de ventas en el sector de la automoción a través del uso de técnicas de Machine Learning. Dicho estudio se llevará a cabo integrando información histórica de ventas junto con variables macroeconómicas que se consideran como potencialmente relevantes para mejorar la capacidad predictiva.

Para ello, se llevará a cabo un proceso de recopilación, limpieza y transformación de los datos de diferentes variables macroeconómicas de diferentes fuentes. Algunos de estos indicadores son el índice de precios al consumo, el precio del crudo, o los tipos de interés bancarios, entre otras. Posteriormente, se realizará una agregación temporal de datos macroeconómicos con datos de ventas, preservando la misma granularidad temporal. Seguidamente, se realizará un estudio de correlación y relevancia de variables macroeconómicas, a fin de identificar las más determinantes e informativas, descartando las demás. Finalmente, se establecerá una partición estándar de entrenamiento, desarrollo y test para la construcción de sistemas predictivos.

Una vez construido el dataset, se plantea el entrenamiento de diferentes modelos de regresión basados en técnicas de Machine Learning, cuya entrada son las variables macroeconómicas seleccionadas, y la predicción es el número de ventas estimado. El objetivo final es ensamblar un sistema capaz de predecir la demanda del sector automovilístico y asistir en el proceso de toma de decisiones corporativas.

Palabras clave:

Predicción de ventas; Machine Learning; Deep Learning; Modelos de regresión; Variables macroeconómicas; Análisis de datos; Selección de variables; Sector automovilístico

Resum

El present Treball de Fi de Grau té com a objectiu final el desenvolupament i l'avaluació de models de predicció de vendes en el sector de l'automoció mitjançant l'ús de tècniques d'aprenentatge automàtic. Aquest estudi es durà a terme integrant informació històrica de vendes junt amb variables macroeconòmiques que es consideren potencialment rellevants per a millorar la capacitat predictiva.

Per a això, es realitzarà un procés de recopilació, neteja i transformació de les dades de diferents variables macroeconòmiques procedents de diverses fonts. Alguns d'aquests indicadors són l'índex de preus al consum, el preu del cru o els tipus d'interés bancaris, entre altres. Posteriorment, es durà a terme una agregació temporal de dades macroeconòmiques amb dades de vendes, mantenint la mateixa granularitat temporal. Seguidament, es realitzarà un estudi de correlació i rellevància de les variables macroeconòmiques, amb la finalitat d'identificar les més determinants i informatives, descartant les altres. Finalment, s'establirà una partició estàndard en conjunts d'entrenament, validació i prova per a la construcció de sistemes predictius.

Una vegada construït el conjunt de dades, es planteja l'entrenament de diferents models de regressió basats en tècniques d'aprenentatge automàtic, en què l'entrada són les variables macroeconòmiques seleccionades i la predicció és el nombre de vendes estimat. L'objectiu final és desenvolupar un sistema capaç de predir la demanda del sector automobilístic i assistir en el procés de presa de decisions corporatives.

Paraules clau:

Predicció de vendes; Aprenentatge automàtic; Aprenentatge profund; Models de regressió; Variables macroeconòmiques; Anàlisi de dades; Selecció de variables; Sector automobilístic

Abstract

This Final Degree Project aims to develop and evaluate sales forecasting models in the automotive sector through the use of Machine Learning techniques. The study is carried out by integrating historical sales data with macroeconomic variables that are considered potentially relevant to improve predictive performance.

To achieve this, a process of data collection, cleaning, and transformation of various macroeconomic variables from different sources is conducted. Some of these indicators include the Consumer Price Index, crude oil prices, and bank interest rates, among others. Subsequently, a temporal aggregation of macroeconomic data with sales data is performed, maintaining the same level of temporal granularity. This is followed by a correlation and relevance analysis of macroeconomic variables in order to identify the most significant and informative ones, discarding the rest. Finally, a standard split into training, validation, and test sets is established for the development of predictive systems.

Once the dataset is constructed, the training of different regression models based on Machine Learning techniques is proposed, where the input consists of the selected macroeconomic variables and the output corresponds to the estimated number of sales. The final objective is to develop a system capable of predicting demand in the automotive sector and supporting corporate decision-making processes.

Keywords:

Sales Forecasting; Machine Learning; Deep Learning; Regression Models; Macroeconomic Variables; Data Analysis; Feature Selection; Automotive Industry

Índice

Resumen	1
Resum	2
Abstract.....	3
Capítulo 1: Objetivos, Motivación y Estructura del Documento	16
1. Objetivos, Motivación, Justificación y Secuenciación del TFG	17
1.1 Objetivos	17
1.1.1 Objetivo general.....	17
1.1.2 Objetivos secundarios	17
1.2 Motivación	19
1.3 Estructura del documento	20
Capítulo 2: Fundamentos del Aprendizaje Automático.....	22
2. Fundamentos del Aprendizaje Automático.....	23
2.1 Contexto histórico	23
2.1.1 Origen de la Inteligencia Artificial y del Aprendizaje Automático	23
2.1.2 El Aprendizaje Automático y sus tipos	28
2.1.3 Aprendizaje Supervisado	32
2.2 Árboles de decisión	33
2.2.1 Concepto y estructura	34
2.2.2 Funcionamiento de los árboles de decisión	36
2.2.2.1 Construcción del árbol.....	36
2.2.2.2 Predicción mediante el árbol	37
2.2.2.3 Funcionamiento en problemas de regresión.....	38
2.2.3 Complejidad sobreajuste y poda	39
2.2.3.1 Complejidad del árbol.....	39
2.2.3.2 Sobreajuste.....	39
2.2.3.3 Subajuste.....	40
2.2.3.4 Control de la complejidad mediante parámetros	41
2.2.3.5 Poda del árbol	42
2.2.4 Ventajas y limitaciones	43

2.2.4.1	Ventajas de los árboles de decisión	43
2.2.4.2	Limitaciones de los árboles de decisión	44
2.3	Naive Baseline	45
2.3.1	Funcionamiento del Naive Baseline	45
2.3.2	Aplicación en este proyecto	46
2.3.3	Ventajas y limitaciones	46
2.4	Ridge Regression	47
2.4.1	Funcionamiento del Ridge Regression.....	47
2.4.2	Aplicación a la regresión en este proyecto	48
2.4.3	Hiperparámetros principales	49
2.4.4	Ventajas y limitaciones	50
2.5	Gradient Boosting	51
2.5.1	Funcionamiento del Gradient Boosting	51
2.5.2	Aplicación a la regresión en este proyecto	52
2.5.3	Hiperparámetros principales	53
2.5.4	Ventajas y limitaciones	54
2.6	Random Forest	55
2.6.1	Funcionamiento de un Random Forest	56
2.6.2	Aplicaciones a la regresión y a la clasificación	57
2.6.3	Aplicación a la regresión en este proyecto	57
2.6.4	Aplicación a la clasificación.....	58
2.6.5	Parámetros principales	59
2.6.6	Ventajas y limitaciones	60
2.7	Métricas de evaluación	61
2.7.1	Comparación de los diferentes modelos	63
2.7.2	Consideraciones sobre la evaluación.....	64
Capítulo 3: Preparación de los datasets para el entrenamiento y evaluación		65
3.	Datasets	66
3.1	Zona geográfica seleccionada.....	66
3.1.1	Europa	66
3.1.2	Unión Europea	66

3.1.3	Eurozona	67
3.2	El dataset BMW	68
3.2.1	Fuentes de los datos	68
3.2.2	Análisis exploratorio de las ventas de BMW	71
3.2.2.1	Estadísticos importantes y variación absoluta y porcentual 71	
3.3	Variables macroeconómicas seleccionadas	76
3.3.1	Tasa de desempleo	76
3.3.2	Crecimiento del PIB	77
3.3.3	Tipo de depósito del BCE	77
3.3.4	Índice Armonizado de Precios al Consumo	78
3.3.5	Producción industrial.....	78
3.3.6	Crecimiento de las ventas minoristas	79
3.3.7	Confianza del Consumidor	79
3.3.8	Compensación por empleado	80
3.3.9	Precio del Petróleo de Brent	80
3.3.10	Euríbor a 12 meses	81
3.3.11	Población total.....	81
3.3.12	Tipo de cambio EUR/USD	82
3.4	Cobertura geográfica de las variables macroeconómicas	83
3.5	Estudio de importancia de las variables macroeconómicas	83
3.5.1	Análisis de las variables macroeconómicas.....	83
3.5.2	Análisis de los estadísticos de las variables macroeconómicas.....	92
3.5.3	Matriz de correlación entre variables macroeconómicas	95
3.5.3.1	Cálculo de la matriz de correlación	95
3.5.3.2	Interpretación de la matriz de correlación.....	96
3.5.3.2.1	Correlaciones muy altas ($ r > 0,85$, posible alta multicolinealidad).....	97
3.5.3.2.2	Correlaciones altas ($0,70 \leq r < 0,85$)	98
3.5.3.2.3	Correlaciones moderadas ($0,50 \leq r < 0,70$)	99
3.5.3.2.4	Correlaciones débiles ($ r < 0,20$)	100

3.5.3.3	Relación con las matriculaciones de BMW	100
3.5.3.3.1	Cálculo de las correlaciones entre las variables macro y las matriculaciones de BMW.....	100
3.5.3.3.1.1	Correlaciones muy fuertes ($ r > 0,80$)	102
3.5.3.3.1.2	Correlaciones fuertes ($0,70 \leq r < 0,80$).....	102
3.5.3.3.1.3	Correlaciones moderadas ($0,50 \leq r < 0,70$)	102
3.5.3.3.1.4	Correlaciones moderadas ($0,50 \leq r < 0,70$)	102
3.5.3.4	Diagnóstico de multicolinealidad.....	103
3.5.3.4.1	Cálculo de la multicolinealidad existente y su interpretación	103
3.6	Estudio de importancia de Variables Macroeconómicas	105
3.6.1	Introducción	105
3.6.2	Estudio inicial	106
3.6.2.1	Carga y unión de los datos.....	106
3.6.2.2	Definición de X e Y.....	107
3.6.2.3	Entrenamiento del RandomForestRegressor	108
3.6.2.4	Interpretación de la tabla y el gráfico de importancias	110
3.6.2.5	Conclusión de los resultados obtenidos	112
3.6.3	Re-evaluación con las variables seleccionadas	112
3.6.3.1	Carga de los datasets y selección de las variables macroeconómicas	112
3.6.3.2	Construcción de las matrices X e Y, y entrenamiento del modelo Random Forest	113
3.6.3.3	Comparación con el modelo completo anterior	115
3.6.4	Re-evaluación con las variables seleccionadas	116
3.6.4.1	Carga del dataset y unión del conjunto de datos	116
3.6.4.2	Partición del conjunto de datos: train/dev/test.....	117
3.6.4.3	Optimización de hiperparámetros	118
3.6.4.4	Selección de la combinación óptima	121
3.6.4.5	Visualización del efecto de los hiperparámetros	122
3.6.4.6	Evaluación final en test	123
3.6.4.7	Comparación entre valores reales y predichos.....	124

3.6.4.8	Conclusiones del apartado	126
Capítulo 4: Modelos predictivos desarrollados		127
4.	Modelos predictivos desarrollados	128
4.1	Metodología experimental	128
4.1.1	Preparación del conjunto de datos	128
4.2	Random Forest	129
4.2.1	Partición temporal de los datos (Enfoque A)	130
4.2.1.1	Creación y evaluación del Baseline Naive	131
4.2.1.2	Random Forest con partición temporal y evaluación en test 132	
4.2.1.3	TimeSeriesSplit.....	135
4.2.1.4	GridSearchCV y evaluación del modelo final en el conjunto de test	136
4.2.1.5	Comparación entre los resultados obtenidos en el Baseline Naive	139
4.2.2	Partición temporal del dataset de variaciones (Enfoque B)	139
4.2.2.1	Creación de X e y	140
4.2.2.2	Creación del baseline naive	141
4.2.2.3	Evaluación del baseline naive en dev y en test.....	142
4.2.2.4	Random Forest con partición temporal y optimización de hiperparámetros	143
4.2.2.5	Selección de los mejores hiperparámetros	145
4.2.2.6	Evaluar Random Forest en test	145
4.2.2.7	Comparación del enfoque B con el Baseline Naive	146
4.2.2.8	TimeSeriesSplit.....	147
4.2.2.9	GridSearchCV con TimeSeriesSplit	148
4.2.2.10	Obtener los mejores hiperparámetros.....	148
4.2.2.11	Obtener los mejores hiperparámetros.....	149
4.2.2.12	Comparación final del enfoque B	149
4.2.3	Comparación entre los enfoques A y B	150
4.2.4	Reconstrucción de las ventas mediante las variables predichas 151	

4.2.5	Evaluación de las ventas reconstruidas	153
4.2.6	Evaluación del Baseline reconstruido	153
4.2.7	Comparación gráfica de las predicciones	154
4.2.8	MAE y R^2	155
4.2.9	Conclusiones	156
4.3	Ridge Regression	156
4.3.1	Preparación de los datos	157
4.3.2	Creación de la partición temporal	157
4.3.3	Creación de X e y	158
4.3.4	Preparar Ridge Regression	158
4.3.5	Búsqueda del mejor valor de Alpha (α)	159
4.3.6	Entrenar el Ridge Regression definitivo	160
4.3.7	Realizar las predicciones sobre test	160
4.3.8	Cálculo de MSE y RMSE	161
4.3.9	Cálculo de MAE y R^2	161
4.3.10	Tabla comparativa de MAE, RMSE y R^2	162
4.4	Gradient Boosting	163
4.4.1	Pasos iniciales	163
4.4.2	Preparación de los datos para TimeSeriesSplit y creación del mismo	163
4.4.3	Creación del modelo Gradient Boosting	164
4.4.4	Definición y búsqueda de los mejores hiperparámetros	164
4.4.5	Obtención del modelo definitivo	166
4.4.6	Realizar las predicciones sobre el conjunto de test	166
4.4.7	Calcular MSE y RMSE	167
4.4.8	Calcular MAE y R^2	167
4.4.9	Tabla comparativa de MAE, RMSE y R^2	168
4.5	Baseline Naive	169
4.5.1	Pasos iniciales	169
4.5.2	Predicciones Naive	169
4.5.3	Predicciones sobre Dev	170

4.5.4	Predicciones sobre test	170
Capítulo 5: Comparación y evaluación de los modelos		172
5.	Comparación y evaluación de los modelos	173
5.1	Comparación de los diferentes modelos	173
Capítulo 6: Conclusiones y Trabajo Futuro		175
6.	Conclusiones y trabajo futuro	176
6.1	Conclusiones.....	176
6.2	Trabajo futuro.....	178
Bibliografía		181
Repositorio de GitHub con el código utilizado		181
Bibliografía del proyecto		181
Anexos.....		187

Índice de tablas

Tabla 1: Comparación de los modelos utilizados mediante diferentes métricas	63
Tabla 2: Reportes Anuales de Ventas de BMW	69
Tabla 3: Ventas de BMW por año en Europa	70
Tabla 4: Estadísticos relevantes	72
Tabla 5: Variación absoluta y porcentual interanual de las ventas de BMW	73
Tabla 6: Cobertura geográfica de las variables macroeconómicas	83
Tabla 7: Variables Macroeconómicas y sus valores	84
Tabla 8: Principales Estadísticos de las variables macroeconómicas	92
Tabla 9: VIF de las variables macro	104
Tabla 10: Unión de los dos conjuntos de datos	107
Tabla 11: Importancia de variables macroeconómicas	109
Tabla 12: Variables de mayor importancia	112
Tabla 13: Cuatro variables macroeconómicas para repetir el estudio	113
Tabla 14: Tabla de importancias de las variables macroeconómicas seleccionadas	114
Tabla 15: Comparación con el modelo completo anterior	115
Tabla 16: Optimización de hiperparámetros	120
Tabla 17: Cálculo del MSE en test	133
Tabla 18: Variación interanual de las variables de entrada	139
Tabla 19: Comparación Enfoque B con el Baseline Naive	147
Tabla 20: Cuatro bloques temporales con TimeSeriesSplit	147
Tabla 21: Comparación final del Enfoque B	150
Tabla 22: Comparación entre los enfoques A y B	151
Tabla 23: Predicciones en test	161
Tabla 24: Tabla comparativa MAE, RMSE y R^2	162
Tabla 25: Predicciones sobre el conjunto de test	167
Tabla 26: Tabla comparativa de MAE, RMSE y R^2	168
Tabla 27: Comparación de los diferentes modelos mediante las métricas MAE, RMSE y R^2	173
Tabla 28: Dataset de ventas de BMW en Europa	187
Tabla 29: Variables Macroeconómicas	187

Índice de gráficos e imágenes

Ilustración 1: Modelo de redes neuronales de McCulloch y Pitts	24
Ilustración 2: ENIAC de John Presper Eckert y John William Mauchly.....	24
Ilustración 3: SNARC (Stochastic Neural Analog Reinforcement Calculator) .	25
Ilustración 4: Perceptrón.....	26
Ilustración 5: Algoritmo Backpropagation	27
Ilustración 6: El Aprendizaje Automático como un área de la Inteligencia Artificial.....	29
Ilustración 7: Aprendizaje Supervisado	30
Ilustración 8: Aprendizaje no supervisado.....	31
Ilustración 9: Aprendizaje por refuerzo.....	32
Ilustración 10: Estructura de un árbol de decisión.....	34
Ilustración 11: Ejemplo de Árbol de regresión	38
Ilustración 12: Sobreajuste en la regresión	40
Ilustración 13: Subajuste en comparación a los resultados de un modelo ajustado y uno sobreajustado	40
Ilustración 14: Altura máxima de un árbol	41
Ilustración 15: Número mínimo de observaciones para dividir un nodo	42
Ilustración 16: Naive Baseline	45
Ilustración 17: Ridge Regression	47
Ilustración 18: Gradient Boosting	51
Ilustración 19: Random Forest Classifier.....	55
Ilustración 20: Vista aérea de Europa	66
Ilustración 21: Mapa de la Unión Europea	67
Ilustración 22: Eurozona	67
Ilustración 23: Dataset de ventas por modelo, año, color y región de Kaggle .	68
Ilustración 24: Cargando el archivo con las ventas de BMW	70
Ilustración 25: Análisis de la variación absoluta y porcentual	71
Ilustración 26: Evolución de las matriculaciones de BMW en Europa (2005-2025).....	73

Ilustración 27: Representación de la variación porcentual de las ventas de BMW	74
Ilustración 28: Tasa de Crecimiento Anual Compuesto (CAGR).....	75
Ilustración 29: Tasa de desempleo	76
Ilustración 30: Producto Interior Bruto	77
Ilustración 31: Índice Armonizado de Precios al Consumo (HICP)	78
Ilustración 32: Producción Industrial	78
Ilustración 33: Crecimiento de las ventas minoristas	79
Ilustración 34: Compensación por Empleado.....	80
Ilustración 35: Precio del Petróleo de Brent.....	80
Ilustración 36: Euríbor a 12 meses.....	81
Ilustración 37: Población Total	82
Ilustración 38: Tipo de Cambio EUR/USD	82
Ilustración 39: Carga de las variables macroeconómicas.....	84
Ilustración 40: Evolución de la Tasa de Desempleo	85
Ilustración 41: Evolución del Crecimiento del PIB	86
Ilustración 42: Evolución del Tipo de Depósito del BCE	86
Ilustración 43: Evolución del HICP	87
Ilustración 44: Evolución del Índice de Producción Industrial	87
Ilustración 45: Evolución de las ventas minoristas	88
Ilustración 46: Evolución del Índice de Confianza del Consumidor.....	88
Ilustración 47: Evolución de la Compensación por empleado	89
Ilustración 48: Evolución del Precio del petróleo de Brent.....	90
Ilustración 49: Evolución del Tipo de Cambio EUR/USD	90
Ilustración 50: Evolución del Euríbor a 12 meses	91
Ilustración 51: Evolución de la Población Total.....	91
Ilustración 52: Análisis de los estadísticos de las variables macroeconómicas	92
Ilustración 53: Cálculo de la matriz de correlación	95
Ilustración 54: Interpretación de la matriz de correlación	96
Ilustración 55: Uniendo los dos datasets.....	100
Ilustración 56: Calculando las correlaciones	101
Ilustración 57: Tabla de correlaciones entre las ventas de BMW y las variables macroeconómicas.....	101
Ilustración 58: Correlaciones de cada variable macro con las ventas de BMW	101
Ilustración 59: Cálculo del diagnóstico de multicolinealidad	103
Ilustración 60: Carga y unión de los datos	106
Ilustración 61: Unión de los dos conjuntos de datos.....	107
Ilustración 62: Definición de X e Y y sus dimensiones.....	108

Ilustración 63: Entrenamiento del RandomForestRegressor	108
Ilustración 64: Obtener y Ordenar la importancia de cada variable macroeconómica dentro del Random Forest	109
Ilustración 65: Importancia de las variables macroeconómicas	110
Ilustración 66: Construcción de matrices X e Y	113
Ilustración 67: Re-entrenando el modelo Random Forest	114
Ilustración 68: Importancia de las variables macroeconómicas seleccionadas	114
Ilustración 69: Función añadir_variables_macro	116
Ilustración 70: Creación de X e y	117
Ilustración 71: Partición aleatoria train / dev / test	118
Ilustración 72: Optimización de hiperparámetros	119
Ilustración 73: Selección de la combinación óptima	121
Ilustración 74: Heatmap del MSE obtenido sobre el conjunto de dev	122
Ilustración 75: Evaluación final en test	123
Ilustración 76: Comparación entre valores reales y predichos	125
Ilustración 77: Unión de los datos	129
Ilustración 78: Creación de X e y	129
Ilustración 79: Partición temporal de los datos	130
Ilustración 80: Creación del baseline naive	131
Ilustración 81: Evaluación del baseline naive	132
Ilustración 82: Random Forest con partición temporal	133
Ilustración 83: Evaluando Random Forest en test	134
Ilustración 84: Preparando los datos para TimeSeriesSplit	135
Ilustración 85: TimeSeriesSplit	136
Ilustración 86: Mejores hiperparámetros y RMSE medio de validación	137
Ilustración 87: Evaluando el último modelo en el test seleccionado	138
Ilustración 88: Comparación entre los resultados obtenidos del Baseline Naive y Random Forest	139
Ilustración 89: Tabla obtenida de 20 x 11	140
Ilustración 90: Partición enfoque B de los datos	140
Ilustración 91: Creación de X e y	141
Ilustración 92: Creación del baseline naive	142
Ilustración 93: Evaluación de Baseline Naive en dev	142
Ilustración 94: Evaluación del Baseline Naive en test	143
Ilustración 95: Optimización de hiperparámetros	144
Ilustración 96: Mejores combinaciones de n_estimators y min_impurity_decrease	144
Ilustración 97: Selección de los mejores hiperparámetros	145
Ilustración 98: Evaluando Random Forest en test	146

Ilustración 99: TimeSeriesSplit	147
Ilustración 100: Obtener los mejores hiperparámetros	148
Ilustración 101: Reconstrucción de las ventas mediante las variaciones predichas	152
Ilustración 102: Reconstrucción de las ventas mediante las variaciones predichas	152
Ilustración 103: Evaluación del Baseline Reconstruido	154
Ilustración 104: Comparación gráfica de las predicciones (Enfoque A)	154
Ilustración 105: Comparación gráfica de las predicciones (Enfoque B)	155
Ilustración 106: MAE, RMSE y R^2	156
Ilustración 107: Selección y unión de todas las variables del modelo	157
Ilustración 108: Partición temporal	158
Ilustración 109: Preparar Ridge Regression	159
Ilustración 110: Buscando el mejor valor de alpha	159
Ilustración 111: Entrenando el Ridge Regression definitivo	160
Ilustración 112: Realizar las predicciones sobre test	160
Ilustración 113: Cálculo MSE y RMSE	161
Ilustración 114: Cálculo MAE y R^2	162
Ilustración 115: Preparación de los datos para TimeSeriesSplit	163
Ilustración 116: Creación del TimeSplitSeries	164
Ilustración 117: Creación del modelo Gradient Boosting	164
Ilustración 118: Definición de los hiperparámetros	165
Ilustración 119: Búsqueda de los mejores hiperparámetros	165
Ilustración 120: Obtención del modelo definitivo de Gradient Boosting	166
Ilustración 121: Predicciones sobre el conjunto de test	166
Ilustración 122: Cálculo MSE y RMSE	167
Ilustración 123: Cálculo MAE y R^2	168
Ilustración 124: Predicciones Naive	169

Capítulo 1:

Objetivos, Motivación y Estructura del Documento

1. Objetivos, Motivación, Justificación y Secuenciación del TFG

La presente memoria tiene como principal finalidad, desarrollar un modelo de predicción de las ventas de vehículos de la firma alemana Bayerische Motoren Werke Aktiengesellschaft (BMW) mediante técnicas de Aprendizaje Automático, utilizando datos históricos de ventas y diferentes variables macroeconómicas.

Por ello, a lo largo de los siguientes apartados se detallarán y analizarán los distintos aspectos asociados al desarrollo del proyecto y a la aplicación de los modelos predictivos.

1.1 Objetivos

A continuación, se detallarán los objetivos que se persiguen con la realización del presente proyecto.

1.1.1 Objetivo general

El objetivo principal de este proyecto consiste en el desarrollo y la evaluación de modelos predictivos de las ventas de vehículos BMW en el mercado europeo mediante técnicas de Aprendizaje Automático, tomando como variable objetivo las matriculaciones anuales de la marca y complementándolas con diferentes variables macroeconómicas que podrían influir en mayor o menor medida en el comportamiento de la Demanda de vehículos y con ello a la predicción de ventas

De este modo, se pretende analizar la capacidad de diferentes técnicas de aprendizaje automático para predecir las ventas de BMW y determinar cuál de ellas presenta un mejor comportamiento predictivo.

Los resultados obtenidos se plantearán como una herramienta de apoyo a la toma de decisiones, proporcionando una estimación de la evolución futura de las ventas que pueda contribuir a la planificación comercial y estratégica de la empresa. Además, es parte de un Plan Estratégico desarrollado en un Trabajo de Fin de Grado complementario desarrollado por la misma autora para el Grado en Administración y Dirección de Empresas para completar la Doble Titulación cursada actualmente.

1.1.2 Objetivos secundarios

Los objetivos secundarios que se pretenden conseguir con la elaboración de esta memoria se encuentran a continuación.

En primer lugar, recopilar y preparar los datos históricos de matriculaciones de vehículos BMW en Europa desde 2005 hasta 2025, construyendo un conjunto de datos que permita analizar la evolución de las ventas durante el periodo estudiado y que pueda ser utilizado posteriormente para el desarrollo de los modelos predictivos.

La elección de este periodo se debe a que los datos de ventas de BMW se encuentran disponibles desde 2005 en los informes anuales de la compañía. Aunque sería posible obtener datos de ventas correspondientes a años anteriores, no sería posible ampliar de forma equivalente el periodo de estudio para las variables macroeconómicas que permitirán entrenar los diferentes modelos. Ya que existen algunas que presentan una disponibilidad temporal más limitada, como es el caso del tipo de cambio EUR/USD, por lo que se establece el periodo 2005-2025 para garantizar la disponibilidad y homogeneidad de los datos utilizados en el modelo.

A continuación, se pretende recopilar diferentes variables macroeconómicas relacionadas con la evolución de la economía europea y con la demanda de vehículos. Entre estas variables se encuentran indicadores como el crecimiento del PIB, la tasa de desempleo, la inflación, los tipos de interés, la producción industrial, las ventas minoristas, la confianza del consumidor, la remuneración por empleado, la evolución de la población y el precio del petróleo, además del tipo de cambio EUR/USD.

Una vez recopiladas las variables, se realizará un estudio de su importancia mediante técnicas de Aprendizaje Automático, utilizando un modelo Random Forest como herramienta de análisis. El objetivo será identificar aquellas variables macroeconómicas que presentan una mayor capacidad explicativa sobre la evolución de las ventas de BMW y reducir el conjunto inicial de variables a aquellas consideradas más relevantes. Posteriormente, se realizará una reevaluación utilizando únicamente las variables seleccionadas, con el objetivo de comprobar el comportamiento de estas variables y determinar el conjunto definitivo que será empleado en los modelos predictivos.

Una vez seleccionadas las variables, se integrarán los datos macroeconómicos con las matriculaciones de BMW mediante la construcción de un dataset definitivo, asociando a cada observación de ventas las variables macroeconómicas correspondientes al mismo año. Posteriormente, el conjunto de datos será dividido en train, dev y test, con el objetivo de disponer de conjuntos diferenciados para el entrenamiento, la optimización de los modelos y su evaluación final.

Posteriormente, a partir del conjunto de datos definitivo, se desarrollarán y evaluarán los diferentes modelos predictivos.

Finalmente, se pretende comparar los resultados obtenidos por los diferentes modelos mediante las métricas de evaluación seleccionadas, analizando sus ventajas, limitaciones y capacidad de generalización. De esta forma, se determinará qué técnica presenta un mejor comportamiento para la predicción de las ventas de BMW y se valorará su posible utilidad como herramienta de apoyo a la toma de decisiones de la empresa.

1.2 Motivación

La principal motivación para elaborar la presente memoria, correspondiente al Trabajo de Fin de Grado de la parte de Ingeniería Informática de la Doble Titulación en Administración y Dirección de Empresa de la EPSA, reside en el interés por culminar la formación universitaria aplicando todos los conocimientos adquiridos a lo largo del grado.

Además, debido al desarrollo y finalización de un Trabajo de Fin de Grado complementario para la titulación en Administración y Dirección de Empresas, surge la oportunidad de complementar los conocimientos adquiridos en ambas titulaciones mediante un proyecto común, abordando el análisis de la empresa BMW desde una perspectiva tecnológica y aplicando técnicas de Aprendizaje Automático a la predicción de sus ventas. Lo que permite establecer una conexión entre los conocimientos propios de la gestión empresarial y los relacionados con la Ingeniería Informática, dando lugar a un proyecto interdisciplinar recogido en dos trabajos de fin de grado.

Durante la búsqueda inicial de los datos necesarios para el desarrollo del proyecto, se localizaron diferentes conjuntos de datos relacionados con el sector de la automoción, entre ellos uno disponible en la plataforma Kaggle que contenía información sobre ventas de vehículos de segunda mano.

Sin embargo, dado que el objetivo del proyecto era trabajar con las matriculaciones de vehículos nuevos, se consideró necesario recurrir a fuentes oficiales. Por ello, finalmente se obtuvieron los datos directamente de los informes oficiales de BMW Group.

Por otro lado, la decisión de explorar el campo del Aprendizaje Automático surge del interés por ampliar los conocimientos adquiridos durante la formación en Ingeniería Informática.

Algunos de los conceptos relacionados con esta área fueron introducidos en la asignatura de Sistemas Inteligentes durante la carrera. Sin embargo, el plan de

estudios de la Doble Titulación en ADE e Ingeniería Informática no incluye una asignatura específica dedicada al Aprendizaje Automático.

Por este motivo, se consideró este proyecto como una oportunidad para comenzar a explorar de manera autónoma este campo dentro de la Inteligencia Artificial y aplicar sus técnicas a un problema real de predicción de ventas.

1.3 Estructura del documento

Con el fin de ofrecerle una forma estructurada a la memoria, ésta se estructurará en diferentes capítulos, en los cuales se desarrolla de forma progresiva el proceso seguido para la construcción, desarrollo y evaluación de los modelos predictivos de ventas de BMW. Dicho proceso será acompañado por su debido marco teórico, comparaciones y conclusiones detallados a continuación.

En el primer capítulo se presenta la introducción al proyecto, incluyendo la motivación que ha llevado a su realización, los objetivos principales que se pretenden alcanzar y la estructura seguida a lo largo de la memoria.

Posteriormente en el segundo capítulo, dedicado a los fundamentos del Aprendizaje Automático, se abordarán los principales conceptos teóricos necesarios para comprender el desarrollo del proyecto. Dichos conceptos incluyen la evolución histórica y la definición del Aprendizaje Automático, entre otros. Además, también se contemplan los modelos utilizados (Baseline Naive, Gradient Boosting, Ridge Regression y Random Forest), además de las principales métricas utilizadas para evaluar los modelos.

A continuación, el tercer capítulo se centrará en los datasets utilizados durante el proyecto. De los cuales, se describirá el conjunto de datos de ventas de BMW y las diferentes variables macroeconómicas consideradas. Además, se analizará la importancia de estas variables con el objetivo de seleccionar aquellas que presentan una mayor relevancia para el modelo. Finalmente, se explicará el proceso de integración de las variables macro seleccionadas con los datos de ventas de BMW y la posterior partición del conjunto de datos en los subconjuntos de entrenamiento, desarrollo y test.

A lo largo del cuarto capítulo, se desarrollarán los modelos predictivos empleados, el procedimiento seguido para la implementación y configuración de los diferentes modelos, así como los procesos de optimización correspondientes.

Posteriormente, en el quinto capítulo se recogerán los experimentos y la evaluación de los modelos desarrollados en función de los resultados obtenidos y la comparación entre los modelos y discutir cuál sería el más válido.

Por último, el sexto capítulo presentará las conclusiones obtenidas a partir del desarrollo del proyecto, así como las principales limitaciones encontradas y las posibles líneas de trabajo futuro que podrían plantearse para continuar y mejorar el estudio.

Capítulo 2:

Fundamentos del Aprendizaje Automático

2. Fundamentos del Aprendizaje Automático

Desde la máquina de vapor hasta un algoritmo capaz de derrotar al entonces campeón mundial de ajedrez Garry Kasparov en 1997. La humanidad ha experimentado numerosos avances tecnológicos que han dado lugar a profundas transformaciones en la sociedad y en la industria.

Dichos avances han marcado diferentes etapas de la evolución tecnológica y han conducido hasta el desarrollo de nuevas disciplinas, entre las cuales destaca el Aprendizaje Automático que es un campo de la Inteligencia Artificial. El cual puede considerarse una de las tecnologías con mayor potencial transformador de la actualidad. Dicho campo cobra especial importancia debido a su capacidad para ajustar modelos matemáticos a partir de datos observados, permitiendo identificar patrones y relaciones que pueden emplearse posteriormente para realizar predicciones y apoyar la toma de decisiones.

Por ello, su aplicación se ha extendido desde la informática a numerosos ámbitos, entre los cuales destaca el empresarial y económico, donde permite aprovechar la información disponible para analizar comportamientos y anticipar su evolución. A continuación, se explicará el contexto histórico, los tipos de aprendizajes, dentro de los cuales se destacará el aprendizaje supervisado, los modelos utilizados y sus métricas de evaluación.

2.1 Contexto histórico

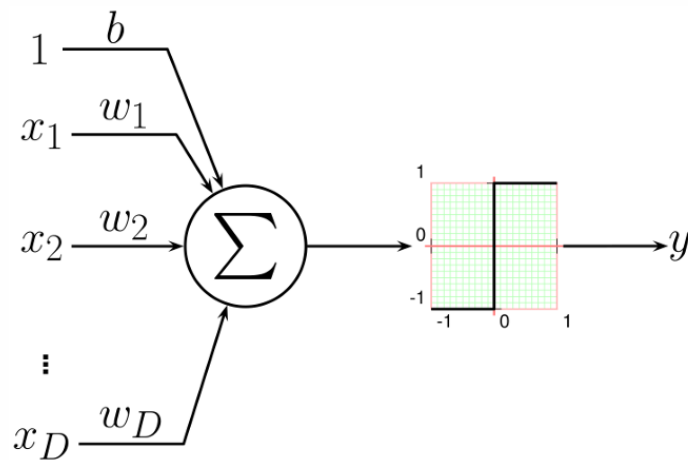
Dado que el Aprendizaje Automático se trata de un campo de la Inteligencia Artificial, conviene mencionar el origen de la misma y del Aprendizaje Automático y cómo se ha llegado al momento actual desde el s.XX.

2.1.1 Origen de la Inteligencia Artificial y del Aprendizaje Automático

Aunque la idea de construir máquinas capaces de realizar tareas asociadas tradicionalmente a la inteligencia humana se remonta a mucho antes de la aparición de los ordenadores modernos, fue durante el siglo XX cuando comenzaron a establecerse las bases que permitirían el desarrollo de la Inteligencia Artificial como campo de estudio.

Uno de los primeros antecedentes relevantes se encuentra en 1943 cuando Warren McCulloch y Walter Pitts publican el primer modelo computacional de red neuronal artificial. Lo que en décadas posteriores daría origen a las bases del Aprendizaje Profundo.

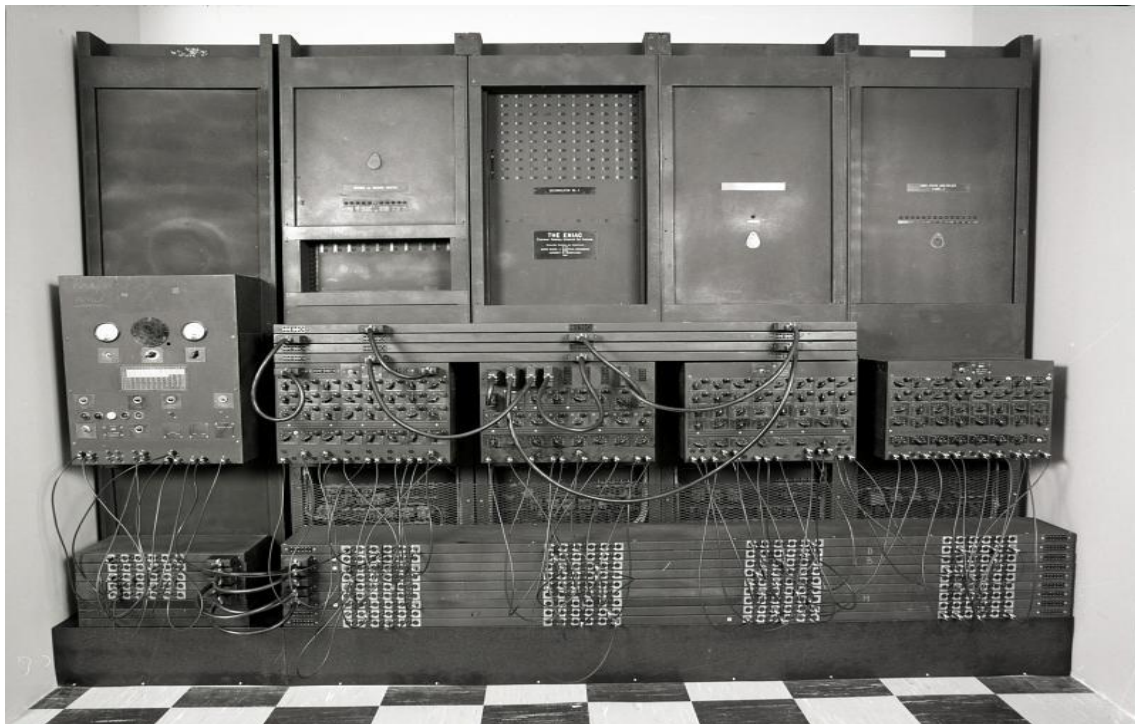
Ilustración 1: Modelo de redes neuronales de McCulloch y Pitts



Fuente: Intel·ligència Artificial: El motor de la quarta revolució industrial, Joan Albert Silvestre Cerdà

Poco después, en 1946 se produjo la puesta en funcionamiento del ENIAC o también conocido como Electronic Numerical Integrator and Computer. El cual es considerado uno de los primeros computadores electrónicos de propósito general que supuso un gran avance en la capacidad de cálculo disponible para el desarrollo de nuevos métodos computacionales.

Ilustración 2: ENIAC de John Presper Eckert y John William Mauchly

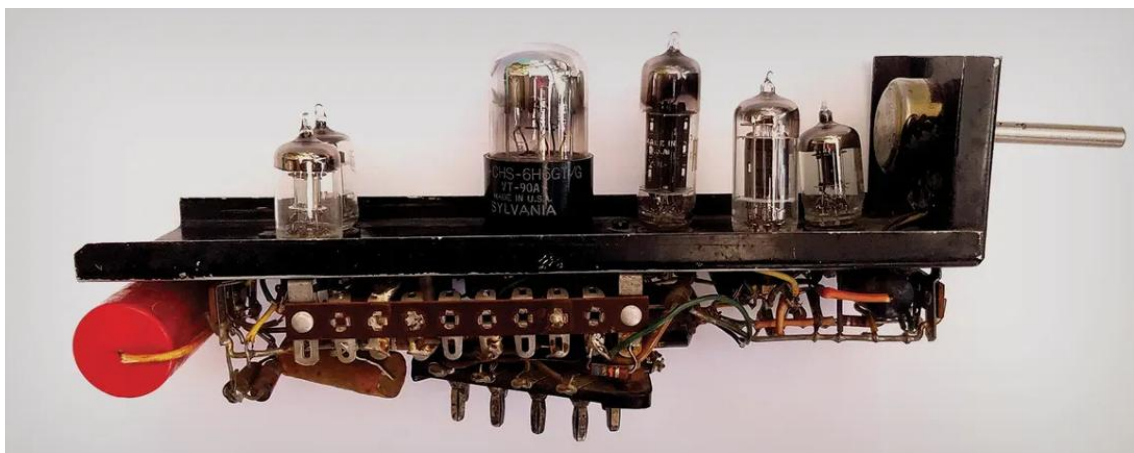


Fuente: *National Museum of American History*

En 1948, Norbert Wiener introdujo el concepto de cibernética, planteándola como el estudio de los sistemas de control y comunicación tanto en los animales como en las máquinas. Dicha disciplina tuvo una importante influencia en los primeros trabajos relacionados con las redes neuronales artificiales. En esta línea, tres años después, Marvin Minsky desarrolló SNARC (Stochastic Neural Analog Reinforcement Calculator), considerada una de las primeras implementaciones de una red neuronal artificial.

Dicho sistema estaba compuesto por 40 neuronas artificiales analógicas y utilizaba mecanismos de refuerzo para modificar su comportamiento.

Ilustración 3: *SNARC (Stochastic Neural Analog Reinforcement Calculator)*



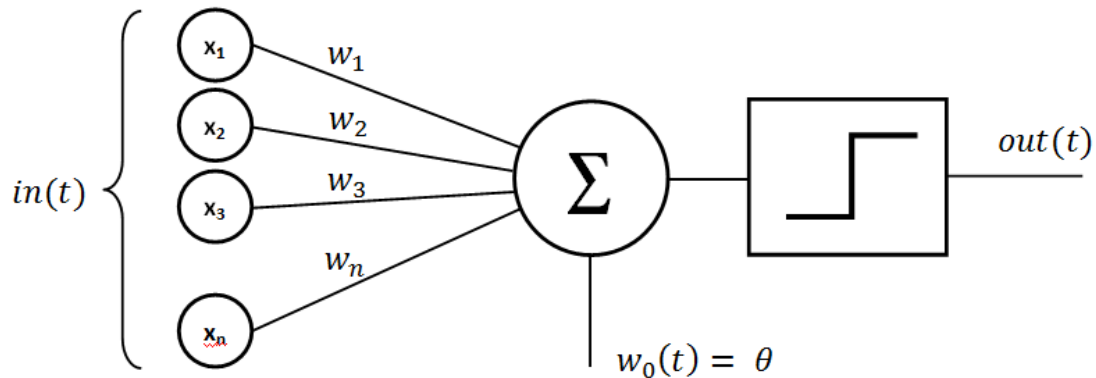
Fuente: *Intel·ligència Artificial: El motor de la quarta revolució industrial, Joan Albert Silvestre Cerdà*

Estos primeros trabajos dieron paso a una etapa de experimentación en la que comenzaron a desarrollarse modelos computacionales capaces de aprender a partir de ejemplos. Dándose en 1954 algunas de las primeras simulaciones digitales de redes neuronales por Belmont Farley y Wesley Clark. Los cuales trasladaron progresivamente estos conceptos desde implementaciones físicas hacia sistemas digitales.

Posteriormente, 1956, tuvo lugar uno de los acontecimientos fundamentales en la historia de la disciplina. El cual se trataba del Dartmouth Summer Research Project. Donde John McCarthy propuso formalmente el término Artificial Intelligence para denominar el nuevo campo de investigación dedicado a estudiar cómo conseguir que las máquinas realizaran tareas que deberían de requerir inteligencia por parte de una máquina. Este acontecimiento suele considerarse uno de los puntos de partida de la Inteligencia Artificial como disciplina científica diferenciada.

Durante este mismo periodo comenzaron a desarrollarse técnicas específicamente orientadas al aprendizaje automático. Como el Perceptrón de Frank Rosenblatt. El cual es un algoritmo capaz de aprender los parámetros necesarios para clasificar determinados patrones a partir de datos de entrenamiento y constituye uno de los antecedentes directos de las redes neuronales modernas. Y posteriormente evolucionaría hacia arquitecturas mucho más complejas, como el Perceptrón Multicapa.

Ilustración 4: Perceptrón



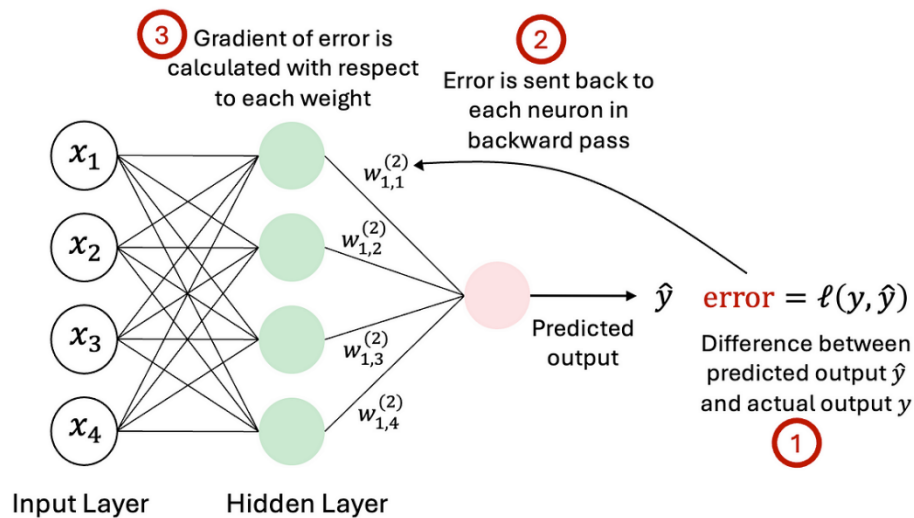
Fuente: Liora

Posteriormente, Arthur Samuel utilizó en 1959 el término de Aprendizaje Automático para describir un sistema desarrollado para jugar a las damas que podía mejorar su rendimiento a partir de la experiencia previa. Este hecho supuso un paso importante hacia una concepción de la Inteligencia Artificial basada no únicamente en la programación explícita de reglas, sino también en la capacidad de los sistemas para aprender a partir de los datos.

Durante las décadas posteriores se produjeron nuevos avances tanto en las redes neuronales como en otros enfoques de la Inteligencia Artificial. Así es tanto que en 1982, John Hopfield propuso las redes neuronales recurrentes conocidas como redes de Hopfield.

Después de varios avances llega el algoritmo de Retropropagación o Backpropagation como método para entrenar redes neuronales multicapa por parte de David Rumelhart y Geoffrey Hinton y de otros investigadores. Dicho procedimiento permitió ajustar de los pesos de las diferentes capas de una red neuronal mediante el cálculo del error cometido por el modelo, constituyendo uno de los fundamentos del posterior desarrollo del Deep Learning.

Ilustración 5: Algoritmo Backpropagation



Fuente: Medium

Posteriormente, en 1989, Yann LeCun propuso una de las primeras aplicaciones de las redes neuronales convolucionales (Convolutional Neural Networks, CNN) al reconocimiento de caracteres manuscritos. Lo que demostraba la capacidad de estos algoritmos de trabajar con datos estructurados.

La evolución de la Inteligencia Artificial también estuvo marcada por sistemas capaces de alcanzar resultados comparables o superiores a los humanos en tareas concretas. Como la victoria del Deep Blue de IBM que pudo derrotar al campeón mundial de ajedrez Garry Kasparov en un enfrentamiento a seis partidas en 1997.

Finalmente, a comienzos del siglo XXI, el desarrollo de la Inteligencia Artificial comenzó a verse impulsado por dos factores fundamentales como el incremento de la cantidad de datos disponibles y el aumento de la capacidad de cálculo.

En 2007, NVIDIA introdujo CUDA (Compute Unified Device Architecture) que se trataba de una plataforma que permitió utilizar las unidades de procesamiento gráfico (GPUs) para realizar cálculos de propósito general. Dicha tecnología resultó especialmente relevante para el entrenamiento de redes neuronales, ya que permitía realizar grandes cantidades de operaciones matemáticas de manera paralela.

Posteriormente, en 2009 se demostró el potencial de utilizar la GPU para acelerar considerablemente el entrenamiento de redes neuronales con grandes cantidades de datos, contribuyendo al desarrollo de una nueva etapa de la Inteligencia Artificial.

Uno de los hitos más importantes de esta nueva etapa tuvo lugar en 2012, cuando Alex Krizhevsky, Ilya Sutskever y Geoffrey Hinton desarrollaron AlexNet que se trataba de una red neuronal convolucional profunda entrenada mediante GPU que obtuvo grandes resultados en la competición ImageNet de reconocimiento de imágenes. Dicho resultado supuso un importante punto de inflexión para el desarrollo del Deep Learning. Ya que demostraba de manera práctica, las posibilidades que ofrecían las redes neuronales profundas cuando se combinaban con grandes cantidades de datos y una elevada capacidad de cálculo.

La evolución posterior estuvo acompañada por la aparición de herramientas que facilitaron el desarrollo y la utilización de estos modelos. En 2015, Google publicó TensorFlow que se trata de una biblioteca de código abierto destinada al diseño, entrenamiento y ejecución de modelos de Aprendizaje Automático y redes neuronales.

Y posteriormente aparecieron en 2017 apareció PyTorch y se publicó “Attention Is All You Need” por parte de Google. El cual introdujo la arquitectura Transformer. Dicha arquitectura fue inicialmente planteada para tareas de traducción automática, pero posteriormente se convirtió en una de las bases tecnológicas de los actuales modelos de lenguaje de gran escala.

A partir de esta arquitectura se produjo la evolución de los grandes modelos de lenguaje también conocidos como Large Language Models o LLM.

En 2018, OpenAI presentó GPT-1, basado en la arquitectura Transformer y orientado al procesamiento y generación de lenguaje natural. Dicha línea de investigación evolucionó posteriormente hacia modelos de mayor escala y capacidad. Hasta que en 2022, se presentó ChatGPT, un sistema conversacional basado inicialmente en la familia GPT, que supuso un importante punto de inflexión en la popularización de la Inteligencia Artificial entre el público general. Finalmente, en 2024 se lanzó GPT-4º. El cual se trataba de un modelo con capacidades multimodales.

En conjunto, esta evolución muestra cómo la Inteligencia Artificial ha pasado de los primeros modelos matemáticos de neuronas y sistemas basados en reglas a modelos capaces de aprender patrones complejos a partir de grandes cantidades de datos como son los que se pueden encontrar hoy en día.

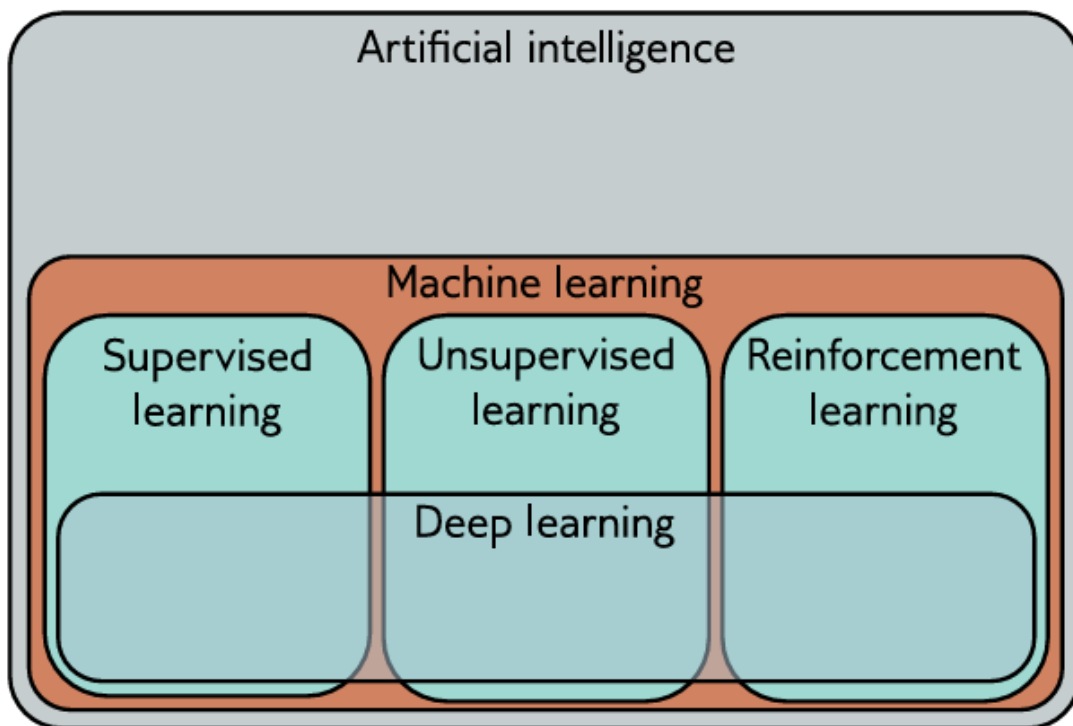
2.1.2 El Aprendizaje Automático y sus tipos

El Aprendizaje Automático es un campo ubicado dentro de la Inteligencia Artificial. El cual está centrado en el desarrollo de sistemas capaces de aprender a partir de datos observados.

Para ello, se ajustan modelos matemáticos a los datos disponibles con el objetivo de identificar patrones y relaciones que permitan posteriormente tomar decisiones o realizar predicciones sobre nuevas observaciones.

Los métodos de pueden clasificarse principalment Aprendizaje Automático en tres tipos, los cuales son el aprendizaje supervisado y el no supervisado y el aprendizaje por refuerzo. Esta diferenciación se debe a la forma en la que el sistema aprende a partir de la información disponible.

Ilustración 6: El Aprendizaje Automático como un área de la Inteligencia Artificial



Fuente: Understanding Deep Learning, Simon J.D. Prince

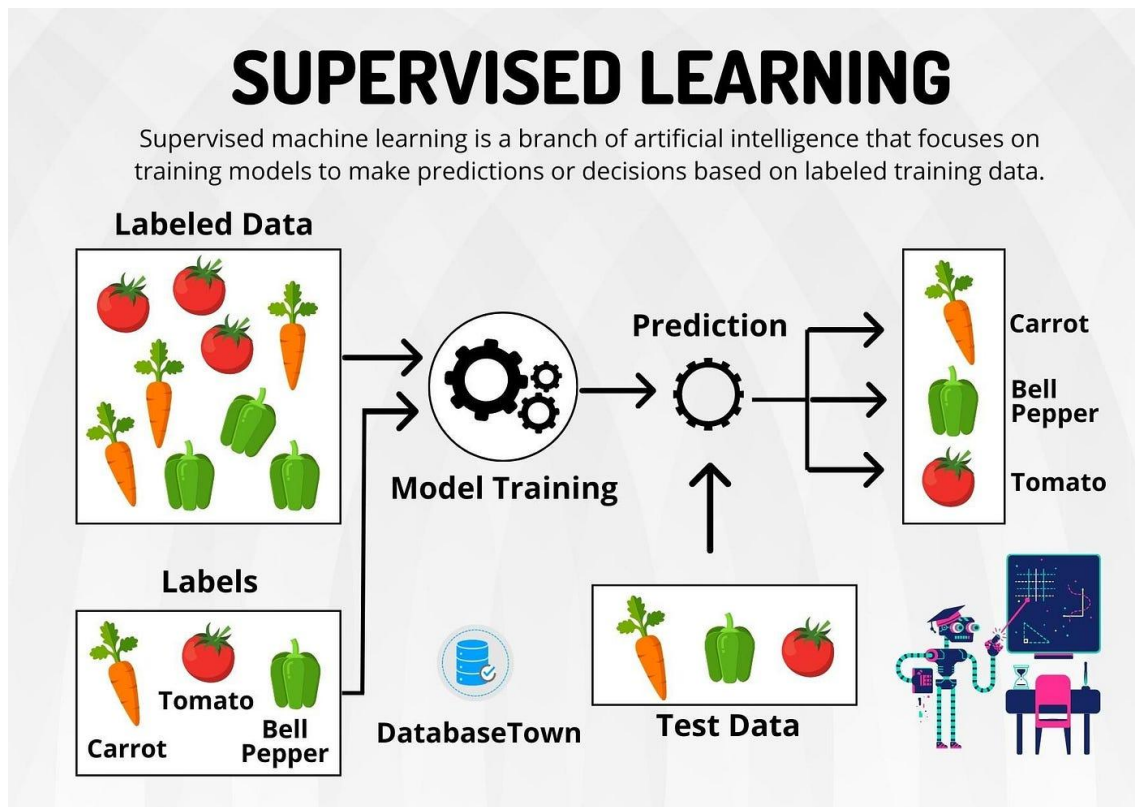
Estas tres áreas descritas, presentan las siguientes características:

- **Aprendizaje supervisado:** comprende aquellos métodos en los que el modelo aprende una correspondencia entre unos datos de entrada y una salida conocida.

Para ello, el conjunto de entrenamiento está formado por observaciones en las que se dispone tanto de las variables de entrada como del resultado asociado, de manera que estas parejas actúan como una referencia durante el proceso de aprendizaje.

El objetivo final consiste en encontrar una relación matemática que describa adecuadamente los datos de entrenamiento y que posteriormente permita realizar predicciones sobre nuevas observaciones. Dentro de esta categoría pueden distinguirse los problemas de regresión y los problemas de clasificación.

Ilustración 7: Aprendizaje Supervisado

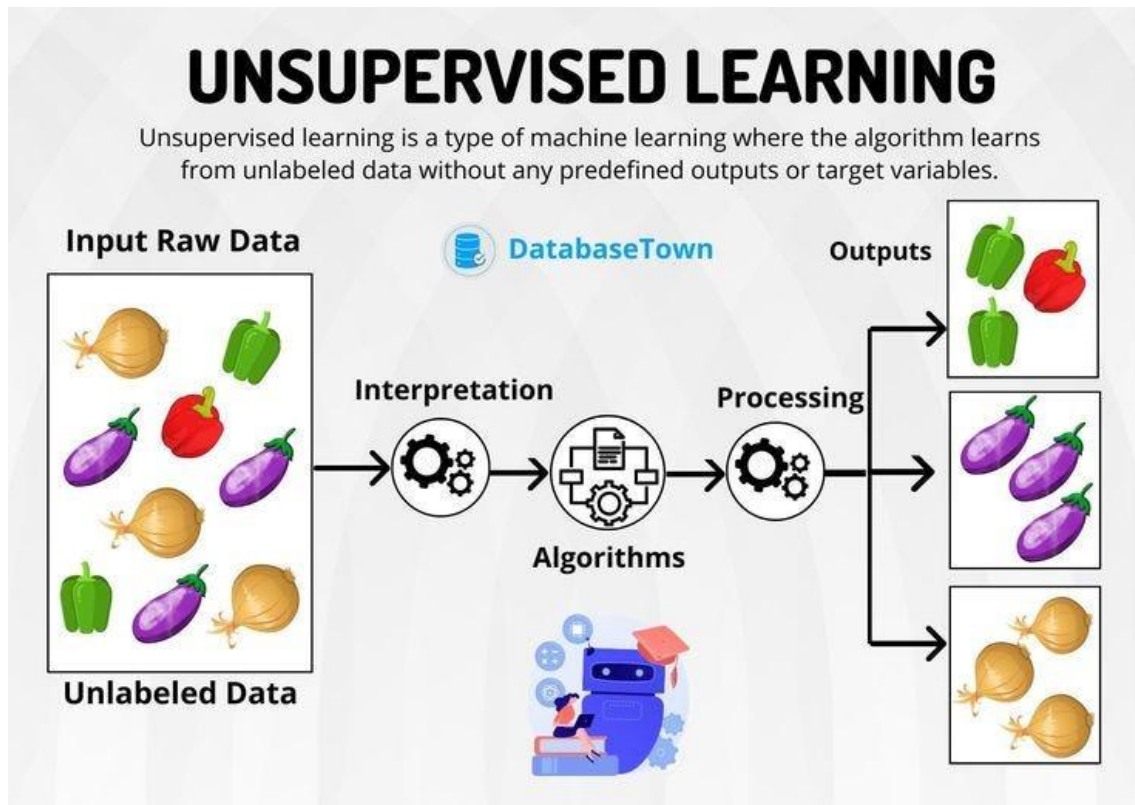


Fuente: Medium

- **Aprendizaje no supervisado:** en esta categoría, el modelo se construye únicamente a partir de los datos de entrada, sin disponer de una salida o etiqueta conocida que sirva como referencia durante el entrenamiento. Por tanto, en lugar de aprender directamente una relación entre entradas y salidas, el objetivo es describir o identificar la estructura de los propios datos.

Dentro de este enfoque se encuentran, los modelos generativos. Los cuales tratan de aprender las características estadísticas de los datos de entrenamiento para poder generar nuevas observaciones que presenten propiedades similares.

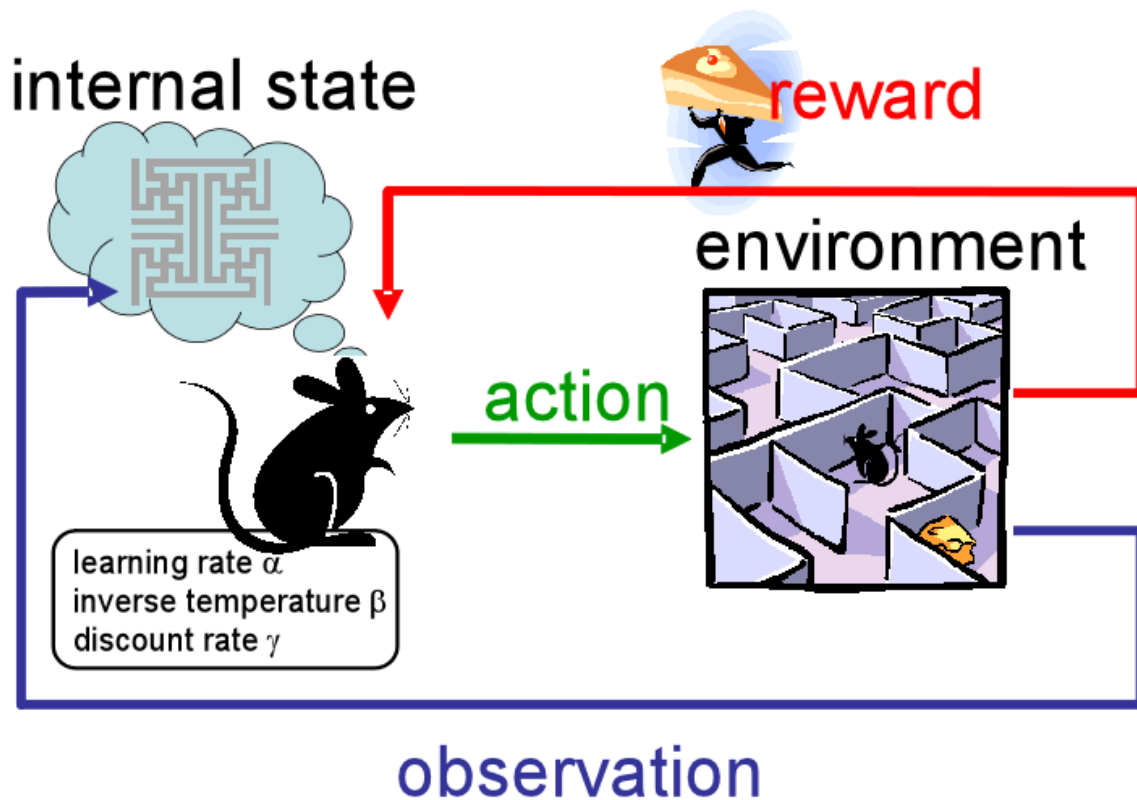
Ilustración 8: Aprendizaje no supervisado



Fuente: Medium

- **Aprendizaje por refuerzo:** éste un escenario en el que un agente interactúa con un determinado entorno y puede realizar diferentes acciones en cada instante. Estas acciones modifican el estado del entorno y pueden producir una recompensa, cuyo valor proporciona información sobre la conveniencia de la acción realizada. El objetivo del agente es aprender progresivamente qué acciones debe seleccionar para obtener recompensas elevadas a lo largo del tiempo. Este proceso presenta características particulares, ya que la recompensa asociada a una acción puede recibirse posteriormente, dificultando determinar qué acciones concretas han contribuido al resultado obtenido. Además, el agente debe equilibrar la exploración de nuevas acciones con la explotación de aquellas estrategias que ya sabe que proporcionan resultados satisfactorios.

Ilustración 9: Aprendizaje por refuerzo



Fuente: Glasswing ventures

Si bien los tres enfoques presentan aplicaciones relevantes, para la elaboración de este Trabajo de Fin de Grado se empleará el aprendizaje supervisado. Ya que se dispone de una variable objetivo conocida que se pretende predecir (matriculaciones de BMW mediante pares de entrada y salida, representados habitualmente como X e Y, respectivamente. Siendo los de entrada X, las diferentes variables macroeconómicas seleccionadas. Mientras que Y corresponderá a las matriculaciones anuales de vehículos BMW en el mercado europeo. De este modo, el modelo podrá aprender la relación existente entre las variables macroeconómicas y las ventas observadas durante el periodo de estudio, para posteriormente utilizar dicha relación en la predicción de nuevas observaciones.

2.1.3 Aprendizaje Supervisado

En la programación tradicional, el programador es capaz de establecer explícitamente aquellas reglas y condiciones que debe seguir un programa para obtener el resultado esperado. Aunque este enfoque continúa siendo especialmente eficaz para una gran variedad de problemas en los que las reglas pueden definirse de forma clara, existen problemas en los que resulta extremadamente complejo establecer manualmente todas las condiciones necesarias para obtener una predicción adecuada. Por ello es por lo que, en estos

casos, el Aprendizaje Automático permite abordar el problema mediante modelos capaces de aprender patrones a partir de los datos.

Dentro de este campo, el Aprendizaje Supervisado, constituye una de sus principales ramas y se basa precisamente en este principio. Ya que en lugar de definir de antemano todas las reglas que relacionan todas las variables de entrada con el resultado, se proporciona al modelo un conjunto de observaciones para las que se conocen tanto las variables de entrada X como el resultado correspondiente Y . De este modo, a partir de estos ejemplos, el modelo trata de identificar la relación existente entre ambas y ajusta sus parámetros para conseguir realizar predicciones lo más próximas posible a los valores reales.

Dentro de éste, pueden distinguirse dos grandes categorías resumidas en los problemas de Regresión y los problemas de Clasificación. Esta distinción es fundamental, ya que determina tanto el tipo de modelo que puede utilizarse como las métricas empleadas para evaluar su rendimiento.

En lo que respecta a los problemas de Clasificación, la variable objetivo representa una categoría o clase perteneciente a un conjunto discreto de posibilidades. Entonces, el objetivo final de este modelo será asignar cada observación a una de estas categorías predefinidas.

Por otro lado, en los problemas de Regresión, la variable objetivo es cuantitativa y puede tomar valores dentro de un intervalo continuo. De esta manera, el objetivo final será predecir un valor numérico a partir de las variables de entrada

Dado que el problema que se aborda en este trabajo trata la predicción de ventas, se utilizará la Regresión. Este enfoque resulta especialmente adecuado debido a que se dispone de datos históricos de las matriculaciones anuales de BMW y de diferentes variables macroeconómicas correspondientes a cada año. Donde las variables macroeconómicas constituyen las variables de entrada X , mientras que las matriculaciones anuales de BMW representan la variable objetivo Y . Lo que permite al modelo poder aprender de las relaciones observadas históricamente entre la evolución del entorno macroeconómico y el comportamiento de las ventas, para posteriormente generar predicciones sobre datos no utilizados durante su entrenamiento.

2.2 Árboles de decisión

Los árboles de decisión constituyen uno de los algoritmos más intuitivos y ampliamente utilizados en el ámbito del aprendizaje automático. Éstos destacan debido a su capacidad para representar reglas de decisión de manera jerárquica y

fácil de comprender. Lo que los convierte en una herramienta especialmente útil tanto para problemas de clasificación como de regresión.

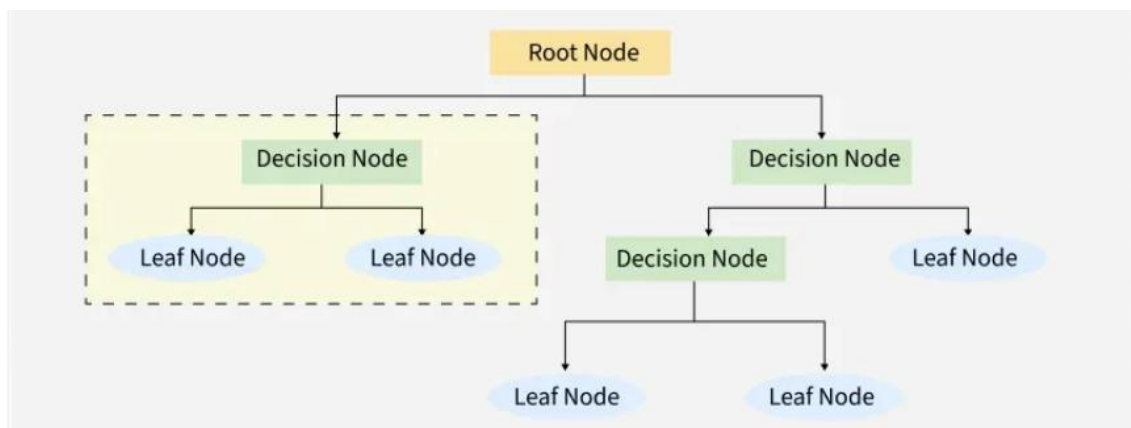
A lo largo de este apartado, se presentarán sus principales fundamentos, funcionamiento y aplicación. Asimismo, se abordará su extensión mediante el algoritmo Random Forest, basado en la combinación de múltiples árboles de decisión.

2.2.1 Concepto y estructura

Los árboles de decisión son modelos de Aprendizaje Automático que representan reglas de decisión de manera jerárquica y estructurada. Su nombre proviene de la analogía de un árbol invertido. Pues la información fluye desde un único nodo inicial hacia múltiples nodos terminales a través de una serie de ramificaciones. Esta estructura permite dividir el espacio de las variables de entrada en regiones cada vez más específicas, asignando una predicción diferente a cada una de ellas.

Por tanto, su estructura está compuesta por cuatro elementos fundamentales. Los cuales se componen del nodo raíz, el cual constituye el punto de partida del árbol y representa la primera condición que debe evaluarse, los nodos internos, que representan decisiones intermedias basadas en condiciones sobre las variables de entrada, las ramas conectan los nodos entre sí y representan los posibles resultados de cada condición y finalmente, los nodos hoja u hojas, que representan las predicciones finales del modelo.

Ilustración 10: Estructura de un árbol de decisión



Fuente: Medium

Como se puede observar en la imagen superior, el nodo raíz es el único nodo del árbol que no tiene nodos padres y representa la primera división que se aplica al conjunto completo de datos de entrenamiento. La selección de la variable y el umbral utilizados en el nodo raíz es especialmente importante, ya que determina cómo se distribuyen las observaciones en las primeras etapas del árbol. En

problemas de regresión, el nodo raíz suele seleccionar la división que produce la mayor reducción en la variabilidad de la variable objetivo.

Por otro lado, los nodos internos son aquellos que tienen tanto un nodo padre como al menos un nodo hijo. Cada nodo interno representa una condición sobre una de las variables de entrada. En el caso de variables numéricas, como las variables macroeconómicas utilizadas en este trabajo, la condición suele tomar la forma de una comparación con un umbral determinado. Y así, cada nodo interno divide las observaciones que llegan hasta él en dos o más subconjuntos, que se dirigen hacia diferentes nodos hijos.

Las ramas son las conexiones entre los nodos del árbol. Cada una de estas ramas representa un posible resultado de la condición evaluada en un nodo interno. En los árboles binarios, que son los más habituales en implementaciones como la de scikit-learn, cada nodo interno tiene exactamente dos ramas. Siendo la primera para las observaciones que cumplen la condición y la segunda para las que no la cumplen. Estas ramas no contienen información adicional por sí mismas, sino que simplemente dirigen el flujo de las observaciones hacia los nodos correspondientes.

Los nodos hoja u hojas, son los nodos terminales del árbol ya que no tienen nodos hijos. Y, por tanto, representan las predicciones finales del modelo. En problemas de clasificación, cada hoja asigna una clase específica a las observaciones que llegan hasta ella. Mientras que en problemas de regresión, como es el que se aborda en este estudio, cada hoja proporciona un valor numérico. Este valor se calcula habitualmente como la media de las observaciones de entrenamiento que llegaron a esa hoja durante la construcción del árbol. De este modo, el árbol aproxima la relación entre las variables de entrada y la variable objetivo mediante una función escalonada, donde cada región del espacio de variables tiene asociado un valor constante. El número de hojas, determina el número de regiones distintas en las que se divide el espacio de las variables. Cada hoja representa una combinación específica de condiciones sobre las variables de entrada. De esta manera, las observaciones que cumplen las mismas condiciones llegan a la misma hoja y reciben la misma predicción. Por tanto, el número de hojas controla la granularidad de las predicciones del modelo.

Por otro lado, existe un concepto conocido como la profundidad del árbol. El cual, se define como la longitud del camino más largo desde la raíz hasta una hoja. Un árbol con una única división tiene profundidad 1, mientras que un árbol con múltiples niveles de divisiones sucesivas tiene mayor profundidad. La profundidad está directamente relacionada con la complejidad del modelo. Por ello, árboles muy profundos pueden capturar relaciones complejas, aunque también pueden

presentar un mayor riesgo de sobreajuste. Árboles poco profundos son más simples y generalizan mejor, pero pueden no capturar patrones importantes en los datos.

Es importante destacar que la estructura del árbol se aprende automáticamente a partir de los datos de entrenamiento. El algoritmo determina cuáles son las variables más informativas para realizar las divisiones, en qué orden deben evaluarse y qué umbrales deben utilizarse. Esta capacidad de aprendizaje automático diferencia a los árboles de decisión de los sistemas de reglas manuales, donde las condiciones deben ser especificadas explícitamente por un experto.

2.2.2 Funcionamiento de los árboles de decisión

El funcionamiento de los árboles de decisión se basa en un proceso sistemático de particionamiento del espacio de las variables de entrada. Este proceso puede dividirse en dos fases principales, la construcción del árbol a partir de los datos de entrenamiento y la utilización del mismo, para generar predicciones sobre nuevas observaciones. Dichas fases se describirán a continuación.

2.2.2.1 Construcción del árbol

La construcción del árbol sigue un enfoque de división recursiva binaria. Para ello, el algoritmo comienza considerando el conjunto completo de datos de entrenamiento en el nodo raíz. En este punto inicial, todas las observaciones se encuentran agrupadas en una única región del espacio de variables para posteriormente dividir este conjunto en subconjuntos cada vez más homogéneos en términos de la variable objetivo.

En cada uno de los pasos de este proceso, el algoritmo evalúa todas las divisiones posibles y selecciona aquella que optimice un criterio de división predefinido. Para variables numéricas, como las variables macroeconómicas utilizadas en este trabajo, el algoritmo considera divisiones del tipo “*variable menor o igual que umbral*” frente a “*variable mayor que umbral*”. El algoritmo evalúa múltiples umbrales potenciales para cada variable y selecciona aquella combinación de variable y umbral que produzca la mejor división según el criterio establecido.

En problemas de regresión, el criterio de división más habitual es la reducción del error cuadrático o de la varianza dentro de los subconjuntos resultantes. Formalmente, el algoritmo busca la división que minimice la suma de los errores cuadráticos en los nodos hijos. Para cada división candidata, se calcula la media de la variable objetivo en cada subconjunto resultante y se evalúa cuánto se reducen las desviaciones respecto a esas medias en comparación con el nodo

padre. Y finalmente, la división que produce la mayor reducción del error será la que se seleccionará para realizar la partición.

Una vez llevada a cabo esta división, el proceso se repite de manera independiente en cada uno de los subconjuntos resultantes. Y así, cada uno de estos subconjuntos, se convierte en un nuevo nodo del árbol y el algoritmo buscará la mejor división para ese nodo específico. Este procedimiento continúa de forma recursiva, creando nuevos nodos internos y nuevas ramas en cada iteración. Lo que hace que el árbol crezca progresivamente, dividiendo el espacio de variables en regiones cada vez más específicas.

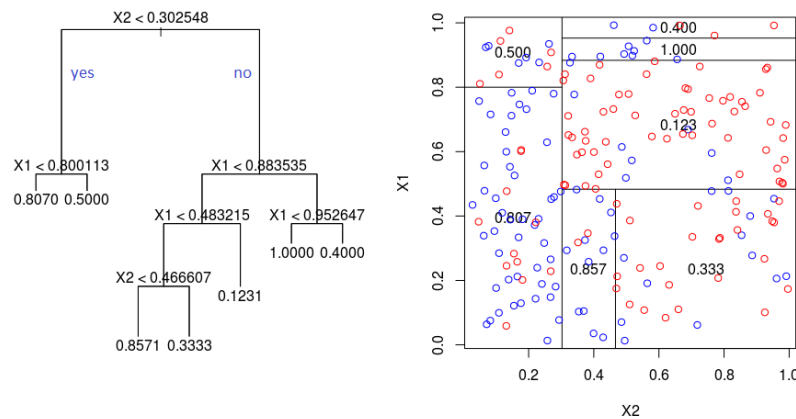
El proceso de construcción se terminará deteniendo cuando se cumpla alguna condición de parada. Entre las condiciones más habituales se encuentran alcanzar una profundidad máxima predefinida, obtener un número insuficiente de observaciones en un nodo para permitir una nueva división, lograr una mejora mínima en el criterio de división o alcanzar un número mínimo de observaciones en las hojas. Esto evitará que el árbol crezca de manera ilimitada y se ajuste excesivamente a los datos de entrenamiento.

Es importante recalcar que el algoritmo de construcción tiene un enfoque codicioso. Pues en cada paso, selecciona la mejor división local sin considerar las consecuencias futuras de esa decisión. Lo que significa que el árbol resultante puede no ser globalmente óptimo. No obstante, este enfoque es computacionalmente más eficiente, ya que evita tener que evaluar todos los árboles posibles y en la práctica, los árboles construidos mediante este método suelen ofrecer un rendimiento satisfactorio.

2.2.2.2 Predicción mediante el árbol

Una vez construido el árbol, éste se utilizará para generar predicciones sobre nuevas observaciones. El proceso comenzará en el nodo raíz. Donde el modelo evaluará la condición asociada a ese nodo utilizando los valores de las variables de entrada de la nueva observación. Y según el resultado de la evaluación, la observación se dirige hacia una de las ramas disponibles. Y este proceso se repite en cada nodo interno hasta alcanzar una hoja donde el proceso de predicción finaliza. Esta hoja, contendrá el valor predicho para esa observación. Y en problemas de regresión, este valor corresponderá a la media de las observaciones de entrenamiento que llegaron a la misma hoja durante la construcción del árbol.

Ilustración 11: Ejemplo de Árbol de regresión



Fuente: Medium

De este modo, todas las observaciones que llegan a una hoja determinada reciben la misma predicción, independientemente de sus valores específicos dentro de la región definida por esa hoja.

2.2.2.3 Funcionamiento en problemas de regresión

En el contexto de este trabajo, se tratarán principalmente los árboles de regresión debido a la naturaleza del problema que se pretende resolver.

Durante la construcción del árbol, cada hoja almacena la media de los valores de la variable objetivo correspondientes a las observaciones de entrenamiento que llegan hasta ella. Por ejemplo, si durante la construcción del árbol llegan a una hoja específica 15 observaciones con matriculaciones de BMW que van desde 700.000 hasta 850.000 vehículos, la hoja almacenará la media de esos valores, por ejemplo 775.000 vehículos. De este modo, cuando una nueva observación llega a esa hoja durante la fase de predicción, recibe como predicción el valor almacenado en la hoja. Esto significa que el árbol aproxima la función subyacente mediante una superficie escalonada, donde cada región del espacio de variables tiene asociado un valor constante y las fronteras entre regiones están definidas por las divisiones realizadas en los nodos internos.

Esta característica tiene implicaciones importantes para la capacidad predictiva del modelo. Por una parte, permite capturar relaciones no lineales entre las variables de entrada y la variable objetivo. Por otra parte, implica que las predicciones son constantes dentro de cada región, lo que puede resultar en una aproximación menos suave que la de otros modelos. Además, el árbol no puede extrapolar valores fuera del rango observado en los datos de entrenamiento, ya que las hojas contienen valores calculados a partir de observaciones existentes.

2.2.3 Complejidad sobreajuste y poda

La capacidad de un árbol de decisión para representar relaciones complejas entre las variables de entrada y la variable objetivo está directamente relacionada con su complejidad estructural. Por tanto, un árbol muy profundo, con múltiples niveles de división y un número elevado de hojas, puede capturar patrones detallados y no lineales en los datos. No obstante, esta mayor capacidad de representación conlleva riesgos importantes que deben ser gestionados adecuadamente para garantizar que el modelo generalice correctamente a nuevas observaciones. Lo cual será tratado a lo largo de todo este apartado.

2.2.3.1 Complejidad del árbol

La complejidad de un árbol de decisión puede medirse mediante diferentes características estructurales. Entre ellas, su profundidad, la cual representa la longitud del camino más largo desde la raíz hasta una hoja.

Por otro lado, el número total de hojas determina cuántas regiones distintas se establecen en el espacio de las variables. Ya que cada hoja representa una combinación específica de condiciones sobre las variables de entrada y todas las observaciones que llegan a una hoja determinada reciben la misma predicción.

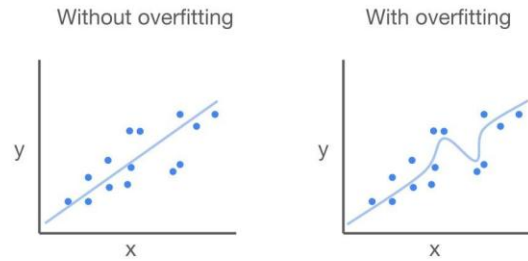
Un árbol con mayor profundidad y más hojas, puede representar funciones más elaboradas y capturar interacciones entre múltiples variables. Sin embargo, esta flexibilidad tiene un coste. Ya que, a medida que el árbol se vuelve más profundo, las divisiones sucesivas se basan en subconjuntos de datos cada vez más reducidos. Lo que puede llevar a que las decisiones tomadas en los niveles más profundos del árbol se basen en muy pocas observaciones y puede llevar a que el modelo se sobreajuste debido a que capture ruido o particularidades específicas de la muestra de entrenamiento en lugar de patrones generales.

2.2.3.2 Sobreajuste

Al igual que se estaba anticipando en el apartado anterior, el sobreajuste constituye uno de los principales problemas asociados con los árboles de decisión.

Dicho fenómeno se produce cuando el modelo se ajusta excesivamente a los datos de entrenamiento, capturando no solo los patrones generales subyacentes, sino también el ruido, las fluctuaciones aleatorias y las particularidades específicas de la muestra utilizada para su construcción. Lo que tiene como consecuencia que el árbol pueda mostrar un rendimiento excelente sobre los datos de entrenamiento, pero generalizar mal a nuevas observaciones no vistas durante el entrenamiento.

Ilustración 12: Sobreajuste en la regresión



Fuente: Crunching the Data

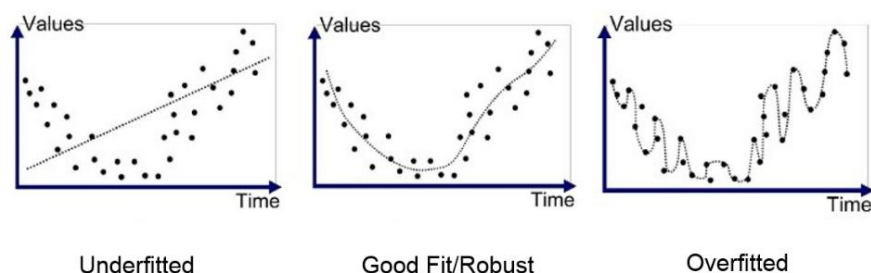
En el contexto de este trabajo, un árbol sobreajustado podría, por ejemplo, aprender que determinados valores muy específicos de las variables macroeconómicas están asociados con niveles particulares de matriculaciones de BMW, simplemente porque esas coincidencias se observaron en algunos años concretos del periodo de entrenamiento. Lo que podría dar lugar a malas predicciones futuras.

El sobreajuste está estrechamente relacionado con la complejidad del árbol. Ya que un árbol muy profundo tiende a tener hojas con muy pocas observaciones, lo que permite que el modelo se adapte a fluctuaciones locales de los datos. Lo que en un caso muy extremo, podría dar lugar a un árbol que creciera hasta tener una hoja por cada observación del conjunto de entrenamiento, logrando un error de entrenamiento nulo pero sin capacidad de generalización.

2.2.3.3 Subajuste

En el extremo opuesto al sobreajuste, existe el subajuste. El cual, ocurre cuando el modelo es demasiado simple para capturar los patrones relevantes presentes en los datos. Esto se suele dar en árboles con poca profundidad, muy pocas hojas y divisiones muy restrictivas. Lo que hace que no pueda ser capaz de representar relaciones importantes entre las variables de entrada.

Ilustración 13: Subajuste en comparación a los resultados de un modelo ajustado y uno sobreajustado



Fuente: Medium

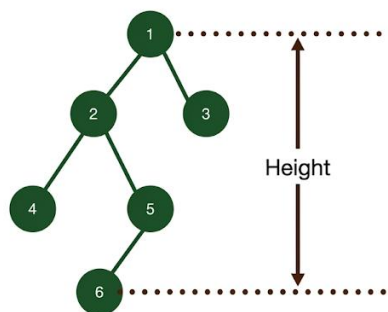
Por tanto, el subajuste se manifiesta cuando el modelo presenta un rendimiento insuficiente tanto sobre los datos de entrenamiento como sobre datos nuevos. Cuando se da este caso, el árbol no ha aprendido adecuadamente la relación subyacente entre las variables de entrada y la variable objetivo. En la práctica, si se diera en este trabajo, un árbol excesivamente simple podría predecir el mismo nivel de matriculaciones para una amplia gama de condiciones económicas, ignorando variaciones importantes que sí existen en la realidad. Lo que podría suponer un problema ya que no está resolviendo adecuadamente este problema.

2.2.3.4 Control de la complejidad mediante parámetros

Para poder controlar y encontrar el equilibrio adecuado entre sobreajuste y subajuste, es necesario controlar la complejidad del árbol mediante diferentes parámetros. Los cuales, limitarán el crecimiento del árbol y evitarán estas situaciones. Dichos parámetros son:

- **Profundidad máxima (max_depth):** establece el número máximo de niveles de división que puede tener el árbol. Un valor reducido de este parámetro produce árboles más simples y con menor riesgo de sobreajuste, pero puede limitar la capacidad del modelo para capturar relaciones complejas. No obstante, permitir una profundidad muy elevada incrementa el riesgo de sobreajuste.

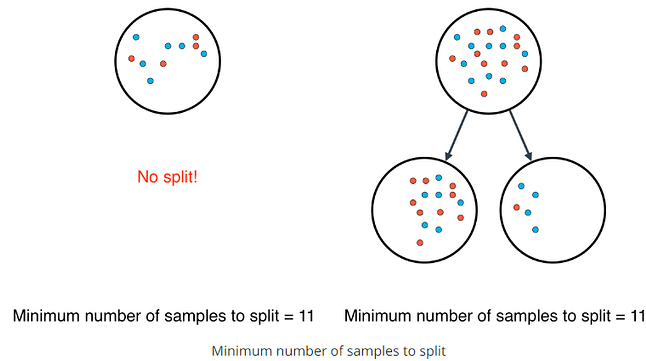
Ilustración 14: Altura máxima de un árbol



Fuente: Find the Maximum Depth or Height of a Binary Tree (YouTube)

- **Número mínimo de observaciones para dividir un nodo (min_samples_split):** determina cuántas observaciones debe contener un nodo para que pueda realizarse una nueva división. Este parámetro evita que se realicen divisiones basadas en muy pocas observaciones, lo que ayuda a prevenir divisiones espurias que capturen ruido en lugar de patrones reales y dé lugar a un sobreajuste.

Ilustración 15: Número mínimo de observaciones para dividir un nodo



Fuente: Dataquest Community

- **Número mínimo de observaciones en una hoja (min_samples_leaf):** garantiza que las predicciones finales se basen en un número suficiente de datos. Este parámetro es especialmente importante en problemas de regresión, ya que las hojas contienen la media de las observaciones que llegan hasta ellas. Y si una hoja tiene muy pocas observaciones, la media calculada puede ser muy sensible a valores atípicos o fluctuaciones aleatorias.
- **Mejora mínima necesaria para realizar una división (min_impurity_decrease):** establece que una división sólo se realizará si produce una reducción suficiente en el criterio de división. Lo que evita que existan divisiones que aporten poca información y que puedan estar capturando ruido en lugar de patrones significativos.

2.2.3.5 Poda del árbol

Además de controlar el crecimiento del árbol mediante restricciones, existen técnicas de poda. Las cuales consisten en eliminar ramas del árbol que aportan poca información o que pueden estar capturando ruido.

Existen dos enfoques principales de poda, la previa y la posterior. Donde, la poda previa limita el crecimiento del árbol durante su construcción, aplicando restricciones como las mencionadas anteriormente. Lo que hace que este enfoque sea más eficiente computacionalmente, ya que evita construir ramas que posteriormente serían eliminadas. Y la poda posterior, permite que el árbol crezca libremente y una vez hecho esto, elimina las ramas menos relevantes basándose en criterios de evaluación sobre un conjunto de validación.

2.2.4 Ventajas y limitaciones

Al igual que cualquier modelo, los árboles de decisión presentan un conjunto característico de ventajas y limitaciones que determinan su idoneidad para diferentes tipos de problemas. Por ello, a lo largo de este apartado, se presentarán las diferentes ventajas y limitaciones de este modelo.

2.2.4.1 *Ventajas de los árboles de decisión*

Una de las principales ventajas de los árboles de decisión es su capacidad para capturar relaciones no lineales entre las variables de entrada y la variable objetivo.

A diferencia de los modelos lineales, como la regresión lineal múltiple, los árboles no requieren que la relación subyacente sea lineal. Ya que, mediante sucesivas divisiones del espacio de variables, pueden aproximar funciones complejas y representar patrones que los modelos lineales no serían capaces de capturar.

Por otro lado, los árboles de decisión no requieren escalado de las variables de entrada. Ya que, las divisiones se basan en umbrales sobre los valores de las variables y no en distancias o medidas de similitud, el hecho de que las variables estén expresadas en escalas diferentes no afecta al funcionamiento del algoritmo. Esto simplifica la preparación de los datos, ya que no es necesario estandarizar o normalizar las variables macroeconómicas antes de utilizarlas.

Los árboles de decisión también tienen la capacidad de capturar interacciones entre variables de forma automática. De esta forma, un árbol puede combinar múltiples condiciones sobre diferentes variables para generar predicciones diferenciadas, sin necesidad de especificar explícitamente estas interacciones en el modelo. En este caso particular, un árbol podría aprender que el efecto del crecimiento del PIB sobre las matriculaciones depende del nivel de desempleo, representando esta interacción mediante divisiones sucesivas.

Además, la facilidad de interpretación constituye otra ventaja significativa de los árboles de decisión, especialmente cuando son de tamaño reducido. Ya que es posible visualizar el árbol completo y comprender las reglas de decisión que utiliza para generar predicciones y así poder validar que las reglas aprendidas son coherentes.

Por último, los árboles de decisión pueden manejar tanto variables numéricas como categóricas sin necesidad de transformaciones especiales. Esta flexibilidad los hace adecuados para una amplia variedad de problemas y tipos de datos como son los utilizados en este trabajo.

2.2.4.2 Limitaciones de los árboles de decisión

A pesar de sus ventajas, los árboles de decisión también presentan limitaciones importantes que deben ser consideradas. La más destacada es su elevado riesgo de sobreajuste, especialmente cuando se permiten árboles muy profundos. Como se ha discutido en el apartado anterior, un árbol complejo puede ajustarse excesivamente a los datos de entrenamiento, capturando ruido y particularidades específicas en lugar de patrones generales. Esta limitación requiere una regulación cuidadosa de la complejidad mediante parámetros de control como la profundidad máxima o el número mínimo de observaciones por hoja.

Por otro lado, también destaca la elevada sensibilidad a los datos de entrenamiento. Ya que los árboles de decisión individuales presentan una elevada varianza, lo que significa que pequeñas variaciones en la muestra de entrenamiento pueden producir árboles significativamente diferentes, lo que reduce la fiabilidad de las predicciones y hace que el modelo sea poco robusto.

Las predicciones escalonadas constituyen otra limitación de los árboles de decisión. Ya que cada hoja asigna un valor constante a todas las observaciones que llegan hasta ella. Lo que hace que las predicciones del modelo sean discontinuas. Esto se debe a que el árbol no puede generar valores intermedios dentro de una región, sino que asigna el mismo valor a todas las observaciones que cumplen las mismas condiciones. Y esto puede resultar en predicciones menos suaves que las de otros modelos, especialmente cuando el árbol tiene pocas hojas.

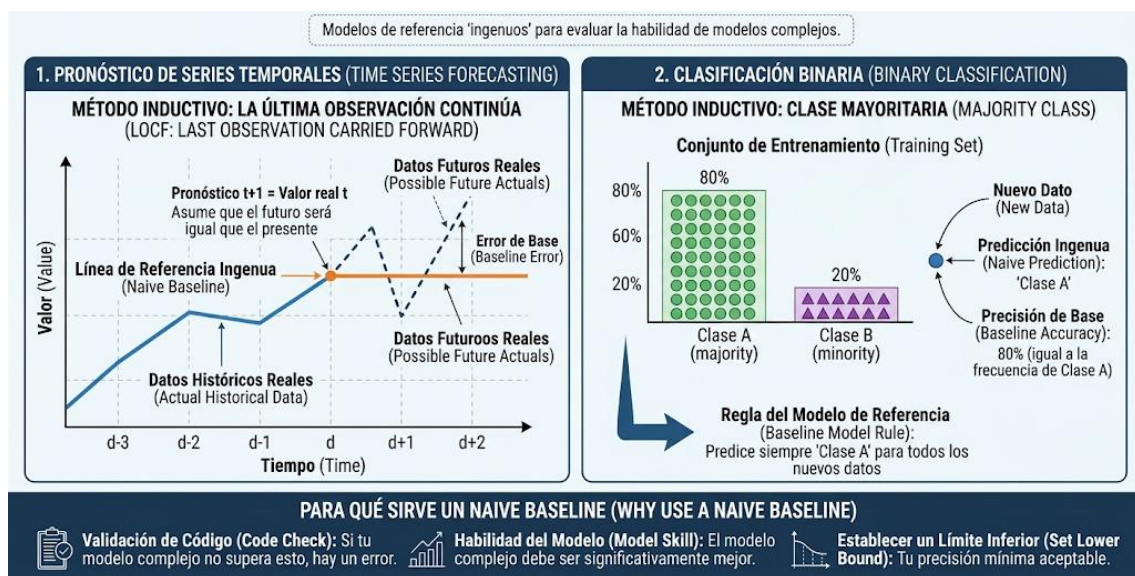
Por otro lado, los árboles de decisión también tienen una capacidad de extrapolación limitada. Es decir, no pueden predecir valores fuera del rango observado en los datos de entrenamiento, ya que las hojas contienen valores calculados a partir de observaciones existentes. Esta limitación es especialmente relevante en el contexto de este trabajo, donde las matriculaciones de BMW pueden presentar tendencias de largo plazo que se extiendan más allá del rango histórico observado.

Por último, los árboles de decisión tienden a favorecer variables con más niveles o categorías al seleccionar divisiones. En el caso de variables continuas, esto puede traducirse en una preferencia por variables con mayor variabilidad o más valores únicos. Aunque esta limitación es menos relevante cuando todas las variables son continuas, como en este trabajo, puede afectar a la interpretación de la importancia de las variables.

2.3 Naive Baseline

El objetivo de este trabajo es el de comparar diferentes modelos. No obstante, éstos no se pueden comparar de cualquier manera. Sino que es importante disponer de un punto de referencia contra el que comparar el rendimiento de los modelos más complejos. Y es por ello por lo que se implementará un naive baseline (línea base “ingenua”). Este es un modelo que no realiza ningún tipo de aprendizaje a partir de las variables macroeconómicas, sino que utiliza una regla de predicción extremadamente simple basada únicamente en la información histórica de la variable objetivo y con la que poder comparar los resultados de los diferentes modelos utilizados.

Ilustración 16: Naive Baseline



Fuente: Elaboración Propia Generada por Inteligencia Artificial (Gemini)

2.3.1 Funcionamiento del Naive Baseline

El naive baseline se define como un modelo de persistencia, que predice para cada año el mismo valor de matriculaciones observado en el año inmediatamente anterior. Y formalmente, la predicción para el año t se calcularía de la siguiente forma:

$$\hat{y}_t = y_{t-1}$$

Donde y_{t-1} representará las matriculaciones reales en el año anterior e $\hat{y}_t$ serían las del año actual. Por tanto, se asume que el nivel de ventas del año previo es la mejor predicción posible para el año siguiente, sin considerar cambios en el entorno macroeconómico ni tendencias a largo plazo.

2.3.2 Aplicación en este proyecto

En el contexto de este trabajo, el naive baseline se aplica únicamente sobre el conjunto de prueba, utilizando los valores reales del año anterior disponibles en el conjunto de entrenamiento o en años previos del propio test. De esta forma, se garantiza que la comparación con los demás modelos sea justa y que el baseline no utilice información futura.

La variable objetivo tiene el siguiente aspecto:

$$Y_t = \text{Matriculaciones BMW}_t$$

Y la predicción del naive baseline para el año t es:

$$\widehat{\text{Matriculaciones BMW}}_t = \text{Matriculaciones BMW}_{t-1}$$

Lo cual se corresponde a la fórmula descrita en el apartado anterior.

2.3.3 Ventajas y limitaciones

Este enfoque presenta varias ventajas en el contexto del presente estudio. Entre ellas que es sencillo de interpretar, ya que establece como predicción para cada año el nivel de matriculaciones observado en el año inmediatamente anterior. Por otro lado, proporciona una referencia de rendimiento que permite evaluar si los modelos más sofisticados aportan información predictiva adicional. Ya que si un modelo como Random Forest o Gradient Boosting no consigue mejorar las métricas de error respecto al naive baseline, esto indicaría que las variables macroeconómicas incluidas podrían no estar aportando información predictiva adicional suficiente o que el modelo no está siendo capaz de aprovecharla adecuadamente.

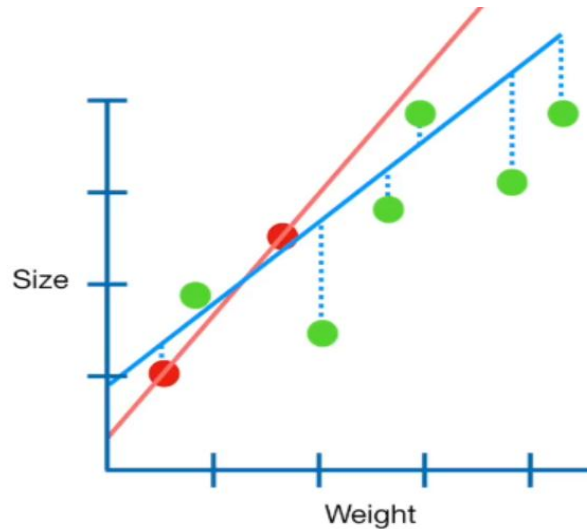
No obstante, el naive baseline también presenta limitaciones importantes. Ya que al no utilizar variables explicativas, no puede capturar las relaciones existentes entre las matriculaciones y la evolución del entorno macroeconómico. Asimismo, al basarse exclusivamente en el valor del año anterior, puede presentar dificultades en periodos caracterizados por variaciones significativas de la demanda o cambios estructurales, entre otros, ya que no incorpora mecanismos explícitos para anticipar estos cambios.

Esta última cuestión resulta relevante debido al reducido número de observaciones y a la dependencia temporal existente entre ellas. Y es por ello por lo que el naive baseline servirá como modelo de referencia, lo que permitirá determinar si los modelos de aprendizaje automático desarrollados posteriormente consiguen aportar una mejora real respecto a una estrategia de predicción basada únicamente en la información del año anterior.

2.4 Ridge Regression

La Regresión Ridge o Ridge Regression, representa un modelo lineal regularizado. A diferencia de la regresión lineal ordinaria, que minimiza únicamente la suma de los errores cuadráticos, éste añade un término de penalización basado en la norma L2 de los coeficientes. Este término de regularización tiene como objetivo principal controlar la complejidad del modelo y reducir la varianza de las estimaciones, especialmente en presencia de multicolinealidad entre las variables de entrada.

Ilustración 17: Ridge Regression



Fuente: Stack Exchange

2.4.1 Funcionamiento del Ridge Regression

La regresión Ridge busca determinar los coeficientes que minimizan simultáneamente el error de predicción y una penalización sobre la magnitud de dichos coeficientes. El objetivo que minimiza el modelo puede expresarse como:

$$\min_{\omega} (\|y - X\omega\|_2^2 + \alpha \|\omega\|_2^2)$$

Donde:

- y es el vector de matriculaciones reales de BMW
- X es la matriz de variables macroeconómicas
- w es el vector de coeficientes del modelo
- α es el parámetro de regularización que controla la intensidad de la penalización

El primer término de la expresión, $\|y - X\omega\|_2^2$, corresponde a la suma de los errores cuadráticos entre los valores reales y las predicciones del modelo, al igual que en

la regresión lineal ordinaria. Por otro lado, el segundo término, $\alpha \|\omega\|_2^2$, es el término de regularización L2 que penaliza la magnitud de los coeficientes.

La inclusión del término $\alpha \|\omega\|_2^2$ tiene como objetivo reducir la magnitud de los coeficientes, especialmente cuando existe multicolinealidad entre las variables de entrada. Cuando las variables están fuertemente correlacionadas, la regresión lineal ordinaria puede producir coeficientes muy grandes y con signos contraintuitivos, ya que pequeños cambios en los datos pueden generar grandes variaciones en las estimaciones. Es por ello por lo que se utiliza la regularización Ridge. Ya que mitiga este problema al restringir el tamaño de los coeficientes, reduciendo así la varianza de las estimaciones a cambio de introducir un cierto sesgo.

La solución al problema de optimización de Ridge tiene la siguiente forma:

$$\omega = (X^T X + \alpha I)^{-1} X^T y$$

Donde I es la matriz identidad y a adición del término αI garantiza que la matriz $X^T X + \alpha I$ sea siempre invertible, incluso cuando $X^T X$ no lo es debido a la multicolinealidad perfecta o casi perfecta entre las variables.

De esta forma, la regresión Ridge consigue producir estimaciones más estables y robustas que la regresión lineal ordinaria, disminuyendo así el riesgo de sobreajuste cuando existe multicolinealidad entre las variables de entrada.

2.4.2 Aplicación a la regresión en este proyecto

En el contexto de este trabajo, la regresión Ridge se utiliza como un modelo lineal de referencia para predecir las matriculaciones anuales de BMW. Esta variable objetivo es cuantitativa y continua y el modelo estima un valor numérico a partir de las diferentes variables macroeconómicas utilizadas como variables de entrada.

En este caso, la variable objetivo sería:

$$Y_t = \text{Matriculaciones BMW}_t$$

Y las variables de entrada serían las variables macroeconómicas de la siguiente forma:

$$X_t = [X_{1,t}, X_{2,t}, \dots, X_{p,t}]$$

Donde cada $X_{j,t}$ representa una de las variables macroeconómicas empleadas por el modelo. Y la predicción del modelo se obtendría como:

$$\widehat{\text{Matriculaciones BMW}_t} = X_t \cdot \omega$$

Donde ω son los coeficientes estimados por el modelo Ridge.

De esta forma, el modelo genera una estimación lineal de las matriculaciones como combinación ponderada de las variables macroeconómicas.

Una de las principales ventajas de este enfoque es que el modelo proporciona estimaciones más estables que la regresión lineal ordinaria cuando existe multicolinealidad entre las variables de entrada. Lo que resulta interesante en el contexto de este proyecto, puesto que en el análisis previo de las variables de entrada, se observó que varias variables macroeconómicas presentan valores de VIF extremadamente elevados, lo que indica una fuerte dependencia lineal entre ellas.

Por otro lado, desde un punto de vista matemático, el modelo de regresión se podría representar de la siguiente forma:

$$\widehat{Matriculaciones\ BMW}_t = Ridge(X_t)$$

Donde *Ridge* representa el modelo de regresión Ridge y las $X_{j,t}$ son cada una de las variables macroeconómicas empleadas por el modelo. Además, el hiperparámetro α controla la intensidad de la regularización. Valores pequeños de este hiperparámetro (próximos a 0), hacen que el modelo se aproxime a la regresión lineal ordinaria, mientras que valores grandes penalizan fuertemente los coeficientes, acercándolos a cero.

2.4.3 Hiperparámetros principales

En el caso de la regresión Ridge, su rendimiento depende principalmente de un hiperparámetro fundamental:

- **Parámetro de regularización (α):** determina la intensidad de la penalización L2 aplicada a los coeficientes del modelo. Valores pequeños de alpha (próximos a 0) hacen que el modelo se aproxime a la regresión lineal ordinaria, lo que permite que los coeficientes adopten valores grandes si es necesario para minimizar el error. Por otro lado, valores grandes de alpha penalizan fuertemente la magnitud de los coeficientes, acercándolos a cero y produciendo un modelo más simple y estable.

La selección del valor óptimo de alpha es crítica. Ya que un valor demasiado pequeño puede no regular lo suficiente frente a la multicolinealidad. Mientras que un valor demasiado grande puede eliminar información relevante y producir un modelo subajustado.

Además, aunque no es un hiperparámetro del modelo en sí, es importante mencionar:

- **La estandarización de las variables:** la regresión Ridge requiere que las variables de entrada estén estandarizadas (media 0 y desviación estándar 1) antes de entrenar el modelo. Ya que la regularización L2 es sensible a la escala de las variables. Y sin esta estandarización, las variables con escalas numéricas mayores podrían verse penalizadas de forma desproporcionada, lo que distorsionaría los resultados.

2.4.4 Ventajas y limitaciones

La regresión Ridge presenta una serie de ventajas e inconvenientes entre los cuales destaca entre sus principales ventajas, su capacidad para proporcionar estimaciones más estables que la regresión lineal ordinaria cuando existe multicolinealidad entre las variables de entrada. Ya que la regularización reduce la varianza de los coeficientes a cambio de introducir un cierto sesgo. Además, al ser un modelo lineal, es fácilmente interpretable. Pues los coeficientes estimados pueden interpretarse directamente como el efecto marginal de cada variable sobre las matriculaciones, manteniendo constantes las demás variables. Además, requiere de menos supuestos sobre la distribución de los errores que otros modelos y es computacionalmente más eficiente, ya que tiene una solución cerrada.

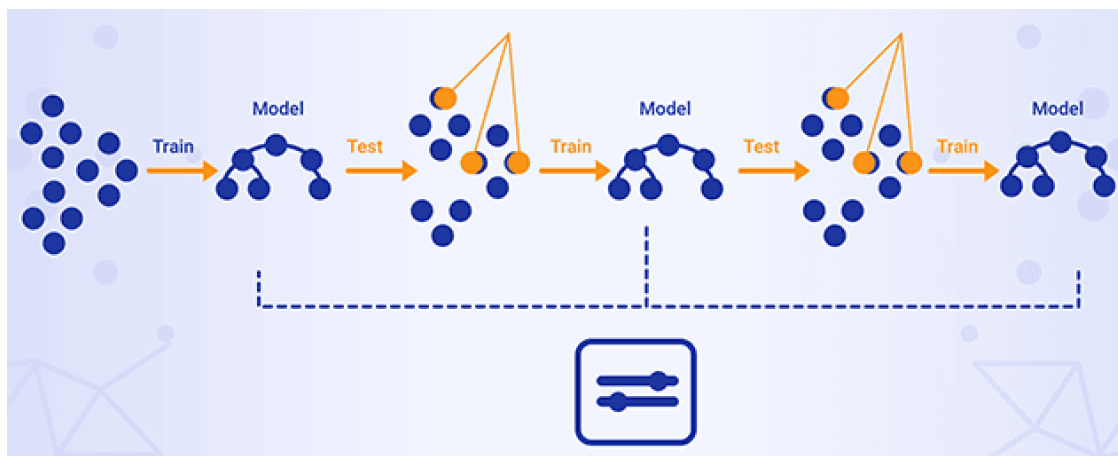
No obstante, también presenta algunas limitaciones. Entre ellas, se encuentra su naturaleza lineal, lo que hace que no pueda capturar relaciones no lineales ni interacciones complejas entre variables. Esta es una diferencia importante respecto a Random Forest y Gradient Boosting, las cuales sí pueden capturar estas relaciones e interacciones. Además, la selección del parámetro α es muy importante, ya que puede distorsionar los resultados si no se elige correctamente. Por otro lado, este modelo no realiza selección de variables, ya que la regularización L2 tiende a reducir todos los coeficientes pero no los lleva exactamente a cero. Esto significa que el modelo final incluye todas las variables de entrada, incluso aquellas que pueden no aportar información predictiva relevante.

En conclusión, la regresión Ridge servirá como un modelo de referencia lineal que permite evaluar si la regularización es suficiente para manejar la multicolinealidad o si se requieren modelos más flexibles. Aunque estos aspectos se desarrollarán posteriormente en los apartados correspondientes a la metodología y a los experimentos realizados.

2.5 Gradient Boosting

El Gradient Boosting Regressor es un algoritmo de aprendizaje conjunto basado en árboles de decisión, al igual que Random Forest. Sin embargo, mientras que Random Forest construye múltiples árboles de forma independiente y combina sus predicciones mediante promediado, Gradient Boosting construye los árboles de forma secuencial. Donde cada nuevo árbol intenta corregir los errores cometidos por los árboles anteriores. Este enfoque iterativo permite al modelo aprender progresivamente de sus propios errores, mejorando la precisión de las predicciones en cada etapa.

Ilustración 18: Gradient Boosting



Fuente: Xenon Stack

2.5.1 Funcionamiento del Gradient Boosting

El funcionamiento de Gradient Boosting sigue los siguientes pasos:

1. Se entrena un primer árbol de decisión sobre los datos originales. Este árbol inicial puede ser un modelo simple que proporciona una predicción básica.
2. Se calculan los residuos (errores) entre las predicciones del árbol y los valores reales. Estos residuos representan la parte de la variable objetivo que el modelo no ha sido capaz de explicar.
3. Se entrena un nuevo árbol para predecir esos residuos. En lugar de predecir directamente la variable objetivo, este árbol se centra en corregir los errores del modelo anterior.
4. Las predicciones del nuevo árbol se combinan con las anteriores, multiplicadas por un factor de aprendizaje (*learning rate*). Este factor controla la contribución de cada árbol al modelo final.

5. El proceso se repite durante un número determinado de iteraciones ($n_estimators$), añadiendo secuencialmente nuevos árboles que corrigen los errores residuales de los árboles anteriores.

Matemáticamente, la predicción final tendrá este resultado:

$$\hat{y} = \sum_{m=1}^M \eta \cdot h_m(X)$$

Donde:

- M es el número total de árboles ($n_estimators$),
- η es la tasa de aprendizaje ($learning_rate$),
- $h_m(X)$ es la predicción del árbol m -ésimo.

El primer árbol $h_1(X)$ se entrena sobre los valores originales de y . Y los árboles posteriores $h_m(X)$ se entrenan sobre los residuos de los árboles anteriores. Respecto a la predicción final, ésta es la suma ponderada de las predicciones de todos los árboles, donde el peso de cada árbol está determinado por la tasa de aprendizaje η .

Esta tasa de aprendizaje juega un papel crucial en el funcionamiento del modelo. Ya que valores como 0,01 o 0,05 hacen que cada árbol contribuya poco al modelo final, lo que requiere de más árboles para alcanzar un buen ajuste pero generalmente produce modelos que generalizan mejor. Mientras que valores grandes de η (por ejemplo, 0,1 o superiores) hacen que el modelo aprenda más rápido, pero pueden aumentar el riesgo de sobreajuste.

De esta forma, Gradient Boosting consigue construir un modelo potente combinando muchos árboles débiles. De los cuales, cada uno corrige parcialmente los errores de los anteriores, mejorando progresivamente la capacidad predictiva del modelo.

2.5.2 Aplicación a la regresión en este proyecto

En el contexto de este trabajo, Gradient Boosting se utilizará como un modelo de regresión basado en árboles para predecir las matriculaciones anuales de BMW.

La variable objetivo es cuantitativa y continua y el modelo estima un valor numérico a partir de las diferentes variables macroeconómicas utilizadas como variables de entrada.

En este caso, esta variable objetivo sería:

$$Y_t = \text{Matriculaciones BMW}_t$$

Y las variables de entrada serían las variables macroeconómicas de la siguiente forma:

$$X_t = [X_{1,t}, X_{2,t}, \dots, X_{p,t}]$$

Donde cada $X_{j,t}$ representa una de las variables macroeconómicas empleadas por el modelo. Y la predicción del modelo se obtendría como:

$$\widehat{\text{Matriculaciones BMW}_t} = GB(X_t)$$

Donde GB representa el modelo Gradient Boosting.

De esta forma, el modelo genera una estimación de las matriculaciones como suma ponderada de las predicciones de múltiples árboles de decisión, donde cada árbol corrige los errores de los anteriores.

Una de las principales ventajas de este enfoque es que el modelo puede capturar relaciones no lineales e interacciones entre variables macroeconómicas, al igual que Random Forest. Sin embargo, su enfoque secuencial y la posibilidad de controlar la tasa de aprendizaje le otorgan un perfil de regularización diferente, que en algunos conjuntos de datos puede traducirse en un mejor rendimiento.

Desde un punto de vista matemático, el modelo de regresión se podría representar de la siguiente forma:

$$\widehat{\text{Matriculaciones BMW}_t} = \sum_{m=1}^M \eta \cdot h_m(X_t)$$

donde M representa el número total de árboles que forman el Gradient Boosting, η la tasa de aprendizaje y las $h_m(X_t)$, las predicciones de cada árbol individual.

2.5.3 Hiperparámetros principales

Los hiperparámetros principales de Gradient Boosting son:

- **Número de árboles (n_estimators):** determina el número de árboles o etapas de boosting. Un mayor número permite un ajuste más fino, pero incrementa el coste computacional y el riesgo de sobreajuste si no se controla adecuadamente.
- **Tasa de aprendizaje (learning_rate):** factor de contracción que reduce la contribución de cada árbol. Valores muy bajos hacen que el modelo sea más conservador y requiera más árboles para alcanzar un buen ajuste, pero generalmente generalizan mejor. Existe un compromiso (*trade-off*) entre learning_rate y n_estimators.

- **Profundidad máxima (max_depth):** establece la profundidad máxima de cada árbol. Valores reducidos limitan la complejidad de cada árbol individual, ya que en Gradient Boosting se prefiere combinar muchos árboles débiles en lugar de unos pocos muy complejos.
- **Número mínimo de observaciones para dividir un nodo (min_samples_split) y número mínimo de observaciones por hoja (min_samples_leaf):** controlan el número mínimo de observaciones necesarias para dividir un nodo y para formar una hoja, respectivamente. Lo que ayuda a prevenir el sobreajuste.
- **Submuestreo (subsample):** fracción de observaciones utilizadas para entrenar cada árbol. Valores inferiores a 1 introducen aleatoriedad adicional y pueden mejorar la generalización, de forma similar al bootstrap en Random Forest.

2.5.4 Ventajas y limitaciones

En el caso del modelo Gradient Boosting, éste destaca por su capacidad para representar relaciones no lineales, capturar interacciones entre variables y su enfoque secuencial que permite corregir errores de forma iterativa. Además, el control de la tasa de aprendizaje proporciona un mecanismo adicional de regularización que puede mejorar la generalización. Además, es un modelo flexible y puede adaptarse a diferentes tipos de problemas mediante la modificación de la función de pérdida.

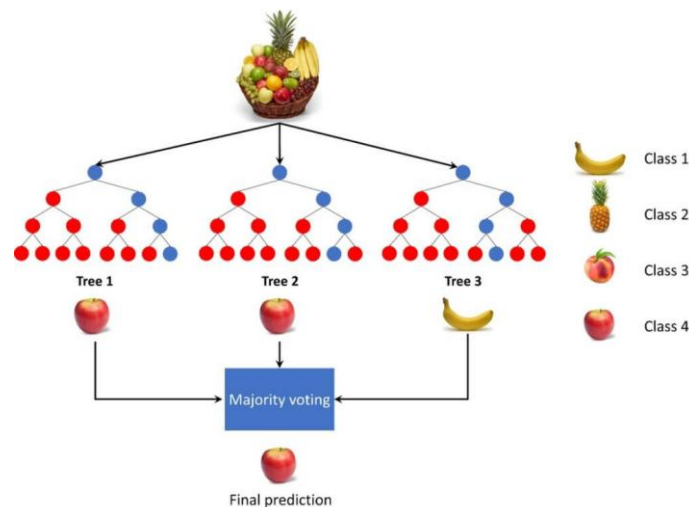
Sin embargo, también presenta algunas limitaciones. Entre ellas, es más sensible a la selección de hiperparámetros que Random Forest, especialmente en cuanto a la tasa de aprendizaje y el número de árboles. Por tanto, un modelo demasiado complejo puede sobreajustarse fácilmente, especialmente con un número reducido de observaciones. Además, el entrenamiento secuencial de los árboles hace que el coste computacional sea mayor que en Random Forest, ya que no es posible paralelizar completamente el entrenamiento de los árboles.

Otra limitación importante es que, al igual que otros modelos basados en árboles, tiene una capacidad de extrapolación limitada y no puede predecir valores fuera del rango observado durante el entrenamiento

2.6 Random Forest

Los Random Forest o también conocidos como bosques aleatorios, son métodos de Aprendizaje Automático basados en la combinación de múltiples árboles de decisión. Su objetivo consiste en mejorar la capacidad predictiva y la estabilidad de un árbol individual mediante la construcción de un conjunto de árboles que posteriormente combinan sus resultados. Estos métodos pueden tener dos enfoques diferentes dirigidos a los problemas de regresión o los problemas de clasificación como el de la imagen inferior.

Ilustración 19: Random Forest Classifier



Fuente: Master Data Science

Un árbol de decisión divide progresivamente el conjunto de datos en diferentes regiones mediante reglas basadas en las variables de entrada. Por ejemplo, un árbol podría establecer una primera división según el crecimiento del PIB y posteriormente, realizar nuevas divisiones según el nivel de desempleo. Y así, cada una de sus divisiones pretende generar grupos de observaciones con valores objetivo lo más similares posible.

Es probable que un árbol individual pueda adaptarse excesivamente a los datos de entrenamiento y lo esté memorizando. Lo que da lugar a un fenómeno, conocido como sobreajuste u overfitting. Para reducir este problema, el método Random Forest combina múltiples árboles de decisión entrenados sobre diferentes muestras de los datos y utilizando subconjuntos aleatorios de las variables. De este modo, las predicciones de los distintos árboles se combinan para obtener una predicción final más estable y robusta. Lo que permite reducir la dependencia de un único árbol y mejorar la capacidad de generalización del modelo ante datos no vistos.

2.6.1 Funcionamiento de un Random Forest

La construcción de un Random Forest se basa principalmente en dos fuentes de aleatoriedad. En primer lugar, cada árbol se entrena utilizando una muestra aleatoria de las observaciones disponibles, obtenida mediante bootstrap. En segundo lugar, en cada división (split) del árbol únicamente se considera un subconjunto aleatorio de las variables de entrada. Estas dos fuentes de aleatoriedad permiten generar árboles diferentes entre sí, reduciendo la correlación entre ellos y evitando que todos los árboles aprendan exactamente los mismos patrones de los datos de entrenamiento. Posteriormente, las predicciones de los diferentes árboles se combinan para obtener la predicción final del bosque.

Como consecuencia de esto, los árboles resultantes no son completamente iguales entre sí, ya que cada uno aprende una representación ligeramente diferente de los datos y sus errores individuales pueden compensarse al combinar las predicciones. Y es que la idea fundamental de esto se basa en que un conjunto de modelos parcialmente independientes pueda presentar un comportamiento más robusto que cualquiera de ellos por separado. De esta forma, al combinar las predicciones de los diferentes árboles, el Random Forest consigue reducir la variabilidad de las predicciones y mejorar la capacidad de generalización del modelo, disminuyendo así el riesgo de sobreajuste respecto a un único árbol de decisión.

Matemáticamente, en un problema de clasificación, cada árbol proporciona una clase y la predicción final se obtiene mediante votación mayoritaria que tiene la siguiente forma:

$$\hat{Y}(X) = \text{moda}\{T_1(X), T_2(X), \dots, T_B(X)\}$$

Donde $T_B(X)$ representa la predicción realizada por el árbol para una determinada observación x , y B es el número total de árboles del bosque. En la práctica, también pueden calcularse probabilidades de pertenencia a las diferentes clases a partir de la proporción de árboles que votan por cada categoría.

En problemas de regresión, el procedimiento es diferente. Ya que, en lugar de realizar una votación entre categorías, cada árbol proporciona un valor numérico y la predicción final del Random Forest se obtiene calculando la media de las predicciones de todos los árboles de la siguiente manera:

$$\hat{Y}(X) = \frac{1}{B} \sum_{b=1}^B T_b(X)$$

De esta forma, la combinación de las predicciones de los diferentes árboles permite obtener una estimación más estable y reducir la variabilidad asociada a cada árbol individual.

Es importante hablar de problemas de clasificación y de regresión ya que Random Forest se puede aplicar a ambos casos y la diferencia que tienen se basa principalmente en cómo combinan sus predicciones como se especifica con anterioridad:

- **Random Forest Classifier:** problemas de clasificación. Cada árbol predice una categoría y el resultado final se obtiene mediante votación mayoritaria.
- **Random Forest Regressor:** problemas de regresión. Cada árbol predice un valor numérico y el resultado final se obtiene mediante la media de las predicciones de todos los árboles.

2.6.2 Aplicaciones a la regresión y a la clasificación

Como se ha mencionado anteriormente, Random Forest puede emplearse tanto en problemas de clasificación como de regresión. Por ello, a continuación, se describirán las principales características de su posible aplicación en los diferentes tipos de problemas que surgen en este proyecto.

2.6.3 Aplicación a la regresión en este proyecto

En el contexto de este trabajo, el Random Forest puede utilizarse como un modelo de regresión para predecir las matriculaciones anuales de BMW. Ya que la variable objetivo es cuantitativa y continua. Y el modelo permite estimar un valor numérico a partir de las diferentes variables macroeconómicas utilizadas como variables de entrada. Lo que permitirá al modelo aprender la relación existente entre las variables predictoras y las matriculaciones observadas para posteriormente generar una estimación de las ventas en años no observados.

En este caso, la variable objetivo sería:

$$Y_t = \text{Matriculaciones BMW}_t$$

Y las variables de entrada serían las variables macroeconómicas de la siguiente forma:

$$X_t = [X_1, X_2, \dots, X_p]$$

Donde cada una de las X_i representa una de las variables macroeconómicas empleadas por el modelo como por ejemplo GDP_Growth o HICP.

De esta forma, cada árbol generaría una estimación diferente de las matriculaciones y el bosque calcularía la media de todas ellas para obtener la predicción final.

Una de las principales ventajas de este enfoque es que el modelo no necesita asumir que la relación entre las variables macroeconómicas y las ventas es estrictamente lineal, ya que los árboles de decisión pueden representar relaciones más complejas entre las variables de entrada y la variable objetivo. Lo que resulta interesante en el contexto de este proyecto, puesto que el comportamiento de las matriculaciones puede depender de determinados umbrales, relaciones no lineales o interacciones entre diferentes variables macroeconómicas.

Desde un punto de vista matemático, el modelo de regresión se podría representar de la siguiente forma:

$$\widehat{\text{Matriculaciones BMW}} = RF(X_1, X_2, \dots, X_p)$$

Donde RF representará el conjunto de árboles que forman el Random Forest y las X_i , cada una de las variables macroeconómicas empleadas por el modelo.

2.6.4 Aplicación a la clasificación

Dado que el Random Forest también puede utilizarse como un modelo de clasificación para asignar cada observación a una categoría determinada, la variable objetivo no sería un valor numérico continuo, sino una variable categórica que representaría la clase a la que pertenece cada observación. De esta forma, el modelo aprendería la relación existente entre las variables de entrada y las categorías observadas para, posteriormente, clasificar nuevas observaciones en una de las clases disponibles.

En el caso de este proyecto, se podría utilizar para predecir si las matriculaciones de BMW aumentarán de la siguiente forma:

$$Y_t = \begin{cases} 1 & \text{si las matriculaciones aumentan} \\ 0 & \text{si las matriculaciones disminuyen o se mantienen} \end{cases}$$

De esta forma, cada árbol clasificaría el año como perteneciente a una de las dos categorías y la predicción final se obtendría mediante la clase que recibiera el mayor número de votos. No obstante, este problema también se podría definir como un problema multiclase con tres posibles categorías:

$$Y_t = \begin{cases} 0 & \text{descenso} \\ 1 & \text{estabilidad} \\ 2 & \text{crecimiento} \end{cases}$$

Y de esta manera, se calcularía la probabilidad estimada de pertenencia a cada una de estas categorías y asignaría cada observación a la clase con mayor probabilidad.

2.6.5 Parámetros principales

En el caso del Random Forest, su rendimiento depende de varios hiperparámetros entre los cuales destacan:

- **Número de árboles (n_estimators):** determina cuántos árboles de decisión forman el bosque. Cada uno de ellos genera una predicción y éstas se combinan para obtener el resultado final. Un mayor número de árboles suele producir predicciones más estables y reducir la variabilidad del modelo, aunque también aumenta el coste computacional y el tiempo necesario para entrenarlo.
- **Profundidad máxima (max_depth):** establece el número máximo de niveles que puede alcanzar cada árbol. Una mayor profundidad permite que los árboles aprendan relaciones más complejas presentes en los datos, pero también puede aumentar el riesgo de sobreajuste u overfitting. No obstante, limitar la profundidad puede ayudar a obtener modelos más sencillos y que posean una mayor capacidad de generalización.
- **Número mínimo de observaciones para dividir un nodo (min_samples_split):** establece el número mínimo de observaciones que debe contener un nodo para que pueda realizarse una nueva división. Valores pequeños permiten construir árboles más detallados y complejos, mientras que valores mayores restringen el crecimiento de los árboles y pueden contribuir a reducir el sobreajuste.
- **Número mínimo de observaciones por hoja (min_samples_leaf):** es el número mínimo de observaciones que debe contener una hoja, es decir, un nodo que ya no se divide. Establecer un valor mayor evita que los árboles creen hojas basadas en un número muy reducido de observaciones y favorece predicciones más estables.
- **Número de variables consideradas en cada división (max_features):** determina el número o proporción de variables de entrada que se consideran como candidatas en cada división de los árboles. Al limitar las variables disponibles en cada decisión se incrementa la diversidad entre los diferentes árboles del bosque, evitando que todos ellos se construyan siguiendo exactamente los mismos patrones.

- **Muestreo con reemplazamiento (bootstrap):** determina si cada árbol se entrena utilizando una muestra obtenida mediante bootstrap, es decir, seleccionando observaciones aleatoriamente con reemplazamiento a partir del conjunto de entrenamiento. De esta forma, cada árbol puede entrenarse con una muestra ligeramente diferente de los datos, contribuyendo a aumentar la diversidad del bosque y a mejorar la robustez de las predicciones.

2.6.6 Ventajas y limitaciones

Los Random Forest presentan una serie de características que los hacen adecuados para abordar diferentes tipos de problemas de aprendizaje supervisado.

Entre sus principales ventajas se encuentra su capacidad para representar relaciones no lineales entre las variables de entrada y la variable objetivo, así como para capturar interacciones entre variables sin necesidad de especificarlas previamente. Por otro lado, al combinar las predicciones de múltiples árboles, el modelo es generalmente menos propenso al sobreajuste que un árbol de decisión individual. Además, requiere de menos supuestos sobre la relación entre las variables que otros modelos y puede aplicarse tanto a problemas de clasificación como de regresión. Asimismo, permite obtener estimaciones de la importancia relativa de las variables utilizadas por el modelo.

Sin embargo, los Random Forest también presentan algunas limitaciones. Entre las que destaca su estructura formada por múltiples árboles. Lo que hace que sean menos interpretables que modelos más sencillos. Además, el entrenamiento de un número elevado de árboles puede incrementar el coste computacional. Por otro lado, las predicciones pueden presentar una menor capacidad de extrapolación cuando se realizan fuera del rango de valores observado durante el entrenamiento. También es necesario tener especial precaución cuando se dispone de un número reducido de observaciones, ya que un modelo excesivamente complejo puede ajustarse a patrones específicos de la muestra y presentar una peor capacidad de generalización.

Asimismo, la importancia de las variables proporcionada por el modelo debe interpretarse con cautela, ya que una mayor importancia predictiva no implica necesariamente la existencia de una relación causal. Además, cuando los datos presentan una dependencia temporal, como ocurre en este proyecto, la partición y validación del conjunto de datos deben realizarse respetando el orden cronológico de las observaciones, evitando que información procedente del futuro pueda influir en el entrenamiento del modelo.

Esta última cuestión resulta especialmente relevante para el estudio de las matriculaciones anuales de BMW, debido al reducido número de observaciones disponibles y a la existencia de una dependencia temporal en los datos. En consecuencia, será necesario controlar la complejidad del modelo y emplear procedimientos de partición y validación que permitan evaluar adecuadamente su capacidad de generalización. Estos aspectos se desarrollarán posteriormente en los apartados correspondientes a la metodología y a los experimentos realizados.

2.7 Métricas de evaluación

Las métricas de evaluación son aquellas medidas cuantitativas que permiten determinar la capacidad de un modelo de Aprendizaje Automático para realizar predicciones adecuadas. Su cálculo se basa en comparar los valores reales de la variable objetivo con los valores estimados por el modelo. Lo que permite cuantificar el error cometido y analizar la capacidad predictiva de los diferentes algoritmos.

En el presente trabajo, el problema planteado es principalmente un problema de regresión, ya que el objetivo consiste en predecir el número de matriculaciones anuales de BMW a partir de un conjunto de variables macroeconómicas. Por este motivo, las métricas utilizadas para evaluar los modelos serán principalmente el error absoluto medio (MAE), la raíz del error cuadrático medio (RMSE) y el coeficiente de determinación (R^2).

Y para calcular el error, para una observación i , se denotará Y_i como el valor real de las matriculaciones y por $\hat{Y}_i$ el valor predicho por el modelo. Por tanto, podría expresarse de la siguiente forma:

$$e_i = Y_i - \hat{Y}_i$$

Y a partir de estos errores se obtienen las diferentes métricas de evaluación. Las cuales se calculan de maneras determinadas que se pueden ver a continuación:

1. **Error absoluto medio:** también conocido como MAE (*Mean Absolute Error*), representa la diferencia absoluta media entre los valores reales y las predicciones como se puede observar a continuación:

$$MAE = \frac{1}{n} \sum_{i=1}^n |Y_i - \hat{Y}_i|$$

Esto permite conocer cuánto se aleja, en promedio, la predicción del valor real. Y al utilizar el valor absoluto, los errores positivos y negativos no se compensan entre sí.

Una de las principales ventajas que presenta, es su fácil interpretación, ya que mantiene las mismas unidades que la variable objetivo. Y además, al no elevar los errores al cuadrado, presenta una menor sensibilidad a valores extremos que el RMSE que se verá a continuación.

2. **Raíz del error cuadrático medio:** también denominada como RMSE (*Root Mean Squared Error*), se calcula mediante la siguiente expresión:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$$

Y como se puede observar, a diferencia del MAE, esta métrica eleva los errores al cuadrado antes de calcular su media. Lo que tiene como consecuencia que los errores de mayor magnitud tengan un peso superior en el resultado final. Aunque al aplicar posteriormente la raíz cuadrada, hace que el RMSE vuelva a expresarse en las mismas unidades que la variable objetivo ya que hay exponentes al cuadrado dentro de la expresión.

El hecho de comparar entre MAE y RMSE puede resultar especialmente útil. Ya que, si ambas métricas presentan valores similares, los errores del modelo tienden a presentar magnitudes relativamente homogéneas. No obstante, si el RMSE es considerablemente superior al MAE, puede indicar la existencia de algunas predicciones con errores especialmente elevados.

Esta característica resulta relevante en el estudio de las matriculaciones de BMW, ya que determinados años pueden presentar variaciones excepcionales asociadas a situaciones económicas o acontecimientos que no hayan sido recogidos completamente por las variables utilizadas.

3. **Coefficiente de determinación:** también conocido como R^2 , permite medir la proporción de la variabilidad de la variable objetivo que es explicada por el modelo. Y su expresión es:

$$R^2 = 1 - \frac{\sum_{i=1}^n (Y_i - \hat{Y}_i)^2}{\sum_{i=1}^n (Y_i - \bar{Y})^2}$$

Donde $\bar{Y}$ representa la media de los valores reales de la variable objetivo y los resultados obtenidos de la expresión completa tienen la siguiente interpretación:

- $R^2 = 1$: significa que el modelo realiza un ajuste perfecto sobre las observaciones evaluadas
- $R^2 = 0$: quiere decir que el modelo obtiene un resultado equivalente al de utilizar la media de la variable objetivo como predicción constante
- $R^2 < 0$: el modelo presenta un rendimiento inferior al de dicha predicción basada en la media

Lo que significaría que un $R^2 = 0,70$ explica aproximadamente el 70% de la variabilidad observada en las matriculaciones dentro del conjunto sobre el que se calcula. No obstante, un valor elevado de R^2 no implica necesariamente que las predicciones sean suficientemente precisas. Por este motivo, esta métrica deberá interpretarse conjuntamente con el MAE y el RMSE. En particular, en este trabajo será importante comprobar que un modelo con un R^2 elevado también presenta errores suficientemente reducidos en términos de matriculaciones.

2.7.1 Comparación de los diferentes modelos

Las métricas anteriores permitirán comparar los diferentes modelos en el trabajo bajo unas condiciones homogéneas y se compararían utilizando una tabla que tendrá un aspecto similar al de la tabla siguiente:

Tabla 1: Comparación de los modelos utilizados mediante diferentes métricas

Modelo	MAE	RMSE	R^2
BASELINE NAIVE	-	-	-
GRADIENT BOOSTING	-	-	-
RIDGE REGRESSION	-	-	-
RANDOM FOREST	-	-	-

Fuente: *Elaboración Propia*

En términos generales, un enfoque presentará un mejor comportamiento cuando obtenga valores reducidos de MAE y RMSE y un valor elevado de R^2 . No obstante, la selección del mejor enfoque no deberá basarse exclusivamente en una única métrica, sino que será necesario considerar conjuntamente los resultados obtenidos.

Además, dado que el conjunto de datos utilizado en este trabajo contiene un número reducido de observaciones, pequeñas diferencias en el planteamiento pueden estar condicionadas por los años concretos incluidos en el conjunto de prueba. Por ello, los resultados deberán interpretarse con precaución y teniendo en cuenta también la estabilidad del modelo y su capacidad para generalizar a años no utilizados durante el entrenamiento. Aunque es cierto que esa es la particularidad de este proyecto y lo que demostrará la capacidad de los diferentes enfoques de tratar con conjuntos muy reducidos de datos como el caso de este proyecto que cuenta con 21 observaciones.

2.7.2 Consideraciones sobre la evaluación

Es importante remarcar que la evaluación de los modelos se realizará sobre observaciones que no hayan sido utilizadas para su entrenamiento para que las métricas obtenidas proporcionen una estimación de la capacidad del modelo para realizar predicciones sobre datos no observados.

Además, debido a que las observaciones corresponden a años consecutivos, la separación entre entrenamiento, validación y prueba deberá respetar la naturaleza temporal del conjunto de datos. Por ello, se utilizará información correspondiente a años posteriores para entrenar el modelo y así poder evaluar su rendimiento sobre años anteriores. De esta forma se podrá complementar el análisis de las métricas anteriores en el capítulo 5 una vez llevadas a cabo las pruebas del capítulo 4.

Capítulo 3:

Preparación de los datasets para el entrenamiento y evaluación

3. Datasets

Una vez explicados los modelos y las métricas utilizadas que se utilizarán para compararlos, se llevará a cabo a lo largo de todo este apartado, el recopilado y preparado de datos provenientes de diferentes fuentes de información que contendrán tanto los datos de ventas de BMW como las principales variables macroeconómicas consideradas relevantes para el estudio.

3.1 Zona geográfica seleccionada

Es importante denotar que el área geográfica seleccionada es Europa. Ya que es lógico trabajar con datos provenientes del continente en el que nos encontramos. Aunque algunas variables pueden diferir en la zona geográfica ya que están referidas a la Unión Europea o a la Eurozona. Por lo que es importante tener este aspecto en cuenta ya que el ámbito geográfico específico de cada variable puede influir en la relación observada entre los indicadores macroeconómicos y las ventas de BMW.

3.1.1 Europa

Europa se conforma de 44 a 49 países. Dependiendo de si se deciden incluir los estados parcialmente reconocidos o los territorios transcontinentales que comparten algunas zonas de Asia.

Ilustración 20: Vista aérea de Europa



Fuente: deutschland.de

3.1.2 Unión Europea

La Unión Europea (UE) es una comunidad política y económica integrada por 27 países europeos. Lo que reduce la cobertura geográfica de algunas variables macroeconómicas.

Ilustración 21: Mapa de la Unión Europea

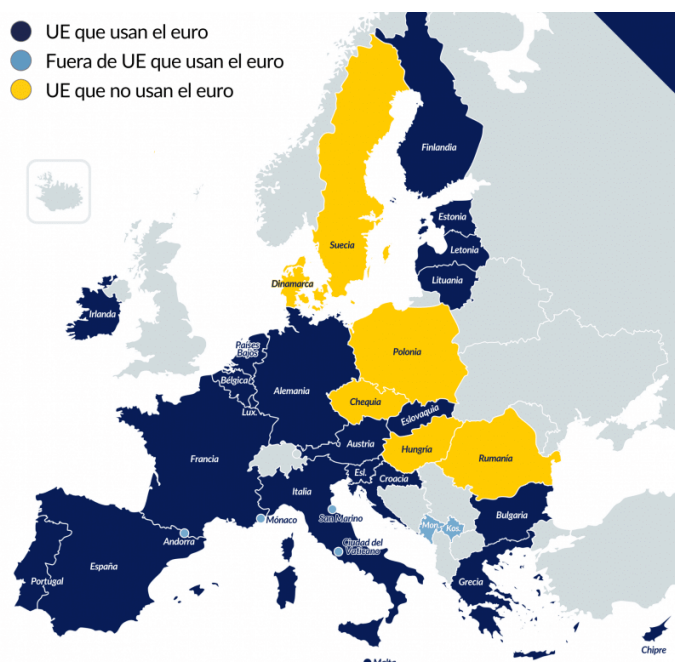


Fuente: Economía Simple

3.1.3 Eurozona

La Eurozona es el área económica y monetaria formada por los Estados miembros de la Unión Europea (UE) que han adoptado el euro como su moneda oficial y comparten una política monetaria única dirigida por el Banco Central Europeo (BCE) y está conformada por 21 países. Que son menos que los de la Unión Europea (UE) y Europa ya que son una región menor.

Ilustración 22: Eurozona



Fuente: Lisa News

Por otro lado, es importante denotar que todos los estudios llevados a cabo para este proyecto, se están llevando a cabo en diferentes cuadernos Jupyter que recogen todos los pasos llevados a cabo desde la carga de los datos hasta todas las operaciones llevadas a cabo con los mismos.

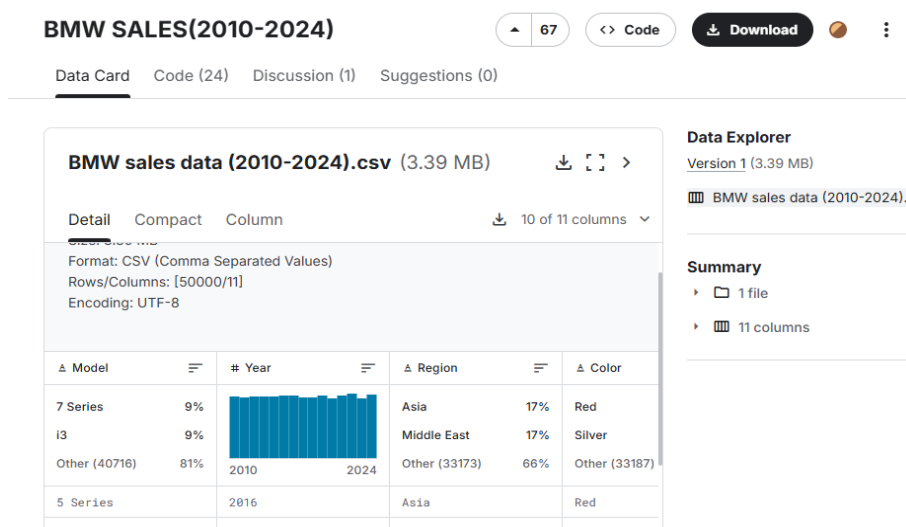
3.2 El dataset BMW

El paso inicial de este proceso será construir un dataset con las ventas anuales de BMW en Europa para el periodo comprendido entre 2005 y 2025. La razón por la que se ha optado por este intervalo temporal, se debe a que es el rango completo para el que se dispone de datos anuales de ventas en Europa, lo que permite trabajar con una serie histórica continua.

3.2.1 Fuentes de los datos

En primera instancia se había comenzado a trabajar con los datos obtenidos de Kaggle para comenzar a experimentar con ellos. Y dado que son de ventas de vehículos BMW de segunda mano en Europa y parecían elaborados sintéticamente, permitieron comenzar a explorar las aplicaciones del Aprendizaje Automático utilizando Python3.

Ilustración 23: Dataset de ventas por modelo, año, color y región de Kaggle



Fuente: Kaggle

No obstante, se optó por una fuente más fiable que en efecto, pudiese representar las ventas o matriculaciones de BMW en Europa como región sobre la que se trabajaría. Por lo que se obtuvieron todos los datos de ventas de BMW de los reportes oficiales de BMW. Concretamente, de los BMW Group Annual Report y los BMW Group Report como se puede observar en la tabla inferior.

Tabla 2: Reportes Anuales de Ventas de BMW

Año	Informe
2005	BMW Group Annual Report 2005
2006	BMW Group Annual Report 2006
2007	BMW Group Annual Report 2007
2008	BMW Group Annual Report 2008 (<i>Investor Relations Archive</i>)
2009	BMW Group Annual Report 2009 (<i>Investor Relations Archive</i>)
2010	BMW Group Annual Report 2010 (<i>Investor Relations Archive</i>)
2011	BMW Group Annual Report 2011 (<i>Investor Relations Archive</i>)
2012	BMW Group Annual Report 2012 (<i>Investor Relations Archive</i>)
2013	BMW Group Annual Report 2013 (<i>Investor Relations Archive</i>)
2014	BMW Group Annual Report 2014 (<i>Investor Relations Archive</i>)
2015	BMW Group Annual Report 2015 (<i>Investor Relations Archive</i>)
2016	BMW Group Annual Report 2016 (<i>Investor Relations Archive</i>)
2017	BMW Group Annual Report 2017 (<i>Investor Relations Archive</i>)
2018	BMW Group Annual Report 2018 (<i>Investor Relations Archive</i>)
2019	BMW Group Annual Report 2019 (<i>Investor Relations Archive</i>)
2020	BMW Group Report 2020
2021	BMW Group Report 2021 (<i>Investor Relations Archive</i>)
2022	BMW Group Report 2022 (<i>Investor Relations Archive</i>)
2023	BMW Group Report 2023 (<i>Investor Relations Archive</i>)
2024	BMW Group Report 2024
2025	BMW Group Report 2025

Fuente: *Elaboración Propia*

Es importante denotar que todos estos estudios se están llevando a cabo en diferentes cuadernos Jupyter. Por lo que, para trabajar con todos estos datos, se ha creado una carpeta personal con los diferentes datos con los que se trabajará en este proyecto.

En el caso de las ventas de BMW, se ha cargado desde Google Drive el archivo “ventas_bmw.csv” mediante la librería pandas. Y se han almacenado los datos en un *DataFrame* para facilitar su posterior tratamiento y análisis. Finalmente, se ha mostrado el contenido completo del conjunto de datos para comprobar su estructura y verificar la información disponible:

Ilustración 24: Cargando el archivo con las ventas de BMW

```
import pandas as pd

ruta = "/content/drive/MyDrive/TFG_BMW/datasets/ventas_bmw.csv"
df = pd.read_csv(ruta)

with pd.option_context('display.max_rows', None, 'display.max_columns', None):
    display(df)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

El resultado obtenido, tiene el aspecto siguiente:

Tabla 3: Ventas de BMW por año en Europa

Índice	Año	Ventas BMW (unidades)
0	2005	1.126.768
1	2006	1.185.088
2	2007	1.276.793
3	2008	1.202.239
4	2009	1.068.770
5	2010	1.224.280
6	2011	1.380.384
7	2012	1.540.085
8	2013	1.655.138
9	2014	1.811.719
10	2015	1.905.234
11	2016	2.003.359
12	2017	2.088.283
13	2018	2.125.026
14	2019	2.168.516
15	2020	2.028.659
16	2021	2.213.795
17	2022	2.100.692
18	2023	2.253.835
19	2024	2.200.177
20	2025	2.169.761

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

3.2.2 Análisis exploratorio de las ventas de BMW

Para complementar el análisis posterior y comprender mejor los resultados obtenidos, conviene también elaborar un análisis exploratorio de los datos de ventas de BMW en Europa. Para ello, a lo largo de este apartado se llevarán a cabo diferentes tipos de estudios para identificar la tendencia general, la variabilidad interanual y los principales acontecimientos económicos que hayan podido influir en la evolución de las ventas a lo largo del período de estudio.

3.2.2.1 Estadísticos importantes y variación absoluta y porcentual

El primer paso para llevar esto a cabo, será el de calcular la variación absoluta y porcentual interanual de las matriculaciones. Con el fin de identificar de forma más clara los años en los que se produjeron incrementos o descensos significativos en las ventas, facilitando posteriormente su interpretación en relación con el contexto macroeconómico y los acontecimientos relevantes ocurridos durante cada ejercicio.

Ilustración 25: Análisis de la variación absoluta y porcentual

```
df["Ventas_BMW_Unidades"].describe()
import matplotlib.pyplot as plt

df["Variacion_Absoluta"] = df["Ventas_BMW_Unidades"].diff()
df["Variacion_Porcentual"] = df["Ventas_BMW_Unidades"].pct_change() * 100

with pd.option_context('display.max_rows', None, 'display.max_columns', None):
    display(df)
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Como se puede observar en la captura de pantalla anterior, para llevar esto a cabo, se han calculado diferentes medidas estadísticas mediante la función describe(). Y el resultado obtenido con su respectiva interpretación es el siguiente:

Tabla 4: Estadísticos relevantes

Estadístico	Valor	Interpretación
count	21	Hay 21 observaciones, correspondientes al periodo 2005-2025.
mean	1.748.981	Las ventas medias fueron de aproximadamente 1.748.981 unidades anuales.
std	431.996	Existe una variabilidad considerable de las ventas respecto a la media.
min	1.068.770	El mínimo de ventas se registró en 2009.
25 %	1.276.793	El 25 % de los años presentó ventas inferiores o iguales a 1.276.793 unidades.
50 %	1.905.234	La mediana fue de 1.905.234 unidades.
75 %	2.125.026	El 75 % de los años presentó ventas inferiores o iguales a 2.125.026 unidades.
max	2.253.835	El máximo de ventas se alcanzó en 2023.

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar, el conjunto de datos está compuesto por 21 observaciones correspondientes al periodo 2005-2025. Y durante este periodo, las ventas medias de BMW fueron de 1.748.981 unidades anuales, mientras que la mediana se situó en 1.905.234 unidades. Es importante recalcar que la mediana fue mayor que la media dado que los primeros años del periodo, especialmente durante 2008-2010, las ventas fueron considerablemente menores y terminan reduciendo la media.

La desviación estándar es de 431.996 unidades. Lo que refleja una variabilidad considerable en las ventas a lo largo del periodo. Esto quiere decir las ventas anuales presentan una diferencia considerable respecto a la media que es de 1.748.981 unidades. En otras palabras, las ventas no se mantienen alrededor de un valor estable, sino que han experimentado cambios importantes a lo largo del periodo 2005-2025.

Por otro lado, el valor mínimo registrado fue de 1.068.770 unidades en 2009 que podría deberse a la influencia de la crisis financiera de 2008, mientras que el máximo alcanzó las 2.253.835 unidades en 2023.

Por otro lado, respecto a las variaciones de las ventas entre años consecutivos, tanto en términos absolutos como porcentuales, se han obtenido los siguientes resultados:

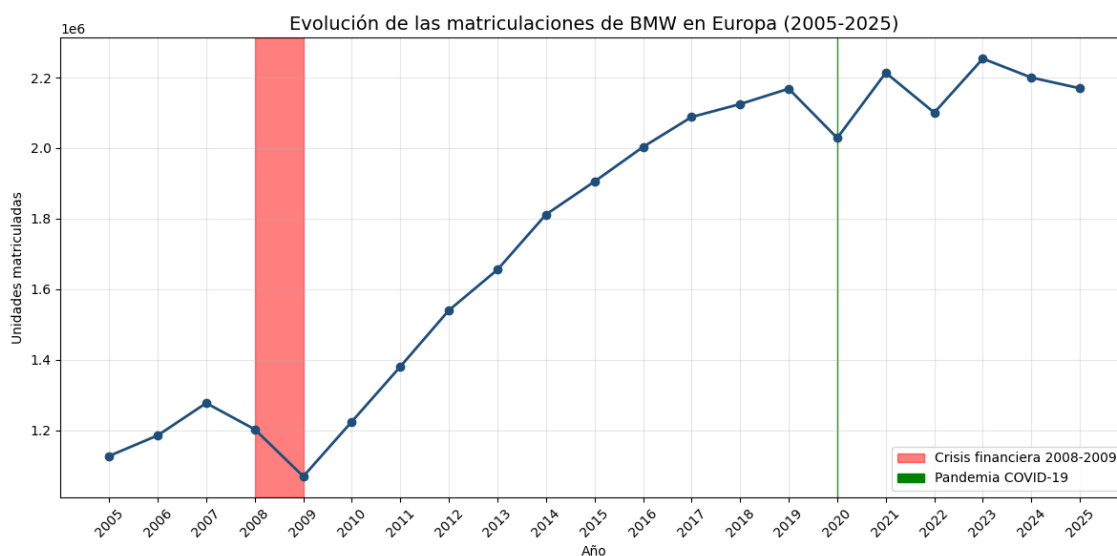
Tabla 5: Variación absoluta y porcentual interanual de las ventas de BMW

Año	Ventas BMW (unidades)	Variación absoluta	Variación porcentual
2005	1.126.768	—	—
2006	1.185.088	58.320	5,18 %
2007	1.276.793	91.705	7,74 %
2008	1.202.239	-74.554	-5,84 %
2009	1.068.770	-133.469	-11,10 %
2010	1.224.280	155.510	14,55 %
2011	1.380.384	156.104	12,75 %
2012	1.540.085	159.701	11,57 %
2013	1.655.138	115.053	7,47 %
2014	1.811.719	156.581	9,46 %
2015	1.905.234	93.515	5,16 %
2016	2.003.359	98.125	5,15 %
2017	2.088.283	84.924	4,24 %
2018	2.125.026	36.743	1,76 %
2019	2.168.516	43.490	2,05 %
2020	2.028.659	-139.857	-6,45 %
2021	2.213.795	185.136	9,13 %
2022	2.100.692	-113.103	-5,11 %
2023	2.253.835	153.143	7,29 %
2024	2.200.177	-53.658	-2,38 %
2025	2.169.761	-30.416	-1,38 %

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

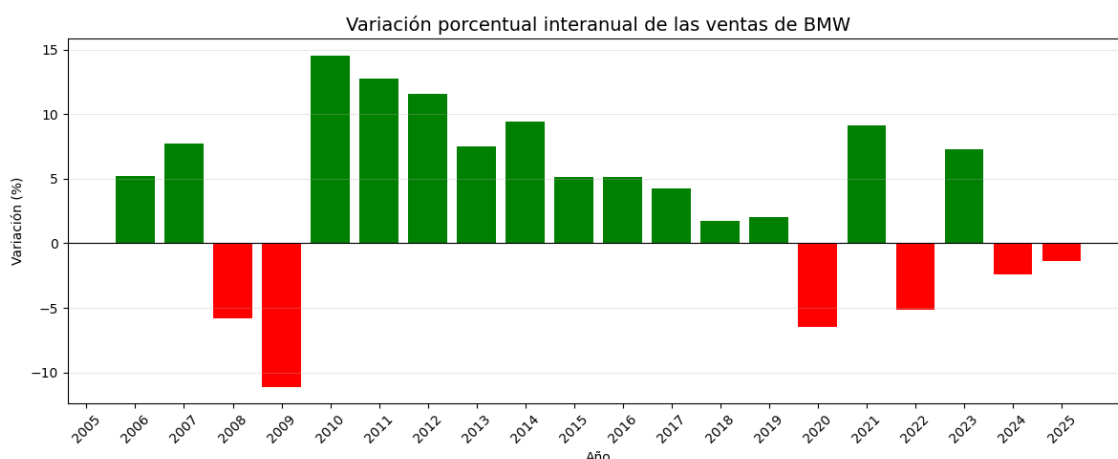
Si analizamos gráficamente estos datos, se obtendrá el resultado siguiente:

Ilustración 26: Evolución de las matriculaciones de BMW en Europa (2005-2025)



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Ilustración 27: Representación de la variación porcentual de las ventas de BMW



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Se ha optado por representar gráficamente los resultados ya que facilitan el análisis de la tabla anterior. Donde se puede observar que en general, la evolución de las matriculaciones muestra una tendencia general creciente, aunque interrumpida por diferentes periodos de descenso. Marcados en el gráfico de la evolución de las ventas con los colores rojo y verde. Siendo el rojo para simbolizar la caída de ventas entre 2008 y 2009 y el verde para representar la caída que ocurre en 2020 debido a la pandemia del Covid-19. No obstante, aunque se puede observar gráficamente, se calculará el CAGR o también conocido como Tasa de Crecimiento Anual Compuesto para demostrar que la tendencia es en efecto, creciente.

Se puede observar que entre 2008 y 2009 se produjo la mayor caída de la serie, coincidiendo con la crisis financiera internacional, alcanzándose en 2009 el mínimo absoluto de toda la serie en 1.068.770 unidades. Ya se observa que en 2008 las matriculaciones disminuyen un 5,84%, mientras que al año siguiente, la caída alcanza el 11,10% lo que sitúa las matriculaciones en este valor.

Posteriormente, en 2010 el porcentaje de ventas aumenta en gran medida +14,55%, alcanzando las 1.224.280 unidades vendidas ya que se produce una recuperación económica, lo que lo hace ver como un incremento muy alto. El cual lleva a un crecimiento de las ventas sostenido hasta alcanzar las 1.811.719 unidades en 2014 debido a la mejora del entorno económico europeo y la reactivación del mercado del automóvil.

Después de este periodo, continúa un periodo de estabilidad económica y entre 2015 y 2019 las matriculaciones continúan aumentando, aunque a un ritmo más moderado, con tasas comprendidas aproximadamente entre el 2% y el 5% anual.

No obstante, En 2020 se produce una nueva ruptura de la tendencia creciente debido a la crisis del Covid-19. Lo que obliga a millones de personas a confinarse en sus casas y a paralizar parcialmente la economía global.

Esto lleva a que las matriculaciones descieran un 6,45%, pasando de 2.168.516 a 2.028.659 unidades de vehículos vendidos.

Posteriormente, llega la recuperación de la crisis del Covid-19 en 2021. El cual presenta el mayor incremento absoluto de toda la serie histórica (+185.136 vehículos) y un crecimiento interanual del 9,13%. Debido a que las medidas de cuarentena fueron retiradas y con ello aumentó el Consumo y la Demanda de diferentes bienes. Por ello el incremento ha sido tan alto. Ya que la economía se había paralizado parcialmente.

En 2022 vuelve a observarse una disminución del 5,11% debido a que las ventas han vuelto a sus valores usuales y en 2023 se alcanza el mayor volumen de matriculaciones de todo el periodo analizado, con 2.253.835 vehículos, tras crecer un 7,29% respecto al año anterior.

En los años posteriores, entre 2024 y 2025 las matriculaciones experimentan ligeras disminuciones del 2,38% y 1,38%, respectivamente. Aunque pueden ser interpretadas como una fase de estabilización del mercado tras varios años de crecimiento intenso, sin alterar significativamente la tendencia general de largo plazo.

Ilustración 28: Tasa de Crecimiento Anual Compuesto (CAGR)

```
In [ ]: # Tasa de crecimiento anual compuesto (CAGR)
n_años = df["Año"].iloc[-1] - df["Año"].iloc[0]
cagr = ((df["Ventas_BMW_Unidades"].iloc[-1] / df["Ventas_BMW_Unidades"].iloc[0]) ** (1 / n_años) - 1) * 100
print(f"CAGR {df['Año'].iloc[0]}-{df['Año'].iloc[-1]}: {cagr:.2f}% anual")

CAGR 2005-2025: 3.33% anual
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Para afirmar si la tendencia de la serie es creciente, se ha optado por utilizar el CAGR o Tasa de Crecimiento Anual Compuesto, ya que permite calcular la tasa media de crecimiento anual de las matriculaciones durante un periodo determinado, considerando el valor inicial y el valor final de la serie.

Esta medida resulta especialmente útil en este caso, ya que las matriculaciones presentan variaciones importantes entre determinados años, como consecuencia de diferentes acontecimientos económicos. De esta forma, es posible resumir la evolución global de la serie mediante una única tasa, evitando que las fluctuaciones interanuales dificulten la interpretación de la tendencia a largo plazo.

Como se puede observar, se ha obtenido un CAGR de un 3,3% anual. Lo que quiere decir que en términos agregados, las matriculaciones han experimentado un crecimiento medio anual del 3,3 % durante el periodo 2005-2025. Es decir, que a pesar de las fluctuaciones y descensos registrados en determinados años, la tendencia general de la serie es creciente.

3.3 Variables macroeconómicas seleccionadas

Es importante denotar que los modelos de predicción suelen mejorar sus predicciones cuando cuentan con mayores cantidades de datos. Por ello, para reducir las probabilidades de que existan sobreajustes u overfitting, se ha decidido seleccionar una serie de variables macroeconómicas que según la literatura económica podrían influir en las matriculaciones anuales de BMW en Europa. Ya que las ventas dependen de una gran cantidad de variables. Dichas variables serán expuestas a lo largo de este apartado.

3.3.1 Tasa de desempleo

La tasa de desempleo, obtenida de Eurostat es un indicador directo de la capacidad adquisitiva y la confianza de los hogares. Ya que representa el porcentaje de la población activa que se encuentra desempleada.

Ilustración 29: Tasa de desempleo



Fuente: Fundación BBVA

Un aumento del desempleo reduce la renta disponible y genera incertidumbre sobre ingresos futuros, lo que tiende a posponer decisiones de compra de bienes de alto valor y de carácter no esencial, como son los vehículos, especialmente en el segmento premium en el que opera BMW como marca. Por ello es por lo que es importante a tener en cuenta.

3.3.2 Crecimiento del PIB

Obtenido de DatosMacro, el crecimiento del Producto Interior Bruto o PIB, refleja la evolución y la salud general de la economía.

Ilustración 30: Producto Interior Bruto



Fuente: UNIR Ecuador

Es importante tenerlo en cuenta, pues existe una relación históricamente positiva entre el ciclo económico y la demanda de automóviles. Ya que, durante las fases de expansión económica, aumentan la renta y la inversión de familias y empresas y por tanto, la adquisición de vehículos.

Por el contrario, en periodos de recesión, como los registrados en 2009 y 2020, la baja actividad económica y la reducción de la capacidad de gasto pueden provocar una contracción de la demanda de automóviles y de otros bienes. Es similar en este aspecto al desempleo, ya que afecta a la capacidad de adquirir bienes duraderos.

3.3.3 Tipo de depósito del BCE

Proveniente del Banco Central Europeo o BCE, el tipo de depósito del BCE es uno de los principales instrumentos de política monetaria y determina las condiciones de financiación de la economía.

Se considera que podría afectar a las matriculaciones de vehículos ya que cuando los tipos de interés son bajos, el coste del crédito al consumo y de la financiación de vehículos disminuye, lo que puede favorecer la adquisición de automóviles. No obstante, un endurecimiento de la política monetaria, incrementa el coste de la financiación y puede reducir la demanda de vehículos adquiridos mediante crédito.

3.3.4 Índice Armonizado de Precios al Consumo

Obtenida del Banco Central Europeo o BCE, El Índice Armonizado de Precios al Consumo (HICP) mide la evolución del nivel general de precios en la Eurozona.

Ilustración 31: Índice Armonizado de Precios al Consumo (HICP)



Fuente: Channel Partner

Es importante a tener en cuenta, ya que una inflación elevada reduce el poder adquisitivo real de los consumidores y esto puede limitar su capacidad para adquirir bienes de elevado valor, como son este caso, los vehículos.

Además, el aumento de los precios puede incrementar los costes de producción, especialmente en materias primas y componentes, repercutiendo sobre el precio final de los automóviles.

3.3.5 Producción industrial

Proveniente de Eurostat, la producción industrial refleja el nivel de actividad del sector industrial europeo y está estrechamente relacionada con la evolución del sector automovilístico y de sus cadenas de suministro. Pues, una disminución de la producción industrial puede ser indicativa de una menor actividad económica o de posibles restricciones en la oferta, lo que puede afectar tanto a la fabricación como a la comercialización de vehículos.

Ilustración 32: Producción Industrial



Fuente: elEconomista

3.3.6 Crecimiento de las ventas minoristas

Proveniente de Eurostat, el crecimiento de las ventas minoristas actúa como un indicador general del dinamismo del consumo privado en Europa. Ya que, al reflejar la evolución del comercio minorista en su conjunto, permite aproximar las tendencias de gasto de los hogares.

Ilustración 33: Crecimiento de las ventas minoristas



Fuente: Marketing Insider Review

Por tanto, podría resultar importante, ya que una evolución positiva de esta variable puede asociarse con una mayor disposición de los consumidores a realizar compras de elevado valor, como la adquisición de un vehículo.

No obstante, una evolución negativa puede reflejar una menor actividad del consumo privado y en consecuencia, una posible reducción de la demanda de automóviles.

3.3.7 Confianza del Consumidor

Obtenido de la Comisión Europea, el índice de confianza del consumidor es un indicador adelantado (leading indicator) del ciclo económico, ya que refleja las expectativas de los hogares sobre su situación económica futura y su disposición al consumo.

De esta manera, una mayor confianza puede favorecer el gasto de los consumidores, mientras que una disminución puede llevar a posponer decisiones de compra de bienes de elevado valor, como los vehículos. Especialmente los premium. Por este motivo, su inclusión podría resultar especialmente relevante para analizar la evolución de la demanda de automóviles.

3.3.8 Compensación por empleado

Obtenida de la AMECO (Comisión Europea), la compensación por empleado permite aproximar la evolución de los salarios nominales en la economía europea y, por tanto, de la capacidad adquisitiva de los consumidores.

Ilustración 34: Compensación por Empleado



Fuente: ShiftBase

Esto podría resultar especialmente interesante, ya que un incremento sostenido de esta variable puede favorecer a la capacidad de los hogares para adquirir bienes de elevado valor, como los vehículos, especialmente en el segmento premium en el que opera BMW.

Por el contrario, un menor crecimiento de la compensación puede limitar la capacidad de gasto de los consumidores y reducir su disposición a realizar este tipo de adquisiciones.

3.3.9 Precio del Petróleo de Brent

Obtenido del U.S. Energy Information Administration (EIA), el precio del petróleo Brent es un indicador que refleja el precio de referencia del petróleo crudo Brent en los mercados internacionales.

Ilustración 35: Precio del Petróleo de Brent



Fuente: El Periódico de la Energía

Es importante tenerlo en cuenta ya que puede influir directamente en el coste de utilización de los vehículos, especialmente a través del precio del combustible. Por lo que puede afectar a las decisiones de compra de los consumidores.

Pues un aumento del precio del petróleo incrementa el coste asociado al uso de los vehículos y puede reducir la demanda. Además, esta variable refleja parte del contexto energético y geopolítico internacional y puede influir tanto en los costes de producción como en la confianza de los consumidores.

3.3.10 Euríbor a 12 meses

Extraído de EuríborDiario / Banco Sabadell, el Euríbor a 12 meses es un indicador de los tipos de interés del mercado interbancario europeo y resulta relevante para analizar el coste de la financiación.

Ilustración 36: Euríbor a 12 meses



Fuente: BankinterCard

Su evolución puede influir en el coste de los préstamos destinados a la adquisición de vehículos, ya que un aumento del Euríbor puede encarecer la financiación y reducir la capacidad o disposición de los consumidores para realizar estas compras.

Esta relación puede ser especialmente relevante en el segmento premium en el que opera BMW, donde las operaciones de financiación y leasing tienen un peso significativo.

3.3.11 Población total

Proveniente de Eurostat, la población total es una variable estructural que permite aproximar el tamaño potencial del mercado automovilístico.

Aunque sus variaciones interanuales son reducidas en comparación con otros indicadores macroeconómicos, su evolución resulta relevante para analizar el crecimiento de la demanda agregada de vehículos a largo plazo. Además, permite contextualizar las matriculaciones en función del tamaño de la población, por ejemplo, mediante el cálculo de matriculaciones por habitante.

Ilustración 37: Población Total



Fuente: Futuribles

3.3.12 Tipo de cambio EUR/USD

El tipo de cambio EUR/USD representa la relación entre el valor del euro y el dólar estadounidense.

Ilustración 38: Tipo de Cambio EUR/USD



Fuente: Top Forex Brokers

Esta variable resulta relevante para BMW debido al carácter internacional de la compañía, ya que sus actividades comerciales y productivas se desarrollan tanto en la zona euro como en mercados internacionales y las variaciones del tipo de cambio pueden afectar a la competitividad de la empresa y a los costes asociados a operaciones realizadas en dólares. Lo que puede tener un impacto indirecto sobre sus resultados y sobre su estrategia comercial.

3.4 Cobertura geográfica de las variables macroeconómicas

Al igual que se ha explicado anteriormente, todas las variables corresponden a ámbitos geográficos diferentes. Aunque sus respectivos indicadores pueden estar relacionados con la evolución de las matriculaciones de BMW en Europa.

Por ello, el ámbito geográfico de cada variable se tendrá en cuenta durante el análisis posterior, donde se estudiará su posible relación con las ventas de BMW mediante el análisis exploratorio y los modelos de aprendizaje automático.

Los ámbitos geográficos a los que pertenece cada variable son los que se encuentran en la siguiente tabla:

Tabla 6: Cobertura geográfica de las variables macroeconómicas

Ámbito	Variables
UNIÓN EUROPEA (UE-27)	Unemployment_Rate, Industrial_Production, Retail_Sales_Growth, Consumer_Confidence, Compensation_Per_Employee, Population_Total
EUROZONA	GDP_Growth, Deposit_Facility, HICP, Euribor_12M
INTERNACIONAL	Brent_Oil_Price, EUR_USD

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

3.5 Estudio de importancia de las variables macroeconómicas

A continuación, se llevará a cabo un análisis exploratorio de las variables macroeconómicas anteriores, con el objetivo de analizar su comportamiento, identificar posibles relaciones entre ellas y estudiar su relación con las matriculaciones de BMW.

3.5.1 Análisis de las variables macroeconómicas

En este apartado, se mostrarán y explicarán los datos y sus tendencias. Por ello, en primer lugar, se cargará el dataset con todas las variables macroeconómicas.

Ilustración 39: Carga de las variables macroeconómicas

```
import pandas as pd

ruta = "/content/drive/MyDrive/TFG_BMW/datasets/bmw_dataset_variables_macro.csv"
df = pd.read_csv(ruta)

with pd.option_context('display.max_rows', None, 'display.max_columns', None):
    display(df)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Tabla 7: Variables Macroeconómicas y sus valores

Año	Unemployment Rate	GDP_Growth	Deposit_Facility	HICP	Industrial_Production	Retail_Sales_Growth	Consumer_Confidence	Compensation_Per_Employee	Brent_Oil_Price	EUR_USD	Euribor_12M	Population_Total
2005	8,9	1,7	1,25	2,18	153,2	0,03	-4,2	31,22	54,57	1,2441	2,19	434.585.887
2006	8,2	3,2	2,5	2,19	147,7	0,04	-3,5	31,95	65,16	1,2556	3,08	436.041.018
2007	7,2	2,9	3	2,13	145,5	0,03	-2,1	32,75	72,44	1,3705	4,24	437.514.477
2008	7,1	0,4	2	3,3	148	0,01	-10,8	33,86	96,94	1,4708	4,63	439.074.280
2009	9	-4,4	0,25	0,29	144,5	-0,03	-21,7	34,41	61,74	1,3948	1,31	440.467.898
2010	9,7	2,1	0,25	1,62	127,2	0,01	-11,2	35,2	79,61	1,3257	0,81	441.160.594
2011	9,7	1,7	0,25	2,7	128,9	0,01	-15,8	35,97	111,26	1,392	1,39	440.526.112
2012	10,5	-1	0	2,5	125,5	0	-20,7	36,55	111,63	1,2848	0,57	441.280.191
2013	10,9	-0,2	0	1,35	120,7	0	-13,7	37,13	108,56	1,3281	0,22	441.739.935
2014	10,2	1,4	-0,2	0,44	116,5	0,01	-10,5	37,64	98,97	1,3285	0,21	442.235.565
2015	9,4	2,1	-0,3	0,18	109	0,02	-6,1	38,22	52,32	1,1095	0	442.883.289
2016	8,6	1,8	-0,4	0,24	100	0,02	-5,3	38,71	43,73	1,1069	-0,08	443.954.935
2017	7,7	2,6	-0,4	1,53	94,6	0,04	-2,5	39,44	54,19	1,1297	-0,19	444.609.939
2018	6,8	1,8	-0,4	1,76	92,4	0,03	-1,8	40,32	71,31	1,181	-0,17	445.227.547
2019	6,3	1,6	-0,5	1,19	88,1	0,03	-6,4	41,23	64,21	1,1195	-0,22	446.060.538
2020	7,1	-6	-0,5	0,25	82,9	0	-15,9	41,08	41,84	1,1413	-0,3	446.923.963
2021	7	6,4	-0,5	2,6	76	0,07	-7	42,88	70,68	1,1833	-0,26	445.767.723
2022	6,2	3,6	2	8,37	77	0,06	-22,9	44,84	100,94	1,053	0,34	445.999.571
2023	5,9	0,4	4	5,46	77,6	0,01	-16,3	47,23	82,41	1,0813	3,86	447.805.685
2024	5,9	1	3	2,37	71,5	-0,07	-12,1	49,36	80,53	1,081	3,68	449.489.862
2025	6	1,2	2	2,1	69	0,03	-13	51,27	68,89	1,08	2,45	451.284.558

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

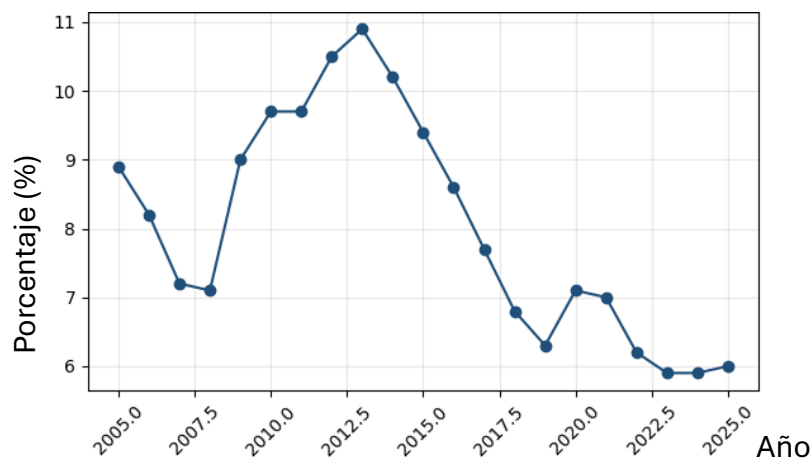
Como se puede observar, el conjunto de datos abarca el período comprendido entre 2005 y 2025, proporcionando un total de 21 observaciones anuales.

Las variables seleccionadas recogen diferentes dimensiones del entorno económico que pueden influir en la evolución de las matriculaciones de BMW, como son el crecimiento económico o la política monetaria.

Gráficamente, presentan las características siguientes:

1. **Tasa de desempleo (Unemployment_Rate):** presenta un comportamiento cíclico a lo largo del periodo analizado. Pues entre 2005 y 2008 se observa un descenso progresivo, pasando del 8,9 % al 7,1 % asociado al periodo de expansión económica previo a la crisis financiera. Posteriormente, la tasa aumenta de forma notable hasta alcanzar su máximo en 2013, con un 10,9 % coincidiendo con el periodo de crisis de deuda soberana. Finalmente, se inicia una tendencia descendente que se mantiene hasta 2025, cuando alcanza el 6,0 % aunque se observa un ligero repunte entre 2020 y 2021, pasando del 7,1 % al 7,0 % en el contexto de la pandemia de COVID-19.

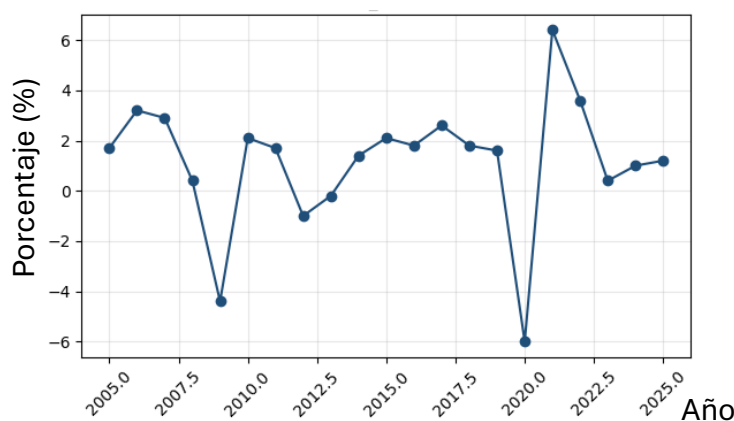
Ilustración 40: Evolución de la Tasa de Desempleo



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

2. **Crecimiento del PIB (GDP_Growth):** presenta oscilaciones entre valores positivos y negativos a lo largo del periodo analizado. Destacando especialmente dos mínimos, registrados en 2009 y 2020, seguidos en ambos casos de una rápida recuperación, especialmente en el periodo posterior a 2020. Durante el resto del periodo, el crecimiento del PIB se mantiene generalmente en niveles más moderados, situándose aproximadamente entre el 0 % y el 3 %.

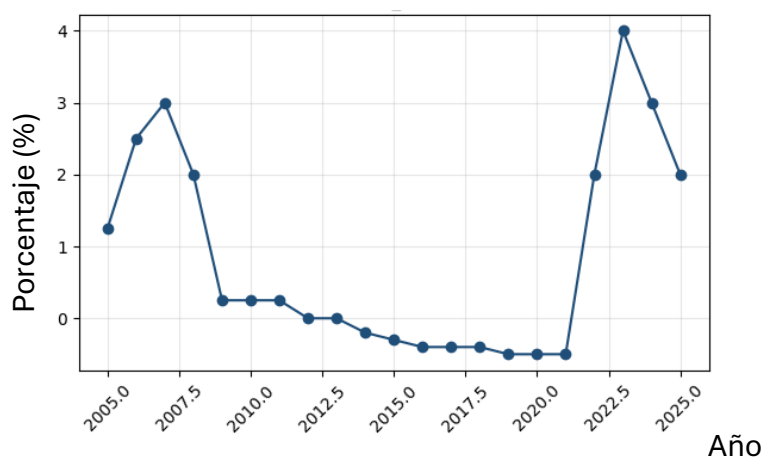
Ilustración 41: Evolución del Crecimiento del PIB



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

- 3. Tipo de depósito del BCE (Deposit_Facility):** La variable presenta una evolución en forma de “U” a lo largo del periodo analizado. Ya que parte de valores positivos en 2005 y desciende progresivamente hasta alcanzar valores negativos, que se mantienen aproximadamente entre 2015 y 2021. Finalmente, a partir de 2022, experimenta un incremento pronunciado, alcanzando su máximo histórico en 2023, seguido de un descenso durante 2024 y 2025. Dándole esa forma tan característica.

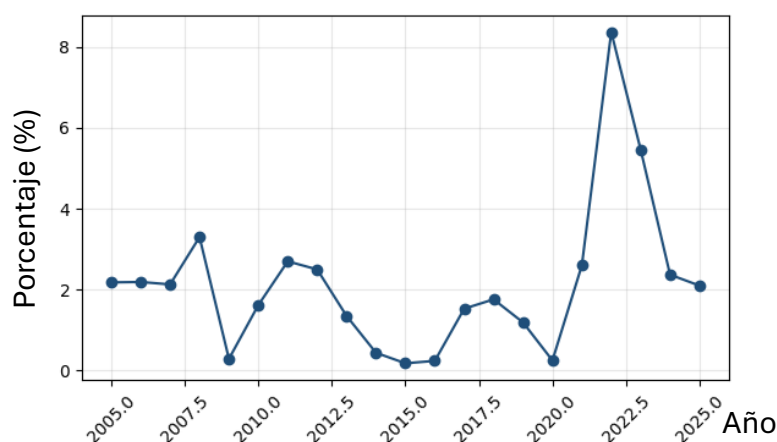
Ilustración 42: Evolución del Tipo de Depósito del BCE



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

- 4. Índice Armonizado de Precios al Consumo (HICP):** se mantiene relativamente estable durante la mayor parte del periodo, con oscilaciones moderadas de entre el 0 % y el 3,5 %. Sin embargo, en 2022 se produce un incremento muy pronunciado y aislado, alcanzando un valor considerablemente superior al resto de la serie. Finalmente, la inflación disminuye de forma progresiva durante 2023-2025, aproximándose nuevamente a los niveles observados antes del repunte.

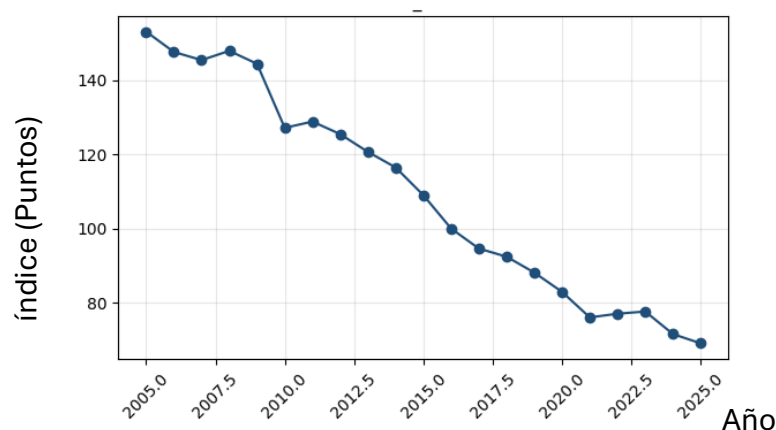
Ilustración 43: Evolución del HICP



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

- 5. Índice de Producción Industrial (Industrial_Production):** presenta una tendencia sostenida y descendente de forma general, pasando de 153,2 puntos en 2005 a 69,0 puntos en 2025. Aunque se producen algunas variaciones puntuales, la evolución general es claramente descendente, situándose en 2025 en menos de la mitad del nivel registrado al inicio de la serie.

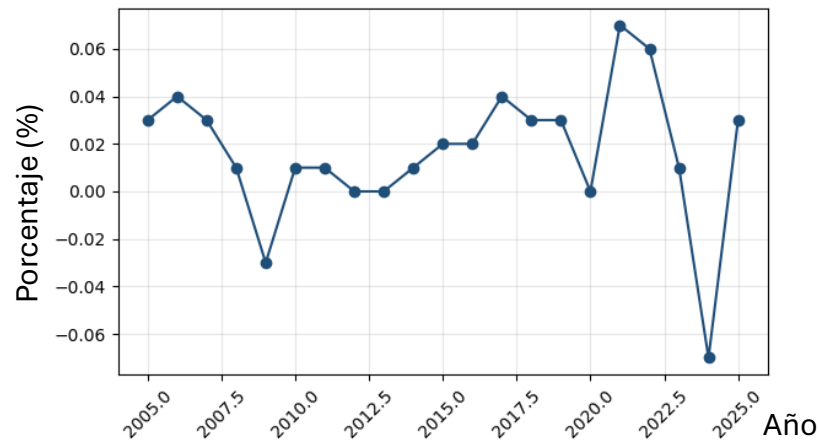
Ilustración 44: Evolución del Índice de Producción Industrial



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

- 6. Crecimiento de las ventas minoristas (Retail_Sales_Growth):** se mantiene dentro de un rango reducido durante la mayor parte del periodo, con pequeñas variaciones alrededor de cero. Destacan especialmente la caída registrada en 2009 (-0,03) y la mayor disminución de la serie en 2024 (-0,07). Por el contrario, en 2021 se alcanza el valor máximo, con un crecimiento del 0,07 coincidiendo con la recuperación del consumo posterior a la pandemia.

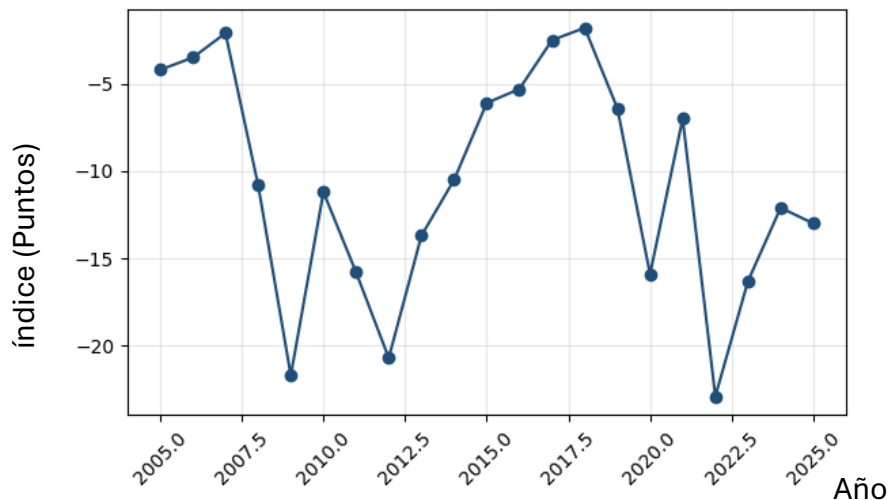
Ilustración 45: Evolución de las ventas minoristas



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

- 7. Índice de Confianza del Consumidor (Consumer_Confidence):** presenta un comportamiento irregular, caracterizado por descensos y recuperaciones pronunciadas. Se puede observar que en 2009 alcanza un valor de -21,7 debido a la crisis de 2008, mientras que en 2012 se sitúa en -20,7. Posteriormente, vuelve a descender hasta -15,9 en 2020 y alcanza su valor mínimo de toda la serie en 2022, con -22,9. En los años siguientes se observa una recuperación, situándose en -16,3 en 2023, -12,1 en 2024 y -13,0 en 2025.

Ilustración 46: Evolución del Índice de Confianza del Consumidor

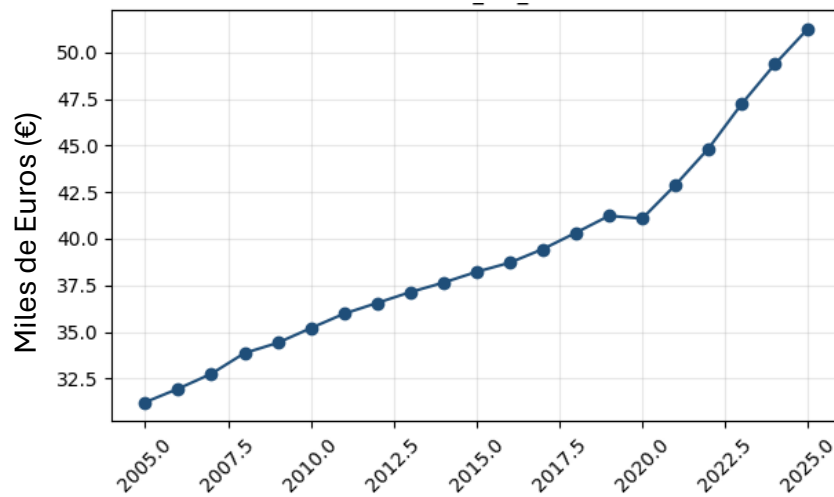


Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

- 8. La compensación por empleado (Compensation_Per_Employee):** presenta una tendencia claramente creciente durante todo el periodo analizado y presenta una evolución relativamente, pasando de 31,22 miles

de euros en 2005 a 51,27 miles de euros en 2025. No se observan grandes retrocesos interanuales, aunque en 2020 se produce una ligera disminución, pasando de 41,23 en 2019 a 41,08 miles de euros. Y finalmente, a partir de 2021, la tendencia vuelve a ser creciente hasta alcanzar el máximo de la serie en 2025.

Ilustración 47: Evolución de la Compensación por empleado

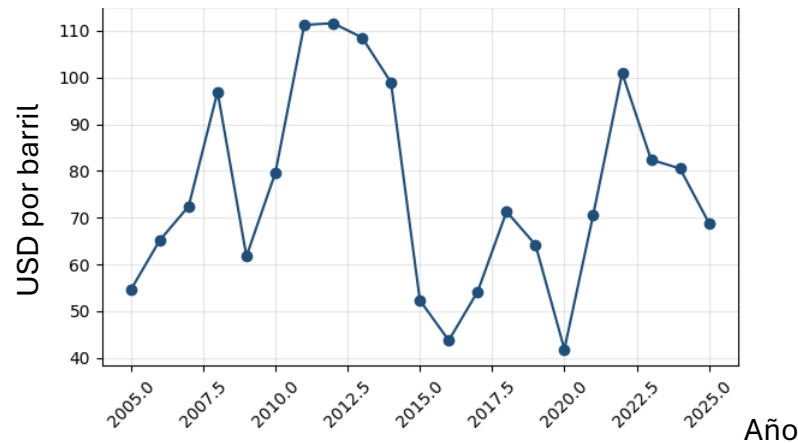


Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Año

- Precio del petróleo Brent (Brent_Oil_Price):** siendo una de las variables más volátiles del conjunto. En 2005 presenta un valor de 54,57 USD por barril y aumenta progresivamente hasta alcanzar 111,26 USD en 2011 y 111,63 USD en 2012. Posteriormente, experimenta una fuerte reducción hasta situarse en 43,73 USD en 2016. Y en 2020 alcanza 41,84 USD, uno de los valores más bajos de la serie, coincidiendo con la pandemia del Covid-19. Finalmente, en 2022 vuelve a producirse un incremento significativo, alcanzando los 100,94 USD, para posteriormente descender hasta 68,89 USD en 2025.

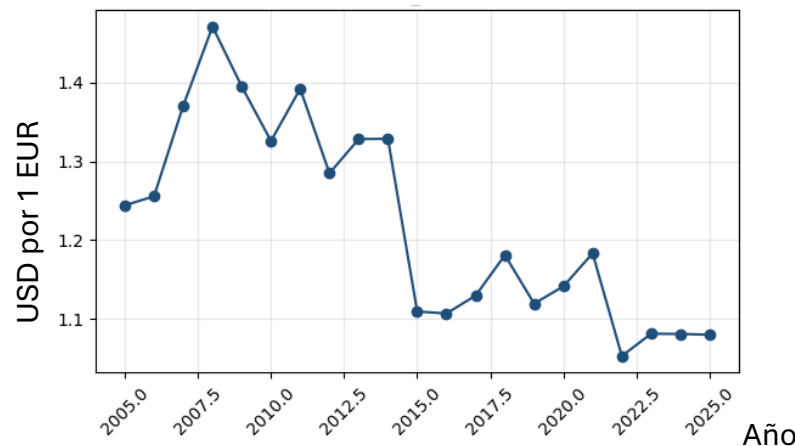
Ilustración 48: Evolución del Precio del petróleo de Brent



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

10. Tipo de cambio EUR/USD (EUR_USD): presenta una tendencia general descendente a lo largo del periodo, aunque acompañada de diferentes fluctuaciones. Se puede observar que en 2005 se sitúa en 1,2441 dólares por euro y alcanza su máximo de 1,4708 en 2008. Posteriormente disminuye, aunque se observan algunos repuntes, como en 2011 (1,3920), 2018 (1,1810) y 2021 (1,1833). Finalmente, alcanza su mínimo en 2022, con 1,0530, manteniéndose posteriormente en valores reducidos de 1,0813 en 2023, 1,0810 en 2024 y 1,0800 en 2025.

Ilustración 49: Evolución del Tipo de Cambio EUR/USD

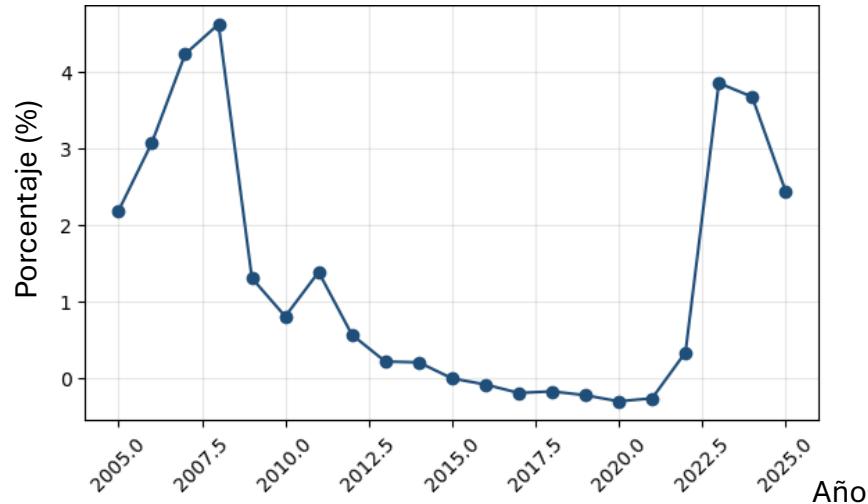


Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

11. Euríbor a 12 meses (Euribor_12M): presenta una evolución similar a la observada en el tipo de depósito del BCE, ya que ambas variables están relacionadas con la evolución de la política monetaria. En 2008 alcanza un valor de 4,63 %, para posteriormente experimentar un descenso progresivo hasta situarse en valores negativos durante varios años, llegando a -0,30 % en 2020. Posteriormente, 2021 se mantiene en -0,26 %, mientras que a partir

de 2022 aumenta de forma pronunciada, pasando a 0,34 % en 2022 y alcanzando su máximo de 3,86 % en 2023. Y finalmente, disminuye hasta 3,68 % en 2024 y 2,45 % en 2025.

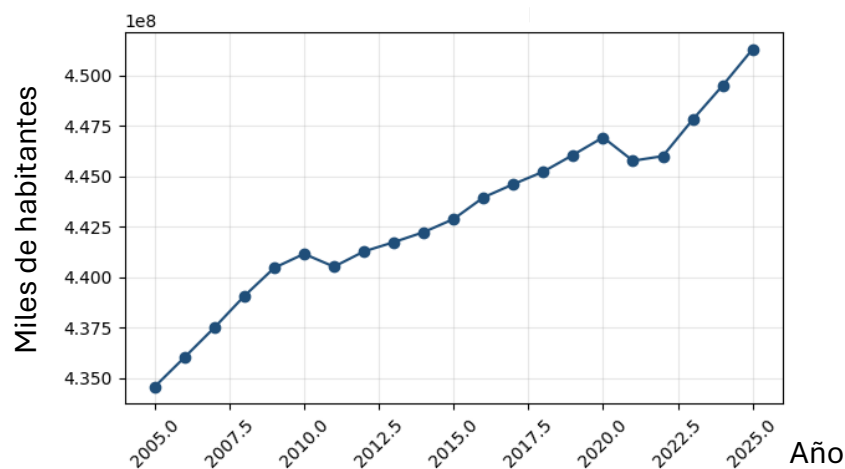
Ilustración 50: Evolución del Euríbor a 12 meses



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

12. Población total (Population_Total): es la variable más estable del conjunto. Presenta un crecimiento gradual y sostenido durante prácticamente todo el periodo analizado, pasando de 434.585.887 habitantes en 2005 a 451.284.558 en 2025, lo que supone un incremento de aproximadamente 16,7 millones de habitantes. Aunque se observan pequeñas fluctuaciones en determinados años, como el descenso de 440.526.112 habitantes en 2011 a 440.526.112 respecto a la tendencia previa, la evolución general es creciente y sin grandes cambios.

Ilustración 51: Evolución de la Población Total



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De los resultados anteriores, se puede observar que las variables de política monetaria (Deposit_Facility, Euribor_12M) presentan una evolución prácticamente idéntica en su forma, lo que podría indicar una correlación muy elevada entre ambas. Aunque esto se comprobará más tarde.

Asimismo, durante los años 2008-2009, 2020 y 2022 aparecen sistemáticamente puntos de inflexión en la mayoría de las variables (GDP_Growth, Unemployment_Rate, Consumer_Confidence, HICP, Deposit_Facility), lo que confirma que estos episodios de crisis afectaron de forma transversal a todo el entorno macroeconómico europeo durante el período analizado.

3.5.2 Análisis de los estadísticos de las variables macroeconómicas

De las variables macroeconómicas anteriores, es importante analizar sus descriptivos para conocer las principales características de los datos antes de incorporarlos al modelo. Lo cual se llevará a cabo a lo largo de todo este apartado.

Si ejecutamos el código inferior, ('display.float_format', lambda x: f'{x:,.2f}') establece que los valores numéricos se muestren con dos decimales y df.describe() calcula los principales estadísticos descriptivos de las variables numéricas del conjunto de datos. Y finalmente, .T transpone la tabla para mostrar cada variable en una fila.

Ilustración 52: Análisis de los estadísticos de las variables macroeconómicas

```
pd.set_option('display.float_format', lambda x: f'{x:,.2f}')
df.describe().T
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De lo que se puede obtener la tabla siguiente:

Tabla 8: Principales Estadísticos de las variables macroeconómicas

Variable	N	Media	Desv. estándar	Mínimo	P25	Mediana	P75	Máximo
Year	21	2.015,00	6,2	2.005,00	2.010,00	2.015,00	2.020,00	2.025,00
Unemployment_Rate	21	8,01	1,63	5,9	6,8	7,7	9,4	10,9
GDP_Growth	21	1,16	2,61	-6	0,4	1,7	2,1	6,4
Deposit_Facility	21	0,82	1,43	-0,5	-0,4	0,25	2	4
HICP	21	2,13	1,89	0,18	1,19	2,1	2,5	8,37
Industrial_Production	21	109,32	28,76	69	82,9	109	128,9	153,2
Retail_Sales_Growth	21	0,02	0,03	-0,07	0,01	0,02	0,03	0,07
Consumer_Confidence	21	-10,64	6,59	-22,9	-15,8	-10,8	-5,3	-1,8
Compensation_Per_Employee	21	39,11	5,56	31,22	35,2	38,22	41,23	51,27
Brent_Oil_Price	21	75,81	21,81	41,84	61,74	71,31	96,94	111,63
EUR_USD	21	1,22	0,13	1,05	1,11	1,18	1,33	1,47
Euribor_12M	21	1,32	1,68	-0,3	-0,08	0,57	2,45	4,63
Population_Total	21	443.077.788,90	4.297.633,91	434.585.887,00	440.526.112,00	442.883.289,00	445.999.571,00	451.284.558,00

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De la tabla anterior se puede obtener una primera visión del comportamiento de las variables macroeconómicas durante el periodo comprendido entre 2005 y 2025.

En primer lugar, se puede observar que GDP_Growth destaca por presentar una elevada variabilidad. Ya que su valor medio se sitúa en torno al 1,16 %, con una desviación estándar de 2,61 puntos porcentuales y valores comprendidos entre -6,00 % y 6,40 %. La amplitud de este intervalo refleja la existencia de situaciones económicas muy diferentes durante el periodo analizado al igual que se puede observar en el gráfico de su evolución temporal. Esta característica permite identificar claramente los distintos ciclos económicos experimentados entre 2005 y 2025, especialmente los asociados a episodios como la crisis financiera de 2008-2009 y la contracción económica provocada por el Covid-19 en 2020.

Por otro lado, Deposit_Facility también destaca por tener una media del 0,82 %, con una desviación estándar de 1,43 puntos porcentuales y valores comprendidos entre -0,50 % y 4,00 %. Lo que refleja los diferentes escenarios de política monetaria existentes durante el periodo, desde una etapa de tipos de interés relativamente elevados hasta el periodo de tipos negativos y finalmente un endurecimiento de la política económica en 2022.

En el caso del HICP, la media alcanza el 2,13 % y la mediana se sitúa en el 2,10 %, valores muy próximos entre sí. Sin embargo, el máximo registrado alcanza el 8,37 %, mientras que el tercer cuartil se sitúa en el 2,50 %. Esta diferencia evidencia la existencia de un episodio excepcional de elevada inflación en los últimos años de la serie, que se corresponde principalmente con el repunte de los precios registrado en 2022. Por tanto, aunque durante la mayor parte del periodo la inflación se mantiene en niveles relativamente moderados, el fuerte incremento registrado en los últimos años introduce una variabilidad considerable en la serie.

Por su parte, Unemployment_Rate presenta una media del 8,01 %, con valores comprendidos entre el 5,90 % y el 10,90 %. Esto se debe a las distintas condiciones del mercado laboral europeo durante el periodo estudiado. De dicha variable se puede observar que el máximo se alcanza en periodos posteriores a la crisis financiera y de deuda europea, mientras que los valores más reducidos corresponden a los últimos años de la serie. Asimismo, Consumer_Confidence muestra una variabilidad significativa, con valores entre -22,90 y -1,80, reflejando las diferentes percepciones de los hogares respecto a la situación económica y sus expectativas a lo largo del periodo.

En lo que respecta a Population_Total, esta variable presenta un comportamiento mucho más estable. Su media asciende a 443.077.788,90 habitantes, mientras que los valores mínimo y máximo son 434.585.887 y 451.284.558 habitantes,

respectivamente. El reducido rango en comparación con el nivel absoluto de la variable es coherente con su carácter estructural, dado que la población de un área geográfica de estas dimensiones experimenta cambios relativamente graduales de un año a otro.

Este tipo de estabilidad también se puede observar en EUR_USD, cuya media es de 1,22 dólares por euro, con valores comprendidos entre 1,05 y 1,47. Aunque el tipo de cambio presenta fluctuaciones a lo largo del periodo, estas se mantienen dentro de un intervalo relativamente acotado en comparación con otras variables analizadas.

Por otro lado, Brent_Oil_Price presenta una media de 75,81 USD por barril y una desviación estándar de 21,81 USD, mientras que sus valores oscilan entre 41,84 y 111,63 USD por barril. Esta amplitud refleja la elevada sensibilidad del precio del petróleo a las condiciones económicas y a distintos acontecimientos internacionales, dando lugar a importantes fluctuaciones durante el periodo analizado como se explica en el apartado de la evolución temporal de la variable.

En lo que respecta a Industrial_Production, ésta presenta también un rango amplio comprendido entre 69,00 y 153,20 puntos. Aunque en este caso, la amplitud observada no responde únicamente a fluctuaciones puntuales, sino también a la propia evolución de la variable a lo largo del periodo, tal y como se observó anteriormente en su representación gráfica.

Finalmente, Retail_Sales_Growth presenta un rango relativamente reducido, entre -0,07 y 0,07 y una desviación estándar de 0,0296. Esta menor dispersión es coherente con la naturaleza de la variable, expresada como una tasa de crecimiento. Y que muestra que las variaciones anuales de las ventas minoristas se mantienen generalmente alrededor de valores próximos a cero, aunque existen determinados años con cambios más pronunciados.

En conjunto, los estadísticos descriptivos muestran que las variables seleccionadas recogen dimensiones económicas diferentes y con distintos grados de variabilidad. Esta heterogeneidad resulta relevante para el análisis posterior, ya que las variables no solo difieren en sus unidades de medida y escalas, sino también en la dinámica temporal que presentan. Por ello, antes de incorporarlas al modelo predictivo será necesario analizar su importancia relativa en apartados siguientes y determinar cuáles aportan una mayor capacidad explicativa sobre la evolución de las matriculaciones de BMW.

3.5.3 Matriz de correlación entre variables macroeconómicas

A continuación, se llevará a cabo el análisis de la matriz de correlación con el objetivo de identificar las relaciones existentes entre las distintas variables macroeconómicas y detectar posibles problemas de multicolinealidad que puedan afectar posteriormente a la fase de modelización de los diferentes modelos.

3.5.3.1 Cálculo de la matriz de correlación

El primer paso será calcular y representar gráficamente la matriz de correlación de Pearson de la siguiente forma:

Ilustración 53: Cálculo de la matriz de correlación

```
import seaborn as sns

plt.figure(figsize=(10, 8))
corr_matrix = df[variables].corr()
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", center=0,
            square=True, linewidths=0.5, cbar_kws={"shrink": 0.8})
plt.title("Matriz de correlación entre variables macroeconómicas", fontsize=14)
plt.tight_layout()
plt.show()
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

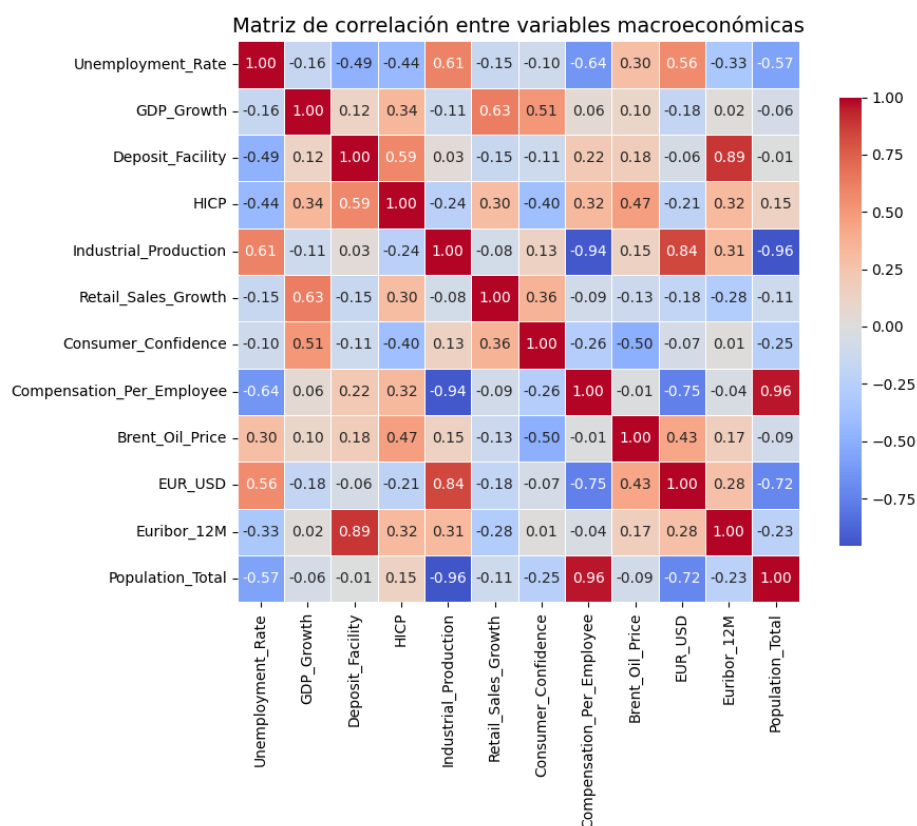
Dicho código calcula el coeficiente de correlación entre cada par de variables cuyos valores oscilan entre -1 y 1. Posteriormente, se utiliza `sns.heatmap()` para representar estos coeficientes mediante un mapa de calor y mostrando su valor numérico con dos decimales.

El uso de colores permite identificar de forma visual, la intensidad y el sentido de las relaciones. Donde los valores próximos a 1 indican una correlación positiva fuerte, los próximos a -1 una correlación negativa fuerte y los cercanos a 0 una relación lineal débil o inexistente. Y finalmente, el resto de parámetros se utilizan para ajustar el tamaño, formato y presentación de la figura.

3.5.3.2 Interpretación de la matriz de correlación

Del código anterior, se obtiene la matriz de correlación siguiente:

Ilustración 54: Interpretación de la matriz de correlación



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

A partir de esta matriz, se pueden obtener diferentes coeficientes r , que permiten analizar la intensidad y la dirección de la relación lineal entre cada par de variables.

Como se ha explicado anteriormente, el coeficiente de correlación de Pearson toma valores comprendidos entre -1 y 1. Donde, los valores próximos a 1, indican una relación lineal positiva, mientras que los valores próximos a -1 representan una relación lineal negativa. Por otro lado, los valores próximos a 0, reflejan una relación lineal débil o inexistente.

Esta matriz constituye una herramienta inicial para detectar posibles problemas de multicolinealidad debida a que dos o más variables independientes presentan una relación lineal elevada entre sí. Lo que quiere decir que contienen información parcialmente redundante. Y esto puede tener como consecuencia que sea difícil de determinar cuál de ellas contribuye individualmente a explicar la variabilidad de la variable objetivo.

No obstante, una correlación elevada no implica necesariamente que exista una relación causal entre las variables. En particular, cuando se trabaja con series temporales, dos variables pueden presentar una correlación elevada simplemente porque ambas siguen una tendencia creciente o decreciente durante el periodo analizado. Lo que quiere decir que puede reflejar una evolución temporal común y no una relación económica directa. Además, esto no afecta necesariamente a los modelos no lineales, sino a la interpretación individual de las variables como se ha especificado con anterioridad.

A continuación, se explicarán los valores de correlación obtenidos y la posible multicolinealidad obtenida y su probable causa.

3.5.3.2.1 Correlaciones muy altas ($|r| > 0,85$, posible alta multicolinealidad)

- **Compensation_Per_Employee y Population_Total ($r = 0,96$):** presentan una correlación muy alta. Esto se debe a que ambas variables muestran una tendencia creciente y relativamente estable a lo largo del periodo analizado, por lo que su elevada asociación parece deberse principalmente a esta evolución temporal compartida. Aunque conceptualmente representan fenómenos diferentes, salarios y población, desde el punto de vista estadístico contienen información parcialmente redundante.

No obstante, aunque la inclusión simultánea de ambas variables en un modelo lineal podría dificultar la interpretación individual de sus coeficientes, su presencia conjunta podría no perjudicar significativamente a la capacidad predictiva de determinados modelos no lineales. Aunque sí puede influir en la forma en que se distribuye la importancia entre las variables.

- **Industrial_Production y Population_Total ($r = -0,96$):** Esto indica una relación lineal negativa casi perfecta durante el periodo considerado. Aunque esto parece estar asociado a la existencia de tendencias opuestas, pues mientras una serie presenta una evolución decreciente, la otra muestra una evolución creciente.

En consecuencia, el valor elevado de la correlación no debe interpretarse automáticamente como evidencia de que el aumento de la población provoque una reducción de la producción industrial.

- **Industrial_Production y Compensation_Per_Employee ($r = -0,94$):** Al igual que en el caso anterior, la relación negativa parece estar determinada

principalmente por la evolución temporal opuesta de ambas series. Lo que podría generar problemas de multicolinealidad si las dos variables se incorporan simultáneamente a modelos lineales.

No obstante, desde una perspectiva económica, ambas variables podrían contener información útil sobre dimensiones diferentes sobre la actividad productiva y la capacidad adquisitiva o situación del mercado laboral. Por ello, su eliminación no debería basarse únicamente en la correlación, sino también en su fundamentación teórica, su comportamiento temporal y su contribución al rendimiento del modelo. Aunque esto es un análisis inicial.

- **Deposit_Facility y Euribor_12M ($r = 0,89$):** lo que indica una relación positiva muy elevada. Esto se debe a que ambas variables están relacionadas con la evolución de los tipos de interés y con el ciclo de política monetaria del Banco Central Europeo. Y esto podría implicar que ambas variables pueden proporcionar información muy parecida al modelo. Lo que podría dar problemas en un modelo lineal para separar lo que explica cada variable.

3.5.3.2.2 Correlaciones altas ($0,70 \leq |r| < 0,85$)

- **Industrial_Production y EUR_USD ($r = 0,84$):** esto una representa una relación lineal positiva fuerte. Aunque esto se debe a que ambas variables parecen compartir una tendencia descendente durante el periodo de estudio. Y tampoco parece existir necesariamente una relación causal directa entre la producción industrial y el tipo de cambio. Aunque es posible que ambas respondan a factores macroeconómicos comunes, como el ciclo económico o la evolución del comercio internacional.
- **Compensation_Per_Employee y EUR_USD ($r = -0,75$):** Este resultado corresponde a una correlación negativa fuerte. Lo que quiere decir que las dos series presentan evoluciones opuestas durante el periodo analizado. La relación puede estar influida por tendencias temporales y por factores macroeconómicos comunes, pero no debe interpretarse directamente como evidencia de que el tipo de cambio determine los salarios o viceversa.
- **EUR_USD y Population_Total ($r = -0,72$):** esto representa una relación negativa fuerte. Al igual que en otros pares, es probable que esta asociación refleje tendencias estructurales opuestas a lo largo del tiempo más que una

relación económica directa. Ya que la población suele evolucionar de forma gradual, mientras que el tipo de cambio puede responder a factores financieros, monetarios y comerciales. Por ello, una correlación elevada entre ambas variables podría estar condicionada por el periodo concreto utilizado y no mantenerse necesariamente en otras muestras temporales.

3.5.3.2.3 Correlaciones moderadas ($0,50 \leq |r| < 0,70$)

- **Unemployment_Rate e Industrial_Production ($r = 0,61$):** esto indica una relación positiva moderada. Aunque este resultado puede parecer contraintuitivo, ya que desde una perspectiva económica podría esperarse que un aumento de la producción industrial se asociara con una reducción del desempleo. Sin embargo, la correlación observada puede estar condicionada por la evolución temporal de ambas variables, por la existencia de retardos entre la producción y el empleo o por cambios estructurales en el sector industrial. Y es que una variación de la producción no tiene por qué trasladarse inmediatamente al mercado laboral, ya que las empresas pueden ajustar primero las horas trabajadas o la propia capacidad de la misma.
- **Unemployment_Rate y Compensation_Per_Employee ($r = -0,64$):** estas dos variables presentan una relación negativa moderada. Lo que puede deberse a cuando el desempleo es reducido, puede existir una mayor competencia por los trabajadores, lo que favorece el crecimiento de los salarios nominales.
- **Retail_Sales_Growth y GDP_Growth ($r = 0,63$):** esto indica una relación positiva moderada. Esto se puede deber a que el crecimiento del PIB se relacione con una mayor actividad económica, un incremento de la renta y un mayor dinamismo del consumo.
- **Deposit_Facility y HICP ($r = 0,59$):** representan una relación positiva moderada. Este resultado puede deberse a determinados periodos en los que el Banco Central Europeo incrementa los tipos de interés oficiales como respuesta a niveles elevados de inflación. Sin embargo, debe tenerse en cuenta que la política monetaria suele actuar con cierto retraso sobre la economía real. Además, los tipos de interés no se modifican únicamente en función de la inflación observada, sino también atendiendo a las expectativas de inflación, la actividad económica y las condiciones financieras.

- **Unemployment_Rate, HICP y Deposit_Facility:** La correlación entre Unemployment_Rate y HICP es $r = -0,44$ y la relación entre Unemployment_Rate y Deposit_Facility alcanza $r = -0,49$. Lo que hace a ambas asociaciones, negativas y moderadas. Estos resultados pueden reflejar las distintas fases del ciclo económico.
- **Consumer_Confidence y Brent_Oil_Price ($r = -0,50$):** indica una relación negativa moderada. Dicho resultado puede deberse a que un aumento del precio del petróleo pueda elevar los costes energéticos, reducir la renta disponible y aumentar la incertidumbre económica. Lo que puede afectar negativamente a la confianza de los consumidores.

3.5.3.2.4 Correlaciones débiles ($|r| < 0,20$)

- **Variables como Retail_Sales_Growth, Consumer_Confidence y Brent_Oil_Price,** presentan correlaciones reducidas con buena parte del resto de las variables analizadas. Lo que sugiere que existe entre ellas, una relación lineal prácticamente inexistente en la muestra.

Este resultado sugiere que dichas variables pueden aportar información relativamente independiente y complementaria. Sin embargo, un modelo no lineal puede demostrar relaciones no lineales que no se estén mostrando aquí.

3.5.3.3 Relación con las matriculaciones de BMW

Una vez llevado a cabo el análisis mediante la matriz de correlación entre las variables macroeconómicas, conviene llevarlo a cabo entre las variables macroeconómicas y las matriculaciones de BMW para identificar qué indicadores presentan una mayor relación lineal con la variable objetivo.

3.5.3.3.1 Cálculo de las correlaciones entre las variables macro y las matriculaciones de BMW

En primer lugar, integramos los datos de matriculaciones de BMW con las variables macroeconómicas previamente analizadas, utilizando el año como elemento común de la siguiente forma:

Ilustración 55: Uniendo los dos datasets

```
df_ventas = pd.read_csv("/content/drive/MyDrive/TFG_BMW/datasets/ventas_bmw.csv")
df_merged = pd.merge(df_ventas, df, left_on="Año", right_on="Year")
df_merged = df_merged.drop(columns=["Year"])

df_merged.head()
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Y posteriormente, calculamos las correlaciones de la siguiente forma:

Ilustración 56: Calculando las correlaciones

```
correlaciones = df_merged[["Ventas_BMW_Unidades"] + variables].corr()["Ventas_BMW_Unidades"].drop("Ventas_BMW_Unidades")
correlaciones = correlaciones.sort_values(ascending=False)

correlaciones
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De lo que se obtiene la tabla siguiente:

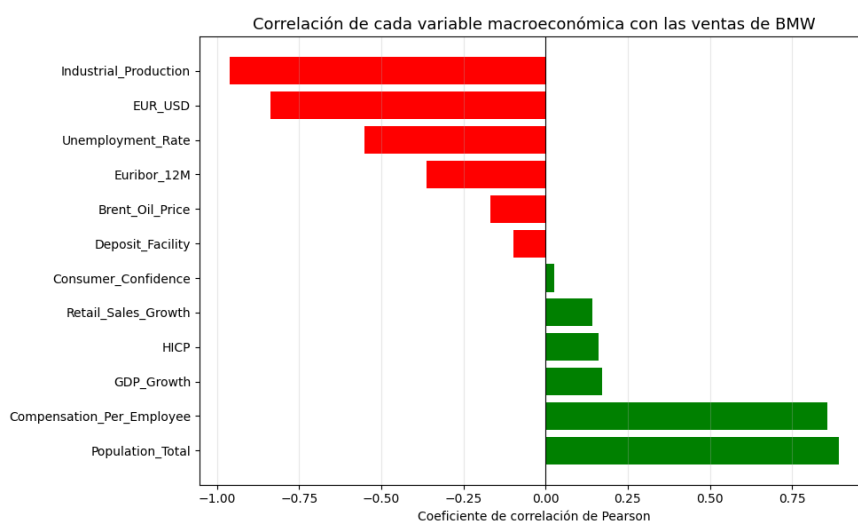
Ilustración 57: Tabla de correlaciones entre las ventas de BMW y las variables macroeconómicas

	Ventas_BMW_Unidades
Population_Total	0,891348
Compensation_Per_Employee	0,858315
GDP_Growth	0,171654
HICP	0,161310
Retail_Sales_Growth	0,143445
Consumer_Confidence	0,025838
Deposit_Facility	-0,097060
Brent_Oil_Price	-0,167470
Euribor_12M	-0,362991
Unemployment_Rate	-0,550192
EUR_USD	-0,836538
Industrial_Production	-0,960519

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Dicha tabla tendría el siguiente aspecto gráficamente:

Ilustración 58: Correlaciones de cada variable macro con las ventas de BMW



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De donde se pueden obtener los siguientes resultados:

3.5.3.3.1.1 *Correlaciones muy fuertes ($|r| > 0,80$)*

- **Industrial_Production ($r = -0,96$):** esto indica que existe una relación negativa prácticamente perfecta. Este resultado puede parecer contraintuitivo, ya que desde el punto de vista económico podría esperarse una relación positiva entre la actividad industrial y la demanda de vehículos. No obstante, dado que ambas variables presentan tendencias temporales claramente opuestas, esto podría explicar la fuerte correlación existente.
- **Population_Total ($r = 0,89$):** esto indica una gran relación positiva debida en gran medida, por la tendencia creciente que presentan ambas variables a lo largo del periodo.
- **EUR_USD ($r = -0,84$):** es una relación negativa muy alta. Lo que indica que, durante el periodo analizado, los incrementos del tipo de cambio han coincidido generalmente con menores niveles de ventas de BMW. Lo que puede deberse a la temporalidad de ambas variables.

3.5.3.3.1.2 *Correlaciones fuertes ($0,70 \leq |r| < 0,80$)*

- **EUR_USD ($r = -0,84$):** es una relación negativa muy alta. Lo que indica que, durante el periodo analizado, los incrementos del tipo de cambio han coincidido generalmente con menores niveles de ventas de BMW. Lo que puede deberse a la temporalidad de ambas variables.

3.5.3.3.1.3 *Correlaciones moderadas ($0,50 \leq |r| < 0,70$)*

- **Unemployment_Rate ($r = -0,55$):** se obtiene una correlación negativa y moderada. Este resultado indica que los periodos con mayores niveles de desempleo tienden a coincidir con menores niveles de matriculaciones de BMW. Lo cual resulta coherente desde el punto de vista económico, puesto que un aumento del desempleo puede reducir la capacidad adquisitiva de los hogares. Y como consecuencia, disminuir la demanda de bienes duraderos y de mayor precio.

3.5.3.3.1.4 *Correlaciones moderadas ($0,50 \leq |r| < 0,70$)*

- El resto de variables presentan correlaciones lineales débiles respecto a las ventas de BMW. En concreto, Euribor_12M presenta un coeficiente de -0,36, mientras que Brent_Oil_Price, Deposit_Facility, Consumer_Confidence, Retail_Sales_Growth, HICP y GDP_Growth presentan coeficientes entre -0,17 y 0,17. Lo que puede indicar que, considerando únicamente la relación lineal contemporánea, estas variables presentan una asociación reducida

durante el periodo analizado. Sin embargo, una correlación baja no implica necesariamente que estas variables carezcan de capacidad explicativa.

3.5.3.4 Diagnóstico de multicolinealidad

Con el objetivo de complementar el análisis realizado mediante la matriz de correlación, se calcula el Factor de Inflación de la Varianza o VIF. Este indicador permite evaluar hasta qué punto una variable macroeconómica puede ser explicada mediante una combinación lineal del resto de variables independientes.

De esta forma, permite identificar posibles problemas de multicolinealidad. Y como referencia general, se considera que valores de VIF inferiores a 5 indican una ausencia de problemas relevantes. Mientras que valores entre 5 y 10 reflejan una multicolinealidad moderada y valores superiores a 10 indican una multicolinealidad elevada.

3.5.3.4.1 Cálculo de la multicolinealidad existente y su interpretación

Para poder calcular el VIF de cada una de las variables macroeconómicas, se importa la función `variance_inflation_factor` de `statsmodels` y se seleccionan las variables macroeconómicas del conjunto de datos. Posteriormente, se crea una tabla en la que se almacena cada variable junto con su correspondiente valor de VIF, calculado de forma individual mediante un bucle. Y finalmente, `sort_values("VIF", ascending=False)` ordenará los resultados de mayor a menor VIF, permitiendo identificar rápidamente las variables que presentan una mayor relación lineal con el resto de predictores.

Ilustración 59: Cálculo del diagnóstico de multicolinealidad

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

X_vif = df_merged[variables]
vif_data = pd.DataFrame()
vif_data["Variable"] = variables
vif_data["VIF"] = [variance_inflation_factor(X_vif.values, i) for i in range(len(variables))]
vif_data.sort_values("VIF", ascending=False)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Del cual se obtiene la tabla siguiente:

Tabla 9: VIF de las variables macro

Variable	VIF
<i>Population_Total</i>	8.382,31
<i>Compensation_Per_Employee</i>	2.993,12
<i>EUR_USD</i>	1.621,20
<i>Industrial_Production</i>	750,90
<i>Unemployment_Rate</i>	278,57
<i>Brent_Oil_Price</i>	123,86
<i>Euribor_12M</i>	52,98
<i>HICP</i>	40,24
<i>Consumer_Confidence</i>	38,87
<i>Deposit_Facility</i>	31,57
<i>GDP_Growth</i>	6,26
<i>Retail_Sales_Growth</i>	4,96

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De la tabla anterior, se pueden obtener los siguientes resultados:

- **Multicolinealidad con valores extremadamente altos ($VIF > 500$):** Population_Total ($VIF = 8.382,3$), Compensation_Per_Employee ($VIF = 2.993,1$), EUR_USD ($VIF = 1.621,2$) y Industrial_Production ($VIF = 750,9$) presentan valores de VIF muy altos, muy por encima de cualquier umbral de referencia habitual. Lo que confirma lo observado tanto en la matriz de correlación como en el análisis de correlación con las ventas. Estas cuatro variables comparten en gran medida la misma información (tendencias estructurales muy marcadas y sostenidas a lo largo del período), hasta ser prácticamente combinaciones lineales unas de otras.
- **Multicolinealidad muy alta ($50 < VIF < 300$):** Unemployment_Rate ($VIF = 278,6$), Brent_Oil_Price ($VIF = 123,9$) y Euribor_12M ($VIF = 53,0$) presentan también niveles de multicolinealidad muy elevados, aunque considerablemente menores que el grupo anterior.
- **Multicolinealidad alta ($10 < VIF < 50$):** HICP ($VIF = 40,2$), Consumer_Confidence ($VIF = 38,9$) y Deposit_Facility ($VIF = 31,6$) superan también el umbral de multicolinealidad al igual que las variables anteriores aunque de menor manera.
- **Únicas variables sin multicolinealidad relevante ($VIF < 10$):** GDP_Growth ($VIF = 6,3$) y Retail_Sales_Growth ($VIF = 5,0$) son las únicas dos variables del

conjunto con niveles de VIF por debajo del umbral estándar de multicolinealidad severa aunque GDP_Growth se sitúa ligeramente por encima del umbral de 5 para multicolinealidad moderada.

La razón por la que esta magnitud adquiere valores tan altos puede explicarse principalmente a las características del conjunto de datos utilizado y a dos factores:

- **El tamaño muestral es muy reducido** en relación con el número de variables predictoras, ya que se dispone únicamente de 21 observaciones anuales y 12 variables macroeconómicas. Con un número tan limitado de observaciones, es más probable que las variables presenten relaciones lineales elevadas entre sí, lo que puede provocar valores de VIF especialmente altos.
- **Existe un predominio de variables con tendencias temporales marcadas.** Por ello, se obtienen tan elevadas correlaciones y valores de VIF.

Cabe destacar que estos valores son coherentes con los resultados previos del análisis de correlación con las ventas de BMW, donde precisamente estas mismas variables (Industrial_Production, Population_Total, EUR_USD, Compensation_Per_Employee) presentaban las correlaciones más elevadas con la variable objetivo. Lo que refuerza la hipótesis de que dichas correlaciones responden en gran medida a tendencias temporales compartidas más que a relaciones económicas causales directas.

3.6 Estudio de importancia de Variables Macroeconómicas

Una vez explicadas y llevado a cabo el análisis exploratorio de todas las variables macroeconómicas, se llevará a cabo el estudio de importancia de estas variables con el fin de determinar cuáles aportan una mayor cantidad de información para explicar y predecir las matriculaciones de BMW.

3.6.1 Introducción

Para llevar esto a cabo, se ha creado un segundo cuaderno de Jupyter en el cual se llevará a cabo este análisis mediante un modelo RandomForestRegressor de la librería scikit-learn. Esto permitirá complementar el estudio de correlaciones lineales realizado previamente con una técnica capaz de capturar relaciones no lineales e interacciones entre variables. Y así, obtener una estimación robusta de qué variables macroeconómicas resultan más relevantes para la construcción del modelo predictivo final de ventas de BMW.

La razón por la que se ha optado por este modelo, se debe a que, frente al análisis de correlación de Pearson, éste es capaz de capturar relaciones no lineales entre las variables macroeconómicas y las ventas de BMW. Mientras que el uso del

Coeficiente del Pearson y el VIF no puede. Además, puede detectar interacciones entre las variables que un análisis bivalente como la correlación no contempla. Y no requiere que las variables predictoras estén en la misma escala, lo que resulta especialmente conveniente dada la elevada heterogeneidad de magnitudes presente en el conjunto de datos.

No obstante, como se discutirá al interpretar los resultados, esta técnica no está exenta de limitaciones, particularmente en presencia de variables altamente correlacionadas entre sí, tal y como se identificó en el análisis de VIF del apartado anterior.

3.6.2 Estudio inicial

Para llevar a cabo el estudio inicial, se aplicará el RandomForest Regressor con el fin de evaluar la importancia relativa de las variables macroeconómicas en la predicción de las matriculaciones de BMW y determinar cuáles presentan una mayor capacidad explicativa dentro del modelo. Y posteriormente, se volverá a repetir el estudio con las variables con la mayor importancia.

3.6.2.1 Carga y unión de los datos

En primer lugar, se cargarán tanto las variables macroeconómicas como las ventas. De manera separada. Para lo que se utilizará la librería pandas, que permitirá importar los archivos en formato CSV y almacenarlos en dos *DataFrames* independientes. Y comprobamos que se ven correctamente.

Ilustración 60: Carga y unión de los datos

```
import pandas as pd

#from google.colab import drive
#drive.mount('/content/drive')

# Variables macro
ruta_macro = "/content/drive/MyDrive/TFG_BMW/datasets/bmw_dataset_variables_macro.csv"

# Ventas anuales BMW
ruta_ventas = "/content/drive/MyDrive/TFG_BMW/datasets/ventas_bmw.csv"

df_macro = pd.read_csv(ruta_macro, sep=",")
df_ventas = pd.read_csv(ruta_ventas, sep=",")

display(df_macro.head())
display(df_ventas.head())
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Una vez hecho esto, unimos ambos datasets con el objetivo de integrar las variables macroeconómicas y las ventas anuales de BMW en un único conjunto de datos, utilizando el año como variable común.

Ilustración 61: Unión de los dos conjuntos de datos

```
In [ ]: df_ventas = df_ventas.rename(columns={
        "Año": "Year"
    })

In [ ]: data = df_macro.merge(
        df_ventas,
        on="Year",
        how="inner"
    )

display(data.head())
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Y así, obtenemos la tabla siguiente de la que sólo se pueden ver las primeras filas:

Tabla 10: Unión de los dos conjuntos de datos

Year	Unemployment_Rate	GDP_Growth	Deposit_Facility	HICP	Industrial_Production	Retail_Sales_Growth	Consumer_Confidence	Compensation_Per_Employee	Brent_Oil_Price	EUR_USD	Euribor_12_M	Population_Total	Ventas_BMW_Unidades
2005	8.9	1.7	1.25	2.18	153.2	0.03	-4.2	31.22	54.57	12.441	2.19	434585887	1126768
2006	8.2	3.2	2.50	2.19	147.7	0.04	-3.5	31.95	65.16	12.556	3.08	436041018	1185088
2007	7.2	2.9	3.00	2.13	145.5	0.03	-2.1	32.75	72.44	13.705	4.24	437514477	1276793
2008	7.1	0.4	2.00	3.30	148.0	0.01	-10.8	33.86	96.94	14.708	4.63	439074280	1202239
2009	9.0	-4.4	0.25	0.29	144.5	-0.03	-21.7	34.41	61.74	13.948	1.31	440467898	1068770

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

3.6.2.2 Definición de X e Y

Y una vez llevado esto a cabo, creamos las variables, X e Y, que serán las variables predictoras y la variable objetivo del modelo, respectivamente.

En X se incluirán las 12 variables macroeconómicas, eliminando Year al tratarse únicamente de un identificador temporal y Ventas_BMW_Unidades, ya que esta última es precisamente la variable que se pretende predecir.

Por otro lado, en Y, se encontrarán las ventas anuales de BMW, que actuarán como variable objetivo.

Con el objetivo de asegurar que las variables cumplen con las dimensiones antes descritas, se utilizará la función `.shape()`. De la cual se ha obtenido que existen 21 observaciones y 12 variables predictoras para X y las 21 observaciones correspondientes a las ventas de Y.

Ilustración 62: Definición de X e Y y sus dimensiones

```
In [ ]: X = data.drop(columns=["Year", "Ventas_BMW_Unidades"])

        y = data["Ventas_BMW_Unidades"]

        print(X.shape)
        print(y.shape)

(21, 12)
(21,)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De esta forma, ya se tienen los datos preparados los datos en el formato requerido para entrenar posteriormente el modelo.

3.6.2.3 Entrenamiento del RandomForestRegressor

A continuación, se entrenará el RandomForestRegressor de la librería scikit-learn. La razón de su uso, se debe a que dado que se está resolviendo un problema de regresión, se utilizará para llevar a cabo predicciones numéricas.

Se basa en la combinación combinación de múltiples árboles de decisión, donde cada árbol realiza una predicción y en el caso de la regresión, el resultado final se obtiene promediando las predicciones de todos ellos.

Para ello, se se establece un bosque formado por 100 árboles de decisión mediante el parámetro `n_estimators=100`. El parámetro `random_state=42` permite que el proceso aleatorio utilizado por el modelo sea reproducible con la misma semilla, de modo que los resultados puedan repetirse en posteriores ejecuciones.

Finalmente, mediante `modelo.fit(X, y)` se entrenará el modelo utilizando las 12 variables macroeconómicas como variables predictoras (X) y las ventas de BMW como variable objetivo (y).

Ilustración 63: Entrenamiento del RandomForestRegressor

```
from sklearn.ensemble import RandomForestRegressor

modelo = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

modelo.fit(X, y)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Y así, en esta fase inicial se podrá obtener la importancia relativa de cada variable macroeconómica dentro del modelo.

Ilustración 64: Obtener y Ordenar la importancia de cada variable macroeconómica dentro del Random Forest

```
importancias = pd.DataFrame({
    "Variable": X.columns,
    "Importancia": modelo.feature_importances_
})

importancias = importancias.sort_values(
    by="Importancia",
    ascending=False
)

display(importancias)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De donde se puede obtener la tabla inferior:

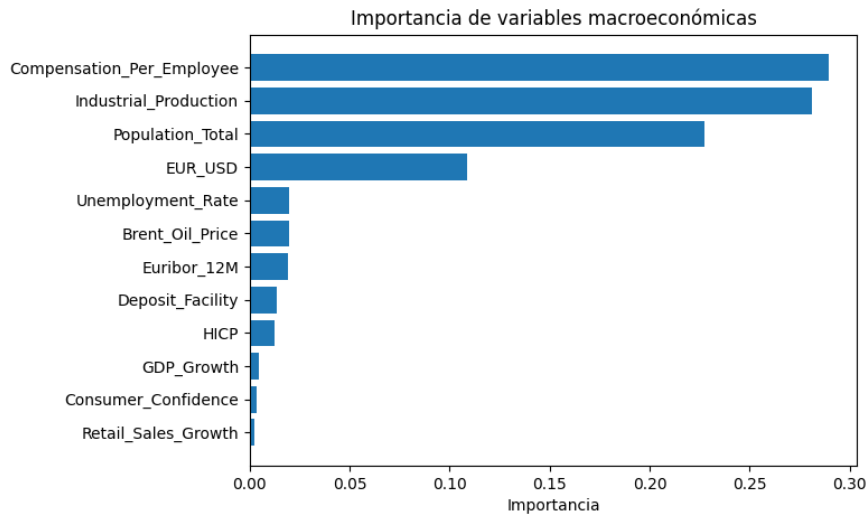
Tabla 11: Importancia de variables macroeconómicas

Variable	Importancia
Compensation_Per_Employee	0.289217
Industrial_Production	0.281089
Population_Total	0.227091
EUR_USD	0.108843
Unemployment_Rate	0.019807
Brent_Oil_Price	0.019490
Euribor_12M	0.018939
Deposit_Facility	0.013337
HICP	0.012211
GDP_Growth	0.004541
Consumer_Confidence	0.003192
Retail_Sales_Growth	0.002243

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De la cual, visualmente se puede obtener el gráfico siguiente:

Ilustración 65: Importancia de las variables macroeconómicas



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

3.6.2.4 Interpretación de la tabla y el gráfico de importancias

Una vez entrenado el modelo RandomForestRegressor, se ha obtenido la importancia relativa de cada variable explicativa utilizando la métrica Mean Decrease in Impurity (MDI). La cual, refleja la contribución de cada variable a la reducción del error de predicción a lo largo del conjunto de árboles que forman el bosque aleatorio.

Los resultados pueden obtener valores comprendidos entre 0 y 1. Y cuanto más alejado esté el valor de 0, mayor será su importancia relativa para el modelo.

Al analizar la tabla y el gráfico anterior, se observan las siguientes importancias:

- **Variables de mayor importancia:**
 - **Compensation_Per_Employee:** Es la variable con mayor importancia dentro del modelo con un valor de 0,289. Lo que indica que la compensación por empleado es la variable que presenta una mayor contribución relativa a las decisiones de división realizadas por el conjunto de árboles del Random Forest. Este resultado es coherente con la naturaleza del producto analizado. Ya que el nivel de renta influye en adquirir vehículos de mayor precio. No obstante, este resultado debe interpretarse con cierta cautela, ya que la variable presenta una tendencia creciente durante el período analizado y muestra una elevada correlación con otras variables del conjunto.

- **Industrial_Production:** ocupa la segunda posición, con una importancia de 0,281. Su elevada relevancia dentro del modelo puede estar relacionada con su capacidad para recoger la evolución general de la actividad económica y del ciclo económico. Pues, un mayor dinamismo de la producción industrial suele estar asociado con una situación económica más favorable, lo que puede repercutir sobre el consumo de bienes duraderos, entre ellos los automóviles. Sin embargo, al igual que en el caso anterior, su elevada importancia puede estar parcialmente condicionada por la fuerte tendencia temporal.
- **Population_Total:** presenta una importancia de 0,227. Lo que puede interpretarse como una indicación de que el tamaño potencial del mercado constituye un elemento relevante para el comportamiento de las matriculaciones. Aunque igual que en los casos anteriores, esta variable debe interpretarse con especial cautela, ya que presenta una tendencia creciente muy marcada durante el período analizado y registra una elevada multicolinealidad con otras variables.
- **EUR_USD:** El tipo de cambio EUR/USD presenta una importancia de 0,109. Las variaciones del tipo de cambio pueden afectar a la actividad de empresas con presencia internacional, tanto a través de los costes de determinados componentes como de la competitividad y de los resultados obtenidos en mercados exteriores. En este caso, su importancia dentro del Random Forest indica que aporta información relevante para el modelo, aunque considerablemente inferior a la proporcionada por las tres variables anteriores. Al mismo tiempo, debe tenerse en cuenta que esta variable también presenta una elevada correlación con otras variables del conjunto.
- **Variables de importancia media-baja:** Las variables Unemployment_Rate (0,020), Brent_Oil_Price (0,019) y Euribor_12M (0,019) presentan una importancia considerablemente menor que las cuatro variables anteriores. En el caso de la tasa de desempleo, ésta puede afectar a la capacidad de gasto de los hogares y a sus decisiones de consumo, mientras que el precio del petróleo puede influir sobre el coste de utilización de los vehículos y el Euribor sobre las condiciones de financiación para la adquisición de vehículos. No obstante, a pesar de su relevancia económica teórica, el

modelo les asigna una contribución relativamente reducida en comparación con las variables anteriores.

- **Variables de escasa importancia:** En esta categoría se encuentran las demás variables. Las cuales presentan valores de importancia próximos a cero. Esto indica que, dentro de este modelo y utilizando conjuntamente las 12 variables, estas variables aportan una contribución relativamente pequeña a las decisiones del Random Forest.

3.6.2.5 Conclusión de los resultados obtenidos

Los resultados obtenidos mediante el Random Forest muestran que las variables con mayor importancia relativa dentro del modelo son las cuatro siguientes:

Tabla 12: Variables de mayor importancia

1.	Compensation_Per_Employee
2.	Industrial_Production
3.	Population_Total
4.	EUR_USD

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Esto quiere decir que estas cuatro variables concentran aproximadamente el 90,6% de la importancia total asignada por el modelo y considerando conjuntamente las 12 variables macroeconómicas, son las que presentan una mayor contribución a las decisiones realizadas por el Random Forest.

A partir de estos resultados, se llevará a continuación una selección de estas variables para repetir el estudio sobre las mismas y conocer cómo se distribuye ese 9,4% restante sobre estas variables.

3.6.3 Re-evaluación con las variables seleccionadas

Una vez llevado a cabo el paso anterior, se procederá a realizar una nueva evaluación utilizando únicamente aquellas variables que hayan mostrado una mayor contribución al modelo. Por tanto, el objetivo de esta segunda etapa será comprobar si la reducción del conjunto de variables permite mantener o mejorar la capacidad predictiva del modelo, al mismo tiempo que se simplifica su estructura.

3.6.3.1 Carga de los datasets y selección de las variables macroeconómicas

En esta segunda fase se comienza cargando ambos datasets, tanto el de las ventas de BMW como el de las variables macroeconómicas, y posteriormente se seleccionarán las cuatro variables seleccionadas en la fase anterior y quedará la tabla siguiente:

Tabla 13: Cuatro variables macroeconómicas para repetir el estudio

Year	Compensation_Per_Employee	Industrial_Production	Population_Total	EUR_USD
2005	31.22	153.2	434585887	12.441
2006	31.95	147.7	436041018	12.556
2007	32.75	145.5	437514477	13.705
2008	33.86	148.0	439074280	14.708
2009	34.41	144.5	440467898	13.948

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

3.6.3.2 Construcción de las matrices X e Y, y entrenamiento del modelo Random Forest

Con el objetivo de repetir el experimento anterior con las variables seleccionadas, se crearán las matrices X (de entrada) e Y (de salida) para el entrenamiento del modelo Random Forest.

Ilustración 66: Construcción de matrices X e Y

```
In [4]: data = pd.merge(
        df_bmw,
        df_macro_reducido,
        left_on="Año",
        right_on="Year"
    )
    data = data.drop(columns=["Year"])

    X = data.drop(columns=["Año", "Ventas_BMW_Unidades"])
    y = data["Ventas_BMW_Unidades"]

    print("X shape:", X.shape)
    print("y shape:", y.shape)
```

X shape: (21, 4)
y shape: (21,)

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Por último, se imprimirá la estructura de ambas matrices, donde se muestra que X está formada por 21 observaciones y 4 variables predictoras, correspondientes a los 21 años comprendidos entre 2005 y 2025. Mientras que y contiene las 21 observaciones de las matriculaciones de BMW que se pretenden predecir.

Una vez llevado esto a cabo, se entrenará el modelo Random Forest Regressor utilizando únicamente las variables seleccionadas en el análisis anterior. Para mantener la comparabilidad con el modelo inicial, se emplean los mismos parámetros de configuración, estableciendo un total de 100 árboles (n_estimators=100) y una semilla aleatoria fija (random_state=42) al igual que se puede observar en la imagen inferior.

Ilustración 67: Re-entrenando el modelo Random Forest

```
from sklearn.ensemble import RandomForestRegressor

modelo_reducido = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

modelo_reducido.fit(X, y)

importancias_reducido = pd.DataFrame({
    "Variable": X.columns,
    "Importancia": modelo_reducido.feature_importances_
}).sort_values(by="Importancia", ascending=False)

print(importancias_reducido)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Una vez hecho esto, se obtiene también la importancia relativa de cada una de las variables mediante el atributo `feature_importances_`. Las cuales permiten analizar cómo se distribuye la contribución de las variables que permanecen en el conjunto reducido y comprobar si, al eliminar las variables de menor relevancia, las restantes adquieren un mayor peso dentro del modelo. El resultado obtenido se encuentra recogido en la siguiente tabla:

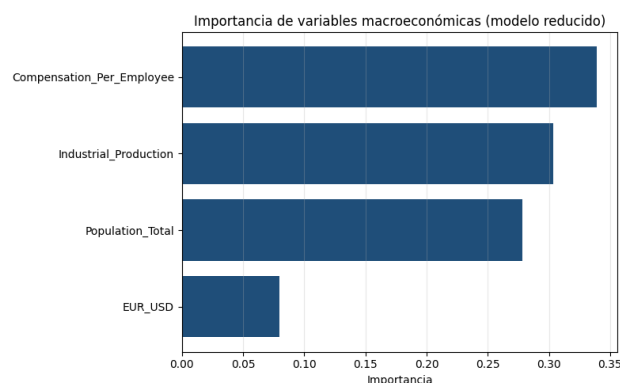
Tabla 14: Tabla de importancias de las variables macroeconómicas seleccionadas

Variable	Importancia
Compensation_Per_Employee	0,339008
Industrial_Production	0,303397
Population_Total	0,278002
EUR_USD	0,079592

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Dichos resultados tienen visualmente el aspecto siguiente:

Ilustración 68: Importancia de las variables macroeconómicas seleccionadas



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Si analizamos la gráfica y tabla anteriores, se puede observar que las variables con mayor peso en las predicciones son *Compensation_Per_Employee* (33,90%), *Industrial_Production* (30,34%) y *Population_Total* (27,80%), mientras que *EUR_USD* (7,96 %) presenta una importancia considerablemente menor.

Estos resultados sugieren que, dentro del conjunto de variables analizado, las condiciones relacionadas con la capacidad económica de los consumidores, la actividad industrial y el tamaño del mercado tienen una mayor relevancia para las predicciones del modelo que la evolución del tipo de cambio. No obstante, la importancia obtenida mediante Random Forest debe interpretarse como una medida de contribución predictiva y no como evidencia de una relación causal entre las variables y las matriculaciones de BMW.

3.6.3.3 Comparación con el modelo completo anterior

Si comparamos los resultados obtenidos con los anteriores, se obtiene la tabla siguiente:

Tabla 15: Comparación con el modelo completo anterior

Variable	Importancia (modelo completo)	Importancia (modelo reducido)	Diferencia
<i>Compensation_Per_Employee</i>	0,2892	0,339	+ 0,0498
<i>Industrial_Production</i>	0,2811	0,3034	+ 0,0223
<i>Population_Total</i>	0,2271	0,278	+ 0,0509
<i>EUR_USD</i>	0,1088	0,0796	-0,0292
Suma	0,9062	1	0,0938

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar, al comparar la importancia de las cuatro variables seleccionadas en el modelo reducido con la importancia que presentaban en el modelo completo, éstas seleccionadas concentraban conjuntamente el 90,62% de la importancia total. Y tras eliminar las ocho variables restantes, estas cuatro variables pasan a concentrar el 100 % de la importancia, produciéndose una redistribución del peso que anteriormente correspondía a las variables excluidas.

Donde, la importancia de *Compensation_Per_Employee* aumenta de 0,2892 a 0,3390, mientras que *Industrial_Production* pasa de 0,2811 a 0,3034 y *Population_Total* aumenta de 0,2271 a 0,2780. En cambio, *EUR_USD* disminuye de 0,1088 a 0,0796.

Además, se puede observar que el orden de importancia de las cuatro variables se mantiene en ambos modelos. Siendo *Compensation_Per_Employee* la variable

más importante, seguida de Industrial_Production, Population_Total y finalmente, EUR_USD. Esta estabilidad indica que la reducción del número de variables no modifica sustancialmente la jerarquía de los principales predictores identificados por el modelo.

No obstante, la redistribución de importancia no es uniforme. Mientras que las tres primeras variables incrementan su peso relativo, EUR_USD experimenta una reducción de aproximadamente 2,92 puntos porcentuales. Este resultado muestra que la importancia de una variable en un Random Forest depende también del conjunto de predictores que se incluye en el modelo, por lo que no debe interpretarse como una medida absoluta e independiente de las demás variables.

Por último, cabe señalar que la importancia obtenida mediante Random Forest representa la contribución de cada variable al proceso predictivo del modelo, pero no implica necesariamente la existencia de una relación causal con las ventas de BMW. Asimismo, los cambios observados en la importancia al modificar el conjunto de variables pueden estar relacionados con la existencia de información compartida o correlacionada entre los predictores.

3.6.4 Re-evaluación con las variables seleccionadas

Una vez llevado a cabo el estudio anterior, se construirá a lo largo de este apartado el dataset definitivo mediante la función añadir_variables_macro, que combinará las ventas anuales de BMW con las variables macroeconómicas Compensation_Per_Employee, Industrial_Production, Population_Total y EUR_USD seleccionadas en el análisis de importancia utilizando Random Forest.

3.6.4.1 Carga del dataset y unión del conjunto de datos

El primer paso es cargar los datos y se utilizará una función auxiliar “añadir_variables_macro”, la cual será reutilizada cada vez que se entrene un nuevo modelo para incorporar al conjunto de datos de ventas de BMW.

Ilustración 69: Función añadir_variables_macro

```
def añadir_variables_macro(df_bmw, df_macro):  
  
    df_final = df_bmw.merge(  
        df_macro[variables_modelo],  
        on="Year",  
        how="left"  
    )  
  
    return df_final
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Una vez integradas las variables, se separan los datos en variables predictoras (X) y variable objetivo (y). Y la variable Year se excluye de las variables predictoras, ya que se trata del identificador temporal de cada observación y no se utilizará directamente como variable explicativa. Del mismo modo, Ventas_BMW_Unidades se establece como variable objetivo, al representar el número de matriculaciones de BMW que se pretende predecir.

Ilustración 70: Creación de X e y

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

RANDOM_STATE = 42 # semilla fija para reproducibilidad

X = data.drop(columns=["Year", "Ventas_BMW_Unidades"])
y = data["Ventas_BMW_Unidades"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

X shape: (21, 4)
y shape: (21,)

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

En este punto, el conjunto de datos está formado por 21 observaciones anuales y 4 variables predictoras, mientras que la variable objetivo contiene las 21 observaciones correspondientes a las ventas de BMW. Por tanto, la matriz de entrada X presenta una dimensión de 21 x 4, y el vector objetivo y una dimensión de 21.

Además, se establece `RANDOM_STATE = 42` como semilla fija para garantizar la reproducibilidad de los resultados en aquellos algoritmos o procedimientos que incorporen componentes aleatorios. Mantener esta semilla constante permite que las diferentes ejecuciones del análisis sean comparables.

Estos datos constituirán la base común sobre la que posteriormente se aplicarán los cuatro modelos analizados: baseline naive, Ridge Regression, Random Forest y Gradient Boosting.

3.6.4.2 *Partición del conjunto de datos: train/dev/test*

A continuación, con el objetivo de evaluar el comportamiento de los modelos sobre observaciones no utilizadas durante su desarrollo, el conjunto de datos se dividirá en tres subconjuntos independientes (train, dev y test).

El conjunto de train se empleará para entrenar los modelos, el de dev se utiliza para seleccionar y ajustar los hiperparámetros y el conjunto de test se reservará exclusivamente para la evaluación final del rendimiento.

En esta primera aproximación, la partición se realiza de forma aleatoria, utilizando una proporción del 70 % para train, 15 % para dev y 15 % para test. Esto será modificado posteriormente debido a la temporalidad de los datos vista también con anterioridad.

Ilustración 71: Partición aleatoria train / dev / test

```
In [7]: # --- Partición train (70%) / dev (15%) / test (15%) ---
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=RANDOM_STATE, shuffle=True
)
X_dev, X_test, y_dev, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=RANDOM_STATE, shuffle=True
)

print(f"Train: {X_train.shape[0]} muestras")
print(f"Dev: {X_dev.shape[0]} muestras")
print(f"Test: {X_test.shape[0]} muestras")
```

```
Train: 14 muestras
Dev: 3 muestras
Test: 4 muestras
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Como se puede observar, dado que el conjunto de datos está compuesto únicamente por 21 observaciones anuales, esta proporción se traducirá en 14 observaciones para train, 3 para dev y 4 para test. Lo que da como resultado, un conjunto de desarrollo y de prueba con un número muy reducido de observaciones.

Esta característica constituye una limitación importante del estudio, ya que las métricas obtenidas a partir de conjuntos tan pequeños pueden presentar una elevada variabilidad.

Además, dado que las observaciones corresponden a años consecutivos, la naturaleza temporal de los datos debe tenerse en cuenta. Por este motivo, posteriormente se analizará también una partición temporal, en la que se respeta el orden cronológico de las observaciones, así como una validación mediante TimeSeriesSplit. Esto permitirá comprobar si las conclusiones obtenidas mediante la partición aleatoria se mantienen cuando se simula de forma más realista el escenario de utilizar información histórica para predecir años futuros.

3.6.4.3 Optimización de hiperparámetros

Una vez realizada la partición de los datos, se procede a seleccionar los valores más adecuados de los hiperparámetros del modelo Random Forest. Los hiperparámetros son aquellos parámetros que no se aprenden directamente

durante el entrenamiento, sino que deben establecerse previamente y su correcta selección puede influir de forma significativa en el rendimiento del modelo. Para ello, se llevará a cabo una búsqueda en rejilla (Grid Search), evaluando diferentes combinaciones de dos hiperparámetros:

- **n_estimators:** determina el número de árboles de decisión que forman el bosque. Se probarán los valores 10, 25, 50, 100, 200 y 300.
- **min_impurity_decrease:** establece la reducción mínima de impureza necesaria para que un nodo pueda dividirse. En un problema de regresión, esta impureza está relacionada con el error cuadrático medio. Valores más elevados hacen que el modelo sea más restrictivo a la hora de realizar nuevas divisiones y, por tanto, pueden actuar como mecanismo de regularización.

Ilustración 72: Optimización de hiperparámetros

```
n_estimators_grid = [10, 25, 50, 100, 200, 300]
min_impurity_decrease_grid = [0.0, 1e6, 5e6, 1e7, 5e7, 1e8]

resultados = []
for n_est in n_estimators_grid:
    for mid in min_impurity_decrease_grid:
        modelo = RandomForestRegressor(
            n_estimators=n_est,
            min_impurity_decrease=mid,
            random_state=RANDOM_STATE
        )
        modelo.fit(X_train, y_train)
        pred_dev = modelo.predict(X_dev)
        mse_dev = mean_squared_error(y_dev, pred_dev)
        resultados.append({
            "n_estimators": n_est,
            "min_impurity_decrease": mid,
            "MSE_dev": mse_dev
        })

df_resultados = pd.DataFrame(resultados).sort_values("MSE_dev")
display(df_resultados.head(10))
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

En total, se evaluarán 36 combinaciones diferentes (6 valores de n_estimators x 6 valores de min_impurity_decrease). Y para cada combinación se entrena un Random Forest utilizando exclusivamente el conjunto de train y posteriormente se calcula el MSE sobre el conjunto de dev. De esta forma, el conjunto de test permanece completamente separado y no interviene en la selección de los hiperparámetros. Además, se mantiene el random_state = 42 en todas las configuraciones, con el objetivo de garantizar que las comparaciones entre modelos sean reproducibles.

Y los resultados obtenidos son los recogidos en la tabla siguiente:

Tabla 16: Optimización de hiperparámetros

n_estimators	n_estimators	min_impurity_decrease	MSE_dev
0	10	0.0	5,72E+15
1	10	1000000.0	5,72E+15
2	10	5000000.0	5,72E+15
3	10	10000000.0	5,72E+15
4	10	50000000.0	5,75E+15
5	10	100000000.0	5,77E+15
35	300	100000000.0	7,72E+15
30	300	0.0	7,83E+15
32	300	5000000.0	7,83E+15
31	300	1000000.0	7,83E+15

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De la cual se obtiene que la combinación que obtiene el menor MSE sobre el conjunto de desarrollo es:

n_estimators = 10
min_impurity_decrease = 0
MSE en dev = 5.724.120.000 aproximadamente

Sin embargo, es importante interpretar este resultado con cautela. Ya que el conjunto de desarrollo contiene únicamente 3 observaciones, por lo que una única predicción muy alejada del valor real puede modificar considerablemente el MSE.

Por tanto, aunque esta combinación es la mejor dentro de las configuraciones evaluadas sobre este conjunto de dev, no puede afirmarse que 10 árboles sea necesariamente el valor óptimo para generalizar a nuevos datos.

De hecho, se observa que el MSE no presenta una relación monótona con n_estimators. Pues el aumento del número de árboles no implica necesariamente una reducción del error sobre este conjunto concreto.

Por otro lado, respecto a min_impurity_decrease, los resultados muestran que los valores más bajos apenas modifican las predicciones. Por ejemplo, para n_estimators = 10, los valores comprendidos entre 0 y 10.000.000 producen exactamente el mismo MSE. Esto indica que para estas configuraciones, el criterio adicional de restricción de las divisiones no llega a modificar de forma relevante la estructura de los árboles.

Este efecto comienza a observarse con valores más elevados, especialmente a partir de 50.000.000 y 100.000.000. En estos casos, el modelo se vuelve más restrictivo y algunas divisiones dejan de realizarse, lo que modifica ligeramente las predicciones y conduce en este caso, a un aumento del MSE. Por tanto, para este conjunto de datos, no se observa una mejora asociada a una regularización fuerte mediante `min_impurity_decrease`.

3.6.4.4 Selección de la combinación óptima

A partir de los resultados obtenidos mediante la búsqueda en rejilla, se selecciona como combinación óptima aquella que presenta el menor error cuadrático medio (MSE) sobre el conjunto de desarrollo.

Ilustración 73: Selección de la combinación óptima

```
In [9]: mejor = df_resultados.iloc[0]
print(f"Mejor combinación -> n_estimators={int(mejor['n_estimators'])}, "
      f"min_impurity_decrease={mejor['min_impurity_decrease']}, "
      f"MSE_dev={mejor['MSE_dev']:.2f}")

Mejor combinación -> n_estimators=10, min_impurity_decrease=0.0, MSE_dev=5724119738.17
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Como se puede observar, en este caso, la mejor configuración corresponde a `n_estimators = 10` y `min_impurity_decrease = 0,0`, con un MSE en el conjunto de desarrollo de 5.724.119.738,17

Esto significa quiere decir que entre las 36 combinaciones de hiperparámetros evaluadas, esta configuración es la que proporciona las predicciones más precisas sobre las tres observaciones pertenecientes al conjunto de desarrollo.

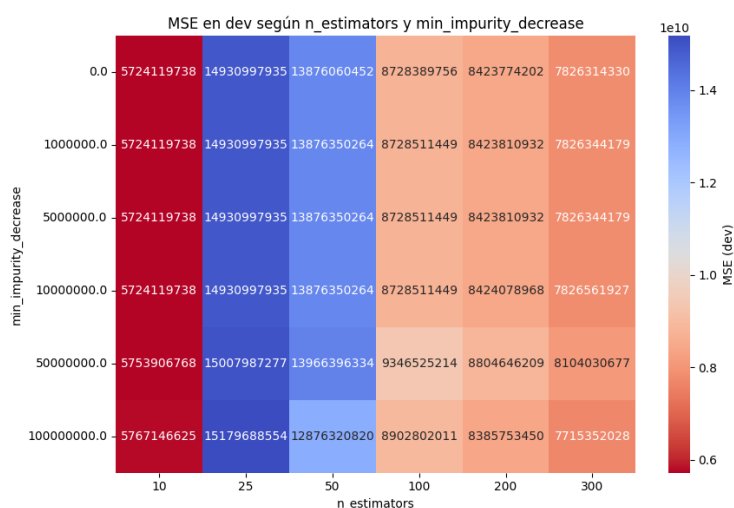
No obstante, al estar compuesto únicamente por tres observaciones, el MSE puede verse afectado de manera significativa por el comportamiento del modelo en una sola de ellas. Por tanto, aunque esta configuración es la mejor dentro de la partición utilizada, no existe evidencia suficiente para afirmar que un bosque con 10 árboles tenga una mejor capacidad de generalización que configuraciones con un mayor número de árboles.

Esta limitación es especialmente relevante en el presente estudio, dado que únicamente se dispone de 21 observaciones anuales. Por ello, posteriormente se utilizará `TimeSeriesSplit` como estrategia de validación cruzada temporal, permitiendo evaluar las distintas configuraciones sobre varios periodos de validación y obtener una selección de hiperparámetros menos dependiente de una única partición de los datos.

3.6.4.5 Visualización del efecto de los hiperparámetros

Una vez llevado esto a cabo, con el objetivo de analizar visualmente el efecto conjunto de los dos hiperparámetros evaluados de la tabla anterior, se representará un mapa de calor (heatmap) en el que cada celda muestra el MSE obtenido sobre el conjunto de dev o desarrollo para una determinada combinación de `n_estimators` y `min_impurity_decrease`. Y tendrá el aspecto siguiente:

Ilustración 74: Heatmap del MSE obtenido sobre el conjunto de dev



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

En el mismo, el eje horizontal representa el número de árboles del bosque (`n_estimators`), mientras que en el eje vertical se muestra el valor de `min_impurity_decrease`. La intensidad de cada celda permite comparar de forma rápida el error asociado a las distintas configuraciones, identificando aquellas que presentan un menor MSE.

Además, si lo analizamos, podemos observar que el comportamiento del MSE no es monótono respecto al número de árboles. Ya que la configuración con 10 estimadores presenta el menor MSE, con un valor de 5.724.119.738,17. Mientras que determinadas configuraciones con un mayor número de árboles presentan errores superiores.

No obstante, debido a que el conjunto de dev, al igual que se comentanba con anterioridad, está compuesto por 3 observaciones. Y en una muestra tan pequeña, el MSE puede verse afectado de forma considerable por la predicción realizada sobre una sola observación. Por tanto, de nuevo comentamos que las diferencias observadas entre las distintas configuraciones no permiten concluir que utilizar 10 árboles sea necesariamente mejor que utilizar un número superior de estimadores en términos de capacidad de generalización.

Por otro lado, en cuanto a `min_impurity_decrease`, se observa que los valores comprendidos entre 0 y 10.000.000 producen resultados muy similares para gran parte de las configuraciones. El efecto comienza a ser más apreciable con valores superiores, especialmente a partir de 50.000.000 como se observaba en la tabla anterior, donde se producen cambios más notables en el MSE. Esto se debe a que valores elevados de este hiperparámetro hacen que los árboles sean más restrictivos a la hora de realizar nuevas divisiones, limitando de esta forma su complejidad.

En conclusión, el mapa de calor muestra que la elección de los hiperparámetros presenta cierta sensibilidad en esta muestra, aunque la reducida cantidad de observaciones disponibles impide extraer conclusiones sólidas a partir del conjunto de desarrollo. Por ello, la combinación seleccionada (`n_estimators = 10` y `min_impurity_decrease = 0`) debe interpretarse como la que obtiene el mejor resultado dentro de la partición de desarrollo concreta utilizada y no como una evidencia definitiva de que sea la configuración óptima del Random Forest.

3.6.4.6 Evaluación final en test

Una vez seleccionada la combinación óptima de hiperparámetros según el MSE en dev, se entrena el modelo utilizando dicha combinación sobre el conjunto de train, y se evalúa su capacidad de generalización sobre el conjunto de test, que no ha sido utilizado en ninguna fase anterior del proceso de la siguiente forma:

Ilustración 75: Evaluación final en test

```
In [11]: modelo_final = RandomForestRegressor(
          n_estimators=int(mejor["n_estimators"]),
          min_impurity_decrease=mejor["min_impurity_decrease"],
          random_state=RANDOM_STATE
        )
          modelo_final.fit(X_train, y_train)

          pred_test = modelo_final.predict(X_test)
          mse_test = mean_squared_error(y_test, pred_test)

          print(f"MSE en test: {mse_test:.2f}")
          print(f"RMSE en test: {np.sqrt(mse_test):.2f}")

          MSE en test: 23814082256.50
          RMSE en test: 154318.12
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

La configuración seleccionada corresponde a `n_estimators = 10` y `min_impurity_decrease = 0`. Ya que mantener el conjunto de test separado durante todo el proceso permite obtener una estimación del rendimiento del modelo sobre observaciones que no han intervenido ni en su entrenamiento ni en la selección de sus hiperparámetros.

Por otro lado, respecto a los resultados, el Random Forest obtiene un MSE de 23.814.082.256,50 y un RMSE de 154.318,12 matriculaciones en el conjunto de test.

El RMSE indica que, en términos absolutos, el modelo presenta un error de aproximadamente 154.318 matriculaciones respecto a los valores reales. Este resultado debe interpretarse teniendo en cuenta que las ventas de BMW durante el periodo analizado se encuentran en un rango comprendido entre 1.068.770 y 2.253.835 unidades.

Además, el conjunto de test está formado únicamente por 4 observaciones, correspondientes a los años 2022-2025. Por tanto, el RMSE obtenido constituye una estimación basada en una muestra muy reducida y puede presentar una elevada variabilidad. Lo que quiere decir que un cambio en cualquiera de las observaciones del conjunto de test podría modificar de forma considerable las métricas obtenidas.

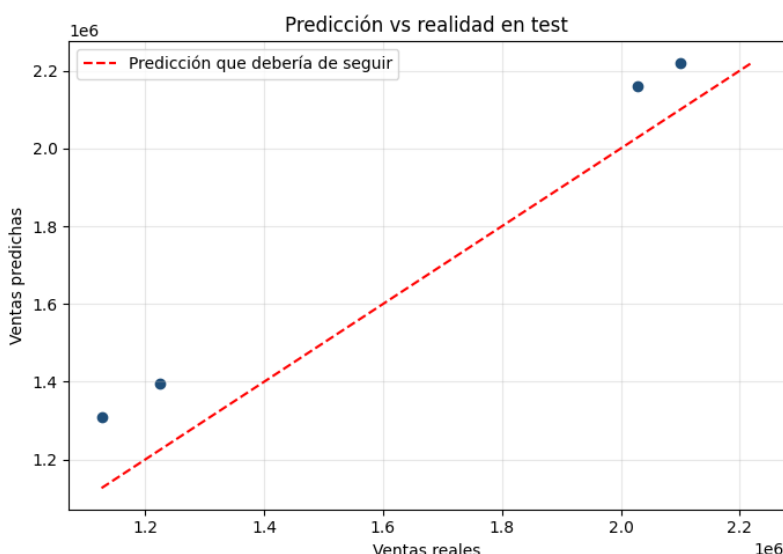
Por otro lado, El MSE obtenido en test es también notablemente superior al mínimo observado durante la búsqueda de hiperparámetros en dev (5.724.119.738,17). Esta diferencia no implica necesariamente un empeoramiento anómalo del modelo. Pues el conjunto de desarrollo se utilizó precisamente para seleccionar la configuración que presentaba el menor error entre las alternativas evaluadas, mientras que el conjunto de test no intervino en este proceso. Por ello, es esperable que el rendimiento obtenido en test sea diferente y en este caso, peor que el observado en dev.

En consecuencia, el resultado del conjunto de test debe considerarse la principal referencia para valorar el comportamiento del modelo ante datos no utilizados durante su desarrollo.

3.6.4.7 Comparación entre valores reales y predichos

Por último, para completar la evaluación del modelo, se representan gráficamente los valores reales de ventas frente a los valores predichos por el modelo en el conjunto de test, incluyendo la recta de predicción perfecta ($y = x$) como referencia visual. La cual tiene el aspecto siguiente:

Ilustración 76: Comparación entre valores reales y predichos



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Este gráfico permite observar el comportamiento de las cuatro predicciones correspondientes al periodo 2022-2025. La línea diagonal representa la situación ideal, en la que el valor predicho coincide exactamente con el valor real.

Por otro lado, se puede observar que las cuatro observaciones se encuentran por encima de la diagonal, lo que indica que el modelo sobreestima las ventas en los cuatro años del conjunto de test. Por tanto, además del valor global del RMSE, el gráfico permite identificar un posible sesgo en las predicciones para este periodo concreto. Este comportamiento es relevante porque muestra que el modelo no distribuye sus errores de manera completamente aleatoria. En concreto, se puede ver que en las cuatro observaciones analizadas, las predicciones se sitúan por encima de los valores reales, lo que indica una tendencia a sobreestimar el nivel de ventas durante el periodo 2022-2025.

Una posible explicación de este fenómeno puede deberse a las características de las variables utilizadas como predictores. Pues algunas de ellas, como `Population_Total` y `Compensation_Per_Employee`, presentan una tendencia creciente a lo largo del periodo analizado. Y estas variables pueden ayudar al modelo a representar la evolución estructural de la demanda, pero tienen una capacidad más limitada para recoger determinados cambios puntuales o descensos de las ventas.

No obstante, de nuevo, esta interpretación debe realizarse con prudencia. Ya que el conjunto de test contiene únicamente cuatro observaciones, por lo que la sobreestimación observada puede corresponder a una característica específica de

este periodo y no necesariamente a un comportamiento sistemático del modelo ante nuevos datos.

3.6.4.8 Conclusiones del apartado

En conclusión, el Random Forest seleccionado presenta un RMSE de 154.318,12 matriculaciones en test. Aunque este resultado proporciona una primera medida de su capacidad predictiva, debe interpretarse con cautela debido al reducido tamaño de la muestra disponible.

La comparación gráfica llevada con anterioridad permite observar también que el modelo sobreestima las ventas en las cuatro observaciones del periodo de test. Lo que pone de manifiesto que las métricas agregadas, por sí solas, no son suficientes para analizar completamente el comportamiento del modelo, siendo necesario estudiar también sus predicciones individuales y la dirección de los errores.

No obstante, una parte de este análisis se repetirá y modificará para comparar el Random Forest con los demás modelos.

Capítulo 4:

Modelos predictivos desarrollados

4. Modelos predictivos desarrollados

Una vez construido el dataset definitivo y seleccionadas las variables macroeconómicas utilizadas como predictores, se procede al desarrollo y evaluación de distintos modelos con el objetivo de estimar las matriculaciones anuales de BMW en Europa. Y dado el carácter temporal de los datos, la metodología experimental se plantea respetando el orden cronológico de las observaciones, evitando utilizar información de años posteriores para predecir años anteriores. Por lo que en este apartado las particiones serán temporales.

A lo largo de este capítulo se desarrollarán y compararán los cuatro enfoques de Naive Baseline, la Regresión Ridge, Random Forest y Gradient Boosting. Además, se analizará el comportamiento de dichos modelos tanto mediante sus métricas de error como mediante la comparación de sus predicciones sobre el periodo de test.

4.1 Metodología experimental

A continuación, se describen las fases de la metodología experimental utilizada para diseñar y validar la estrategia de evaluación de los modelos. Este procedimiento tiene una doble finalidad. Ya que por un lado permite establecer una partición temporal de los datos y, por otro permite implementar una validación cruzada mediante TimeSeriesSplit. De este modo, los hiperparámetros se evalúan sobre diferentes periodos temporales y se obtiene una estimación más robusta de la capacidad de generalización de los modelos.

4.1.1 Preparación del conjunto de datos

En primer lugar, se cargarán los dos conjuntos de datos utilizados en el estudio de las matriculaciones anuales de BMW y de las variables macroeconómicas. Y ambos conjuntos se integrarán utilizando el año como clave de unión mediante la función auxiliar `añadir_variables_macro`.

Es importante denotar que para los experimentos de este cuaderno se mantienen las cuatro variables macroeconómicas seleccionadas en el análisis de importancia realizado anteriormente (`Compensation_Per_Employee`, `Industrial_Production`, `Population_Total` y `EUR_USD`).

Ilustración 77: Unión de los datos

```
In [ ]: df_bmw_filtrado = df_bmw.rename(columns={"Año": "Year"})

data = añadir_variables_macro(df_bmw_filtrado, df_macro_reducido)

display(data.head())
print("Shape del dataset definitivo:", data.shape)
```

	Year	Ventas_BMW_Unidades	Compensation_Per_Employee	Industrial_Production	Population_Total	EUR_USD
0	2005	1126768	31.22	153.2	434585887	1.2441
1	2006	1185088	31.95	147.7	436041018	1.2556
2	2007	1276793	32.75	145.5	437514477	1.3705
3	2008	1202239	33.86	148.0	439074280	1.4708
4	2009	1068770	34.41	144.5	440467898	1.3948

Shape del dataset definitivo: (21, 6)

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Una vez integrado el dataset definitivo, se procede a separar las variables predictoras de la variable objetivo. Y para ello, se excluirán las columnas Year y Ventas_BMW_Unidades del conjunto de entrada (X) y se crearán X e y al igual que en el apartado anterior. Y como resultado, X es una matriz de 21 x 4 y por su parte, y representa una matriz de una dimensión de 21 observaciones anuales.

Ilustración 78: Creación de X e y

```
In [ ]: import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

RANDOM_STATE = 42 # semilla fija para reproducibilidad

X = data.drop(columns=["Year", "Ventas_BMW_Unidades"])
y = data["Ventas_BMW_Unidades"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

X shape: (21, 4)
y shape: (21,)

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.2 Random Forest

A continuación, se llevarán a cabo dos enfoques diferentes para para la predicción de las matriculaciones de BMW, con el objetivo de analizar si la diferencia de expresar la variable objetivo permite mejorar la capacidad predictiva del modelo.

En ambos enfoques se utilizará el modelo Random Forest, evaluando tanto una partición temporal fija como una validación mediante TimeSeriesSplit. De esta

forma, será posible analizar el comportamiento del modelo bajo diferentes estrategias de validación y determinar cuál de ellas ofrece mejores resultados.

4.2.1 Partición temporal de los datos (Enfoque A)

Debido a que las observaciones utilizadas en el estudio corresponden a una serie temporal anual, no resulta adecuado realizar una partición aleatoria del conjunto de datos es por ello por lo que se opta por llevar a cabo una temporal.

De esta forma, el conjunto de datos quedará dividido en tres periodos:

- **Train (2005–2018):** contiene 14 observaciones y se utiliza para entrenar los modelos.
- **Dev (2019–2021):** contiene 3 observaciones y se utiliza para seleccionar y ajustar los hiperparámetros de los modelos.
- **Test (2022–2025):** contiene 4 observaciones y se reserva para la evaluación final.

Y tendría la siguiente forma:

Ilustración 79: Partición temporal de los datos

```
In [ ]: # Partición temporal:
# Train: 2005-2018
# Dev: 2019-2021
# Test: 2022-2025

train = data[data["Year"] <= 2018]
dev = data[(data["Year"] >= 2019) & (data["Year"] <= 2021)]
test = data[data["Year"] >= 2022]

X_train = train.drop(columns=["Year", "Ventas_BMW_Unidades"])
y_train = train["Ventas_BMW_Unidades"]

X_dev = dev.drop(columns=["Year", "Ventas_BMW_Unidades"])
y_dev = dev["Ventas_BMW_Unidades"]

X_test = test.drop(columns=["Year", "Ventas_BMW_Unidades"])
y_test = test["Ventas_BMW_Unidades"]

print(f"Train: {len(train)} muestras ({train['Year'].min()}-{train['Year'].max()})")
print(f"Dev: {len(dev)} muestras ({dev['Year'].min()}-{dev['Year'].max()})")
print(f"Test: {len(test)} muestras ({test['Year'].min()}-{test['Year'].max()})")

Train: 14 muestras (2005-2018)
Dev: 3 muestras (2019-2021)
Test: 4 muestras (2022-2025)
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

El resultado final es la división antes mencionada.

4.2.1.1 Creación y evaluación del Baseline Naive

A continuación, antes de entrenar los modelos de Machine Learning, se establecerá un baseline naive como modelo de referencia para disponer de un método sencillo frente al cual comparar los modelos más complejos desarrollados posteriormente.

Es importante denotar que para el primer año disponible (2005) no existe una observación correspondiente a 2004 dentro del dataset utilizado. Por lo que, esta observación no puede generar una predicción naive válida. Por ello, se mantiene sin valor (NaN). Aunque en el código se establecerá a 0.

Así que esta observación no influirá posteriormente en el cálculo de las métricas, que se realiza exclusivamente sobre los periodos de desarrollo y test.

Ilustración 80: Creación del baseline naive

```
data_baseline = data.copy()

data_baseline["Prediccion_Naive"] = (
    data_baseline["Ventas_BMW_Unidades"].shift(1)
)

# Para 2005 no existe un año anterior
data_baseline.loc[
    data_baseline["Year"] == 2005,
    "Prediccion_Naive"
] = 0
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Y para su evaluación se utilizará el MSE (Mean Squared Error) y el RMSE (Root Mean Squared Error). Las métricas se calculan tanto sobre el conjunto de desarrollo o dev, correspondiente al periodo 2019–2021, como sobre el conjunto de test, correspondiente a 2022–2025. Y los resultados obtenidos son los siguientes.

Ilustración 81: Evaluación del baseline naive

```

pred_naive_dev = data_baseline.loc[
    data_baseline["Year"].between(2019, 2021),
    "Prediccion_Naive"
]

y_naive_dev = data_baseline.loc[
    data_baseline["Year"].between(2019, 2021),
    "Ventas_BMW_Unidades"
]

mse_naive_dev = mean_squared_error(
    y_naive_dev,
    pred_naive_dev
)

rmse_naive_dev = np.sqrt(mse_naive_dev)

print(f"MSE baseline en dev: {mse_naive_dev:,.2f}")
print(f"RMSE baseline en dev: {rmse_naive_dev:,.2f}")

```

MSE baseline en dev: 18,575,566,348.33
RMSE baseline en dev: 136,292.21

```

n [ ]:
pred_naive_test = data_baseline.loc[
    data_baseline["Year"] >= 2022,
    "Prediccion_Naive"
]

y_naive_test = data_baseline.loc[
    data_baseline["Year"] >= 2022,
    "Ventas_BMW_Unidades"
]

mse_naive_test = mean_squared_error(
    y_naive_test,
    pred_naive_test
)

rmse_naive_test = np.sqrt(mse_naive_test)

print(f"MSE baseline en test: {mse_naive_test:,.2f}")
print(f"RMSE baseline en test: {rmse_naive_test:,.2f}")

```

MSE baseline en test: 10,012,345,269.50
RMSE baseline en test: 100,061.71

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Estos resultados servirán como punto de referencia para determinar si los modelos Random Forest consiguen mejorar las predicciones realizadas simplemente utilizando las ventas del año anterior.

- **Dev:** MSE = 18.575.566.348,33 | RMSE = 136.292,21
- **Test:** MSE = 10.012.345.269,50 | RMSE = 100.061,71

El RMSE de 100.061,71 unidades en test, indica que en promedio, las predicciones se alejan de las ventas reales en torno a 100.062 matriculaciones.

Este resultado será el nivel de referencia que deberán superar los modelos desarrollados en apartados inferiores. Por tanto, un modelo será considerado mejor que el baseline si consigue obtener valores inferiores de MSE y RMSE sobre el mismo conjunto de test.

4.2.1.2 *Random Forest con partición temporal y evaluación en test*

En esta parte se entrenará un modelo Random Forest utilizando únicamente el conjunto de entrenamiento (2005-2018) y se evaluarán diferentes combinaciones de hiperparámetros sobre el conjunto de desarrollo (2019-2021). Y se probarán distintos valores de:

- **n_estimators**: número de árboles que forman el bosque.
- **min_impurity_decrease**: reducción mínima de impureza necesaria para realizar una división en un árbol.

Ilustración 82: Random Forest con partición temporal

```
In [ ]: n_estimators_grid = [10, 25, 50, 100, 200, 300]
min_impurity_decrease_grid = [0.0, 1e6, 5e6, 1e7, 5e7, 1e8]

resultados = []

for n_est in n_estimators_grid:
    for mid in min_impurity_decrease_grid:

        modelo = RandomForestRegressor(
            n_estimators=n_est,
            min_impurity_decrease=mid,
            random_state=RANDOM_STATE
        )

        modelo.fit(X_train, y_train)

        pred_dev = modelo.predict(X_dev)

        mse_dev = mean_squared_error(
            y_dev,
            pred_dev
        )

        resultados.append({
            "n_estimators": n_est,
            "min_impurity_decrease": mid,
            "MSE_dev": mse_dev
        })

df_resultados = (
    pd.DataFrame(resultados)
    .sort_values("MSE_dev")
)

display(df_resultados.head(10))
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Para cada combinación se calcula el MSE sobre el conjunto de desarrollo y posteriormente se ordenarán los resultados de menor a mayor error. De esta forma, se seleccionan los hiperparámetros que ofrecen el mejor rendimiento sin utilizar todavía el conjunto de test.

El resultado de esta parte es el siguiente:

Tabla 17: Cálculo del MSE en test

	n_estimators	min_impurity_decrease	MSE_dev
12	50	0	7,48E+09
13	50	1.000.000,00	7,48E+09
14	50	5.000.000,00	7,48E+09
15	50	10.000.000,00	7,48E+09
24	200	0	7,62E+09
25	200	1.000.000,00	7,62E+09
26	200	5.000.000,00	7,62E+09
27	200	10.000.000,00	7,62E+09
16	50	50.000.000,00	7,73E+09
6	25	0	7,86E+09

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Esto quiere decir que La primera configuración del ranking es:

- `n_estimators = 50`
- `min_impurity_decrease = 0`
- `MSE en dev  $\approx$  7.479.421.000`

Este valor supone una reducción del error respecto al baseline naive, cuyo MSE en desarrollo era de 18.575.566.348,33. Por tanto, esta configuración del Random Forest obtiene un mejor resultado sobre el conjunto de desarrollo y se selecciona para realizar la evaluación final sobre el conjunto de test.

No obstante, esta mejora en desarrollo no garantiza un mejor comportamiento sobre datos no utilizados durante la selección, por lo que el rendimiento definitivo del modelo debe determinarse mediante su evaluación sobre el conjunto de test.

A continuación, el modelo se evaluará sobre el conjunto de test (2022-2025), que se ha mantenido separado durante el proceso de selección.

Ilustración 83: Evaluando Random Forest en test

```
In [ ]: mejor = df_resultados.iloc[0]

modelo_final = RandomForestRegressor(
    n_estimators=int(mejor["n_estimators"]),
    min_impurity_decrease=mejor["min_impurity_decrease"],
    random_state=RANDOM_STATE
)

modelo_final.fit(X_train, y_train)

pred_test = modelo_final.predict(X_test)

mse_test = mean_squared_error(
    y_test,
    pred_test
)

rmse_test = np.sqrt(mse_test)

print(f"MSE en test: {mse_test:,.2f}")
print(f"RMSE en test: {rmse_test:,.2f}")
```

MSE en test: 12,114,233,109.67
RMSE en test: 110,064.68

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Como se puede observar, el Random Forest obtiene un RMSE de 110.064,68 matriculaciones, frente a los 100.061,71 obtenidos por el baseline naive. Por tanto, el Random Forest presenta un error aproximadamente un 10 % superior al baseline en el conjunto de test y no consigue mejorar su capacidad predictiva en esta partición temporal.

Este resultado pone de manifiesto que un modelo que obtiene un mejor rendimiento sobre el conjunto de desarrollo no necesariamente mantiene dicha

mejora cuando se evalúa sobre datos posteriores no utilizados durante el entrenamiento.

Esta diferencia entre el rendimiento en desarrollo y en test es relevante debido al reducido número de observaciones disponibles. Por este motivo, en el siguiente apartado se empleará `TimeSeriesSplit`, con el objetivo de realizar una validación cruzada que mantenga el orden temporal de las observaciones y permita evaluar las configuraciones de hiperparámetros sobre varios periodos de validación.

4.2.1.3 *TimeSeriesSplit*

A continuación, se aplicará `TimeSeriesSplit` con el objetivo de realizar una validación cruzada que respete la estructura temporal de los datos.

A diferencia de una validación cruzada convencional, `TimeSeriesSplit` mantiene el orden cronológico de las observaciones. Y de esta forma, en cada partición, el conjunto de entrenamiento está formado exclusivamente por observaciones anteriores a las utilizadas para la validación. Además, el conjunto de entrenamiento se amplía progresivamente a medida que avanzan los bloques temporales.

Ilustración 84: Preparando los datos para TimeSeriesSplit

```
In [ ]: data_train_dev = data[
        data["Year"] <= 2021
        ].copy()

        X_train_dev = data_train_dev.drop(
            columns=["Year", "Ventas_BMW_Unidades"]
        )

        y_train_dev = data_train_dev["Ventas_BMW_Unidades"]
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Para esta validación se utilizarán únicamente las observaciones comprendidas entre 2005 y 2021, manteniendo el periodo 2022-2025 completamente separado para la evaluación final. Y estarán divididos en cuatro bloques de la siguiente forma:

Ilustración 85: TimeSeriesSplit

```
In [ ]: from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
        tscv = TimeSeriesSplit(n_splits=4)

        for i, (train_idx, val_idx) in enumerate(
            tscv.split(X_train_dev,
                    start=1
            )
        ):
            print(f"Bloque {i}")
            print(
                "Train:",
                data_train_dev.iloc[train_idx]["Year"].min(),
                "-",
                data_train_dev.iloc[train_idx]["Year"].max()
            )

            print(
                "Validación:",
                data_train_dev.iloc[val_idx]["Year"].min(),
                "-",
                data_train_dev.iloc[val_idx]["Year"].max()
            )

            print()

        Bloque 1
        Train: 2005 - 2009
        Validación: 2010 - 2012

        Bloque 2
        Train: 2005 - 2012
        Validación: 2013 - 2015

        Bloque 3
        Train: 2005 - 2015
        Validación: 2016 - 2018

        Bloque 4
        Train: 2005 - 2018
        Validación: 2019 - 2021
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Como se puede observar, el primer bloque utiliza los años 2005-2009 para train y los años 2010-2012 para validarlo. En el segundo bloque, el periodo de train se amplía hasta 2012 y la validación se realiza sobre 2013-2015. Y este proceso continúa de forma progresiva hasta el cuarto bloque, en el que se utilizan los años 2005-2018 para train y 2019-2021 para validación.

Una vez finalizada esta validación cruzada, la combinación de hiperparámetros seleccionada se utilizará para entrenar el Random Forest definitivo con todas las observaciones disponibles entre 2005 y 2021. Y el periodo 2022-2025 se mantendrá reservado para la evaluación final del modelo sobre datos posteriores a los utilizados durante el proceso de entrenamiento y selección.

4.2.1.4 *GridSearchCV y evaluación del modelo final en el conjunto de test*

Una vez definida la estructura temporal de validación mediante TimeSeriesSplit, se utilizará GridSearchCV para seleccionar la combinación de hiperparámetros más adecuada para el modelo Random Forest. Y se evalúan las mismas 36 combinaciones consideradas anteriormente, formadas a partir de los siguientes valores:

- **n_estimators:** 10, 25, 50, 100, 200 y 300.
- **min_impurity_decrease:** 0, 1.000.000, 5.000.000, 10.000.000, 50.000.000 y 100.000.000.

Cada una de estas combinaciones se evaluará mediante los cuatro bloques temporales definidos con TimeSeriesSplit. Y para seleccionar la mejor

configuración se utiliza como métrica el RMSE, mediante `neg_root_mean_squared_error`, que es la forma en la que `GridSearchCV` representa esta métrica para realizar la maximización interna. Al recuperar el resultado mediante `best_score_`, se cambia nuevamente el signo para obtener el RMSE en su escala habitual.

Y el resultado final obtenido es el siguiente:

Ilustración 86: Mejores hiperparámetros y RMSE medio de validación

```
In [ ]: print("Mejores hiperparámetros:")
        print(grid_search.best_params_)

        print(
            "RMSE medio de validación:",
            -grid_search.best_score_
        )

Mejores hiperparámetros:
{'min_impurity_decrease': 50000000.0, 'n_estimators': 10}
RMSE medio de validación: 235573.53278545538
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar, la combinación de hiperparámetros seleccionada por `GridSearchCV` es:

- `n_estimators = 10`
- `min_impurity_decrease = 50.000.000`
- **RMSE medio de validación = 235.573,53**

Respecto al RMSE, éste representa el RMSE medio obtenido en los cuatro bloques de validación temporal. Por tanto, no representa el error sobre un único periodo, sino el promedio del rendimiento obtenido al validar el modelo en diferentes periodos históricos. Y es superior al registrado anteriormente al seleccionar los hiperparámetros únicamente mediante el conjunto de desarrollo.

Esta diferencia es esperable, ya que en este caso los hiperparámetros deben funcionar adecuadamente en varios periodos temporales y no únicamente en las observaciones de 2019-2021.

Una vez seleccionada esta configuración, los hiperparámetros se mantienen fijos y se procede a entrenar el modelo utilizando todas las observaciones disponibles entre 2005 y 2021. Y el conjunto de test correspondiente a los años 2022-2025 permanecerá separado y no intervendrá en este proceso.

Ilustración 87: Evaluando el último modelo en el test seleccionado

```
In [ ]: modelo_cv = grid_search.best_estimator_  
  
modelo_cv.fit(  
    X_train_dev,  
    y_train_dev  
)  
  
pred_test_cv = modelo_cv.predict(X_test)  
  
mse_test_cv = mean_squared_error(  
    y_test,  
    pred_test_cv  
)  
  
rmse_test_cv = np.sqrt(mse_test_cv)  
  
print(f"MSE en test: {mse_test_cv:.2f}")  
print(f"RMSE en test: {rmse_test_cv:.2f}")
```

MSE en test: 5,513,784,700.02
RMSE en test: 74,254.86

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

En este caso, el modelo obtiene un RMSE de 74.254,86 matriculaciones en el conjunto de test. Este resultado supone una mejora respecto al baseline naive, cuyo RMSE era de 100.061,71 matriculaciones. Se puede observar por tanto, que se reduce aproximadamente un 25,8 % respecto al baseline. Además, el resultado también mejora al Random Forest seleccionado anteriormente mediante una única partición temporal, cuyo RMSE en test era de 110.064,68 matriculaciones.

Estos resultados muestran que la selección de hiperparámetros mediante TimeSeriesSplit, al considerar varios periodos históricos de validación, conduce en este caso a una configuración que presenta un mejor comportamiento sobre el periodo posterior de test.

Por tanto, este modelo se establece como el Random Forest de referencia del Enfoque A, basado en los valores absolutos de las ventas y de las variables macroeconómicas seleccionadas. Posteriormente, su rendimiento se comparará con el modelo correspondiente al Enfoque B, en el que las ventas y las variables macroeconómicas se expresan mediante variaciones relativas interanuales.

La razón por la que se llevan a cabo dos enfoques, se debe a que se desea comparar si Random Forest predice mejor las ventas BMW cuando trabaja con los valores absolutos (Enfoque A) o cuando trabaja con las variaciones interanuales (Enfoque B).

4.2.1.5 Comparación entre los resultados obtenidos en el Baseline Naive

A continuación, se compararán en una tabla los resultados obtenidos por el baseline naive y las dos configuraciones de Random Forest desarrolladas en el Enfoque A.

Ilustración 88: Comparación entre los resultados obtenidos del Baseline Naive y Random Forest

Modelo	MSE dev	RMSE dev	MSE test	RMSE test
Baseline naive	18.575.566.348,33	136.292,21	10.012.345.269,50	100.061,71
Random Forest - partición temporal	≈7.479.421.000	86.483,65	12.114.233.109,67	110.064,68
Random Forest - TimeSeriesSplit	≈55.494.890.000	235.573,53	5.513.784.700,02	74.254,86
Valor mínimo	≈7.479.421.000	86.483,65	5.513.784.700,02	74.254,86

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar en la tabla superior, el Random Forest seleccionado mediante la partición temporal obtiene mejores resultados en el conjunto dev ya que es menor, pero no consigue superar al baseline en el conjunto test. No obstante, el modelo seleccionado mediante TimeSeriesSplit presenta un mayor RMSE medio de validación, pero obtiene el mejor resultado sobre el conjunto de test.

En concreto, el Random Forest con TimeSeriesSplit reduce el RMSE de 100.061,71 a 74.254,86 matriculaciones respecto al baseline, lo que supone una mejora aproximada del 25,8 %. Por tanto, esta configuración será la que ofrece el mejor rendimiento final dentro del Enfoque A.

4.2.2 Partición temporal del dataset de variaciones (Enfoque B)

En este segundo enfoque, la variable objetivo se transformará en la variación porcentual interanual de las matriculaciones de BMW. Y del mismo modo, las cuatro variables macroeconómicas seleccionadas en el Enfoque A se transformarán en variaciones respecto al año anterior.

Estas variaciones se calculan mediante el cambio porcentual entre cada año y el año inmediatamente anterior y se obtiene la tabla siguiente para los primeros años:

Tabla 18: Variación interanual de las variables de entrada

Año	Ventas BMW	Variación ventas	Compensación	Var. compensación	Producción industrial	Var. producción	Población	Var. población	EUR/USD	Var. EUR/USD
2006	1.185.088	5,18%	31,95	2,34%	147,7	-3,59%	436.041.018	0,33%	1,2556	0,92%
2007	1.276.793	7,74%	32,75	2,50%	145,5	-1,49%	437.514.477	0,34%	1,3705	9,15%
2008	1.202.239	-5,84%	33,86	3,39%	148	1,72%	439.074.280	0,36%	1,4708	7,32%
2009	1.068.770	-11,10%	34,41	1,62%	144,5	-2,36%	440.467.898	0,32%	1,3948	-5,17%
2010	1.224.280	14,55%	35,2	2,30%	127,2	-11,97%	441.160.594	0,16%	1,3257	-4,95%

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Al calcular estas variaciones no es posible obtener un valor para 2005, ya que no existe una observación del año anterior dentro del conjunto de datos. Por este motivo, se elimina únicamente dicho año, manteniéndose 20 observaciones correspondientes al periodo 2006-2025 como se puede observar en la imagen inferior:

Ilustración 89: Tabla obtenida de 20 x 11

```
print("Shape:", data_var.shape)
```

	Year	Ventas_BMW_Unidades	Compensation_Per_Employee	Industrial_Production	Population_Total	EUR_USD	V
1	2006	1185088	31.95	147.7	436041018	1.2556	
2	2007	1276793	32.75	145.5	437514477	1.3705	
3	2008	1202239	33.86	148.0	439074280	1.4708	
4	2009	1068770	34.41	144.5	440467898	1.3948	
5	2010	1224280	35.20	127.2	441160594	1.3257	

Shape: (20, 11)

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Por otro lado, la partición B mantiene los mismos periodos utilizados en el Enfoque A, con el objetivo de garantizar la comparabilidad entre ambos enfoques:

Ilustración 90: Partición enfoque B de los datos

```
In [ ]: train_var = data_var[
        data_var["Year"] <= 2018
        ].copy()

dev_var = data_var[
        (data_var["Year"] >= 2019) &
        (data_var["Year"] <= 2021)
        ].copy()

test_var = data_var[
        data_var["Year"] >= 2022
        ].copy()

print(
    f"Train: {len(train_var)} muestras "
    f"f'({train_var['Year'].min()}-{train_var['Year'].max()})"
)

print(
    f"Dev: {len(dev_var)} muestras "
    f"f'({dev_var['Year'].min()}-{dev_var['Year'].max()})"
)

print(
    f"Test: {len(test_var)} muestras "
    f"f'({test_var['Year'].min()}-{test_var['Year'].max()})"
)
```

Train: 13 muestras (2006-2018)
Dev: 3 muestras (2019-2021)
Test: 4 muestras (2022-2025)

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.2.2.1 Creación de X e y

Una vez realizada la partición temporal, se separan las variables predictoras (X) de la variable objetivo (y) para cada uno de los conjuntos. El resultado de esta separación es el del apartado anterior:

- **Train:** 13 observaciones y 4 variables predictoras.
- **Dev:** 3 observaciones y 4 variables predictoras.
- **Test:** 4 observaciones y 4 variables predictoras.

De esta forma, los tres conjuntos mantienen la misma estructura de variables predictoras, diferenciándose únicamente en el periodo temporal al que corresponden.

Ilustración 91: Creación de X e y

```
In [ ]: variables_X_var = [
        "Compensation_Per_Employee_Variacion",
        "Industrial_Production_Variacion",
        "Population_Total_Variacion",
        "EUR_USD_Variacion"
    ]

    X_train_var = train_var[variables_X_var]
    y_train_var = train_var["Ventas_BMW_Variacion"]

    X_dev_var = dev_var[variables_X_var]
    y_dev_var = dev_var["Ventas_BMW_Variacion"]

    X_test_var = test_var[variables_X_var]
    y_test_var = test_var["Ventas_BMW_Variacion"]

    print("X_train:", X_train_var.shape)
    print("y_train:", y_train_var.shape)

    print("X_dev:", X_dev_var.shape)
    print("y_dev:", y_dev_var.shape)

    print("X_test:", X_test_var.shape)
    print("y_test:", y_test_var.shape)
```

```
X_train: (13, 4)
y_train: (13,)
X_dev: (3, 4)
y_dev: (3,)
X_test: (4, 4)
y_test: (4,)
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

4.2.2.2 Creación del baseline naive

A continuación, se creará el baseline naive que deberá de adaptarse a la nueva variable objetivo. Ya que, en lugar de utilizar las ventas del año anterior directamente, se utiliza como predicción la variación porcentual de las ventas observada en el año anterior.

Este baseline servirá nuevamente como modelo de referencia, permitiendo comprobar si el Random Forest consigue mejorar la predicción de las variaciones interanuales utilizando las variables macroeconómicas.

Ilustración 92: Creación del baseline naive

```

In [ ]: data_baseline_var = data_var.copy()

# Predicción naive:
# La variación del año siguiente se estima igual a
# la variación observada en el año anterior

data_baseline_var["Predicción_Naive_Variación"] = (
    data_baseline_var["Ventas_BMW_Variación"].shift(1)
)

display(data_baseline_var.head())

```

	Year	Ventas_BMW_Unidades	Compensation_Per_Employee	Industrial_Production	Population_Total	EUR_USD	V
1	2006	1185088	31.95	147.7	436041018	1.2556	
2	2007	1276793	32.75	145.5	437514477	1.3705	
3	2008	1202239	33.86	148.0	439074280	1.4708	
4	2009	1068770	34.41	144.5	440467898	1.3948	
5	2010	1224280	35.20	127.2	441160594	1.3257	

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Es importante denotar que para 2006 no existe una variación del año anterior dentro del nuevo dataset, por lo que su predicción queda como NaN y no se utilizará en la evaluación.

4.2.2.3 Evaluación del baseline naive en dev y en test

En primer lugar se evaluará el baseline naive en dev y posteriormente en test para utilizar estos resultados como referencia para determinar si Random Forest consigue mejorar esta predicción.

Ilustración 93: Evaluación de Baseline Naive en dev

```

In [ ]: pred_naive_var_dev = data_baseline_var.loc[
    data_baseline_var["Year"].between(2019, 2021),
    "Predicción_Naive_Variación"
]

y_naive_var_dev = data_baseline_var.loc[
    data_baseline_var["Year"].between(2019, 2021),
    "Ventas_BMW_Variación"
]

mse_naive_var_dev = mean_squared_error(
    y_naive_var_dev,
    pred_naive_var_dev
)

rmse_naive_var_dev = np.sqrt(
    mse_naive_var_dev
)

print(
    f"MSE baseline variaciones en dev: "
    f"{mse_naive_var_dev:.6f}"
)

print(
    f"RMSE baseline variaciones en dev: "
    f"{rmse_naive_var_dev:.6f}"
)

```

MSE baseline variaciones en dev: 0.010495
RMSE baseline variaciones en dev: 0.102447

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De este modo, utilizando las métricas MSE y RMSE, los resultados obtenidos son:

- MSE: 0,010495
- RMSE: 0,102447

Lo que quiere decir que el baseline presenta un RMSE de 0,102447 que equivale aproximadamente a un 10,24% de error en la predicción de la variación interanual de las matriculaciones.

Por otro lado, respecto a la evaluación del baseline naive en test:

Ilustración 94: Evaluación del Baseline Naive en test

```
In [ ]: pred_naive_var_test = data_baseline_var.loc[
        data_baseline_var["Year"] >= 2022,
        "Prediccion_Naive_Variacion"
    ]

    y_naive_var_test = data_baseline_var.loc[
        data_baseline_var["Year"] >= 2022,
        "Ventas_BMW_Variacion"
    ]

    mse_naive_var_test = mean_squared_error(
        y_naive_var_test,
        pred_naive_var_test
    )

    rmse_naive_var_test = np.sqrt(
        mse_naive_var_test
    )

    print(
        f"MSE baseline variaciones en test: "
        f"{mse_naive_var_test:,.6f}"
    )

    print(
        f"RMSE baseline variaciones en test: "
        f"{rmse_naive_var_test:,.6f}"
    )
```

MSE baseline variaciones en test: 0.011272
RMSE baseline variaciones en test: 0.106172

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Como se puede observar, el baseline obtiene un RMSE de 0,106172, equivalente aproximadamente a un 10,62% de error en la predicción de la variación interanual de las matriculaciones. Dicho resultado constituye el punto de referencia final del Enfoque B. Y a partir de este valor se podrá determinar posteriormente si las configuraciones de Random Forest consiguen reducir el error de predicción sobre el mismo periodo de test.

4.2.2.4 *Random Forest con partición temporal y optimización de hiperparámetros*

A continuación, se elaborará el primer modelo de Random Forest del Enfoque B. Y se realizará una búsqueda de hiperparámetros utilizando la partición temporal definida anteriormente. En este caso, se estudiarán dos parámetros del modelo:

- **n_estimators:** número de árboles que forman el bosque. Se prueban los valores 10, 25, 50, 100, 200 y 300.

- **min_impurity_decrease**: reducción mínima de impureza necesaria para realizar una división en un nodo del árbol. Se prueban los valores 0, 0,0001, 0,0005, 0,001, 0,005 y 0,01.

Y se probarán distintas combinaciones de `n_estimators` y `min_impurity_decrease` para seleccionar la configuración con menor MSE en el conjunto de desarrollo.

Ilustración 95: Optimización de hiperparámetros

```
In [ ]: resultados_var = []

for n_est in n_estimators_grid_var:
    for mid in min_impurity_decrease_grid_var:
        modelo = RandomForestRegressor(
            n_estimators=n_est,
            min_impurity_decrease=mid,
            random_state=RANDOM_STATE
        )
        modelo.fit(
            X_train_var,
            y_train_var
        )
        pred_dev_var = modelo.predict(
            X_dev_var
        )
        mse_dev_var = mean_squared_error(
            y_dev_var,
            pred_dev_var
        )
        resultados_var.append({
            "n_estimators": n_est,
            "min_impurity_decrease": mid,
            "MSE_dev": mse_dev_var
        })

df_resultados_var = (
    pd.DataFrame(resultados_var)
    .sort_values("MSE_dev")
)

display(
    df_resultados_var.head(10)
)
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

De donde se puede obtener la siguiente tabla:

Ilustración 96: Mejores combinaciones de `n_estimators` y `min_impurity_decrease`

n_estimators	min_impurity_decrease	MSE dev
10	0,01	0,004601
25	0,01	0,005159
10	0,005	0,005183
10	0	0,005198
50	0,01	0,005414
300	0,01	0,00545
300	0	0,00547
200	0,01	0,005484
100	0,01	0,00549
25	0,005	0,005567

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

De dicha tabla se puede concluir que la configuración que obtiene el menor error está formada por 10 árboles (`n_estimators = 10`) y un valor de `min_impurity_decrease = 0,01`, alcanzando un MSE de 0,004601.

Por tanto, esta configuración se selecciona como la mejor dentro de la búsqueda realizada mediante la partición temporal y será utilizada posteriormente para evaluar el modelo sobre el conjunto de test correspondiente al periodo 2022-2025.

4.2.2.5 Selección de los mejores hiperparámetros

A partir de los resultados obtenidos en la búsqueda de hiperparámetros, se selecciona la configuración que presenta el menor error en el conjunto de desarrollo. En este caso, la mejor combinación corresponde a un Random Forest compuesto por 10 árboles (`n_estimators = 10`) y con un valor de `min_impurity_decrease = 0,01` como se puede observar en la imagen inferior:

Ilustración 97: Selección de los mejores hiperparámetros

```
In [ ]: mejor_var = df_resultados_var.iloc[0]

print("Mejores hiperparámetros:")

print(
    f"n_estimators = "
    f"{int(mejor_var['n_estimators'])}"
)

print(
    f"min_impurity_decrease = "
    f"{mejor_var['min_impurity_decrease']}"
)

print(
    f"MSE en dev = "
    f"{mejor_var['MSE_dev']:.6f}"
)

Mejores hiperparámetros:
n_estimators = 10
min_impurity_decrease = 0.01
MSE en dev = 0.004601
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.2.2.6 Evaluar Random Forest en test

Una vez seleccionados los mejores hiperparámetros, se entrena el modelo Random Forest utilizando el conjunto de entrenamiento correspondiente al periodo 2006-2018. Y posteriormente, el modelo se evalúa sobre el conjunto de test (2022-2025), que se ha mantenido separado durante el proceso de entrenamiento y selección de hiperparámetros y se obtienen los siguientes resultados:

Ilustración 98: Evaluando Random Forest en test

```
In [ ]: modelo_final_var = RandomForestRegressor(
        n_estimators=int(
            mejor_var["n_estimators"]
        ),
        min_impurity_decrease=(
            mejor_var["min_impurity_decrease"]
        ),
        random_state=RANDOM_STATE
    )

    modelo_final_var.fit(
        X_train_var,
        y_train_var
    )

    pred_test_var = modelo_final_var.predict(
        X_test_var
    )

    mse_test_var = mean_squared_error(
        y_test_var,
        pred_test_var
    )

    rmse_test_var = np.sqrt(
        mse_test_var
    )

    print(
        f"MSE Random Forest en test: "
        f"{mse_test_var:,.6f}"
    )

    print(
        f"RMSE Random Forest en test: "
        f"{rmse_test_var:,.6f}"
    )
```

MSE Random Forest en test: 0.004011
RMSE Random Forest en test: 0.063330

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Como se puede observar, el RMSE tiene un valor de 0,063330 y representa un error aproximado de un 6,33% en la predicción de la variación interanual de las ventas de BMW.

Este resultado es inferior al obtenido por el baseline naive, cuyo RMSE en test fue de 0,106172, lo que indica que el Random Forest consigue una mayor capacidad predictiva para estimar las variaciones interanuales de las matriculaciones. Por tanto, dentro de este primer planteamiento del Enfoque B, el modelo basado en Random Forest mejora claramente el rendimiento del modelo de referencia.

4.2.2.7 Comparación del enfoque B con el Baseline Naive

A continuación, se comparan los resultados del Random Forest con los obtenidos mediante el baseline naive, tanto en el conjunto de desarrollo como en el conjunto de test:

Tabla 19: Comparación Enfoque B con el Baseline Naive

Modelo	MSE dev	RMSE dev	MSE test	RMSE test
Baseline naive	0,010495	0,102447	0,011272	0,106172
Random Forest	0,004601	0,06783	0,004011	0,06333

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar, el Random Forest obtiene menores valores de MSE y RMSE en ambos conjuntos, lo que supone una mejora respecto al baseline.

4.2.2.8 TimeSeriesSplit

A continuación, Para realizar una validación cruzada temporal, se combinan los conjuntos de entrenamiento y desarrollo, utilizando las observaciones correspondientes al periodo 2006-2021. De esta forma, se dispone de 16 observaciones para llevar a cabo el proceso de validación, mientras que el conjunto de test, correspondiente a 2022-2025, permanece separado y no interviene en esta fase.

Ilustración 99: TimeSeriesSplit

```
In [ ]: data_train_dev_var = data_var[
        data_var["Year"] <= 2021
    ].copy()

    X_train_dev_var = data_train_dev_var[
        variables_X_var
    ]

    y_train_dev_var = data_train_dev_var[
        "Ventas_BMW_Variación"
    ]

    print(
        "Observaciones train + dev:",
        len(data_train_dev_var)
    )
```

Observaciones train + dev: 16

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

A continuación, se crean un total de 4 bloques con el fin de mantener el orden cronológico de las observaciones. Estos cuatro bloques son los siguientes:

Tabla 20: Cuatro bloques temporales con TimeSeriesSplit

Bloque	Entrenamiento	Validación
1	2006-2009	2010-2012
2	2006-2012	2013-2015
3	2006-2015	2016-2018
4	2006-2018	2019-2021

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como puede observarse, el conjunto de entrenamiento se amplía progresivamente en cada bloque, mientras que las observaciones utilizadas para la validación corresponden siempre a un periodo posterior. De esta forma, el procedimiento respeta la estructura temporal de la serie y evita utilizar información de años futuros durante el entrenamiento de cada iteración.

Además, es importante remarcar que el último bloque utiliza los años 2019-2021 como periodo de validación, mientras que los años 2022-2025 permanecen fuera del proceso de validación. Ya que están reservados exclusivamente para la evaluación final del modelo sobre el conjunto de test.

4.2.2.9 *GridSearchCV con TimeSeriesSplit*

Una vez definidos los bloques temporales, se utiliza GridSearchCV junto con TimeSeriesSplit para seleccionar los mejores hiperparámetros del modelo Random Forest. De esta manera, cada combinación de hiperparámetros se evalúa utilizando los cuatro bloques temporales definidos anteriormente.

En lo que respecta al proceso de búsqueda, se consideran dos hiperparámetros, `n_estimators`, que determina el número de árboles que forman el bosque y `min_impurity_decrease`, que establece la reducción mínima de impureza necesaria para realizar una división en un nodo.

Para esta parte, se prueban 6 valores para cada hiperparámetro, dando lugar a un total de 36 combinaciones diferentes.

4.2.2.10 *Obtener los mejores hiperparámetros*

Una vez finalizada la búsqueda mediante GridSearchCV y TimeSeriesSplit, se obtiene la combinación de hiperparámetros que presenta el mejor rendimiento medio sobre los distintos bloques temporales de validación.

Ilustración 100: Obtener los mejores hiperparámetros

```
In [ ]: print("Mejores hiperparámetros:")
        print(
            grid_search_var.best_params_
        )
        print(
            "RMSE medio de validación:",
            -grid_search_var.best_score_
        )
```



```
Mejores hiperparámetros:
{'min_impurity_decrease': 0.01, 'n_estimators': 25}
RMSE medio de validación: 0.06266359510853145
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Como se puede observar, la configuración seleccionada es:

- `n_estimators = 25`
- `min_impurity_decrease = 0,01`
- **RMSE medio de validación = 0,062664**

Como se puede observar, el RMSE medio obtenido es de 0,062664, lo que equivale aproximadamente a un 6,27% de error en la predicción de la variación interanual de las ventas.

Por tanto, la configuración formada por 25 árboles y un valor de `min_impurity_decrease` de 0,01 se selecciona como la mejor combinación de hiperparámetros mediante validación cruzada temporal.

Estos hiperparámetros se utilizarán para entrenar posteriormente el modelo final con todas las observaciones disponibles del periodo 2006-2021, antes de realizar su evaluación sobre el conjunto de test correspondiente a 2022-2025.

4.2.2.11 *Obtener los mejores hiperparámetros*

Una vez seleccionados los hiperparámetros mediante `GridSearchCV` y `TimeSeriesSplit`, se entrena el modelo final utilizando todas las observaciones disponibles del periodo 2006-2021. Posteriormente, se evalúa sobre el mismo conjunto de test (2022-2025) utilizado en las evaluaciones anteriores.

Los resultados obtenidos son:

- **MSE en test: 0,004369**
- **RMSE en test: 0,066101**

Como se puede observar, el RMSE obtenido es de 0,066101. Lo que equivale a aproximadamente un 6,61% de error en la predicción de la variación interanual de las ventas de BMW.

4.2.2.12 *Comparación final del enfoque B*

Una vez se ha llevado todo esto a cabo, se compararán los resultados obtenidos sobre el conjunto de test (2022-2025) para las tres alternativas evaluadas en el Enfoque B de la siguiente forma:

Tabla 21: Comparación final del Enfoque B

Modelo	MSE en test	RMSE en test
Baseline naive	0,011272	0,106172
Random Forest - partición temporal	0,004011	0,063333
Random Forest - TimeSeriesSplit	0,004369	0,066101

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar, ambas configuraciones de Random Forest mejoran los resultados del baseline naive. En concreto, se puede observar que el mejor resultado corresponde al Random Forest seleccionado mediante la partición temporal, que obtiene un MSE de 0,004011 y un RMSE de 0,063330. Por su parte, el modelo seleccionado mediante TimeSeriesSplit obtiene un MSE de 0,004369 y un RMSE de 0,066101.

En comparación con el baseline, el Random Forest mediante partición temporal reduce el RMSE aproximadamente un 40,35%, mientras que la configuración seleccionada mediante TimeSeriesSplit lo reduce aproximadamente un 37,75%. Por tanto, ambos modelos presentan una mejora significativa respecto al modelo de referencia. Aunque el modelo seleccionado mediante la partición temporal obtiene un rendimiento ligeramente superior sobre el conjunto de test, TimeSeriesSplit permite seleccionar los hiperparámetros evaluando el modelo en varios periodos temporales, manteniendo siempre el orden cronológico de las observaciones. Esto proporciona una evaluación más completa del comportamiento del modelo a lo largo de distintos periodos históricos.

Esto tiene como consecuencia que para el Enfoque B, la configuración que presenta el mejor rendimiento final sobre el conjunto de test es el Random Forest seleccionado mediante partición temporal.

4.2.3 Comparación entre los enfoques A y B

Finalmente, se comparan los resultados obtenidos mediante los dos enfoques planteados. En el Enfoque A, la variable objetivo corresponde al número absoluto de matriculaciones de BMW, mientras que en el Enfoque B se predice la variación porcentual interanual de éstas.

Tabla 22: Comparación entre los enfoques A y B

Enfoque	Modelo	MSE en test	RMSE en test
A - Valores absolutos	Baseline naive	10.012.345.269,50	100.061,71
A - Valores absolutos	Random Forest - partición temporal	12.114.233.109,67	110.064,68
A - Valores absolutos	Random Forest - TimeSeriesSplit	5.513.784.700,02	74.254,86
B - Variaciones relativas	Baseline naive	0,011272	0,106172
B - Variaciones relativas	Random Forest - partición temporal	0,004011	0,06333
B - Variaciones relativas	Random Forest - TimeSeriesSplit	0,004369	0,066101

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar, el enfoque A, el mejor resultado corresponde al Random Forest seleccionado mediante TimeSeriesSplit, que obtiene un RMSE de 74.254,86 matriculaciones sobre el conjunto de test. En cambio, respecto al enfoque B, el mejor resultado corresponde al Random Forest seleccionado mediante la partición temporal, con un RMSE de 0,063330.

No obstante, dado que los resultados de ambos enfoques no pueden compararse directamente ya que las variables objetivo se encuentran expresadas en escalas diferentes, se analizarán las ventas reconstruidas a partir de las variaciones predichas en el Enfoque B y se compararán con las predicciones obtenidas directamente mediante el Enfoque A.

4.2.4 Reconstrucción de las ventas mediante las variables predichas

Para poder interpretar los resultados del Enfoque B en términos de número de matriculaciones, se reconstruyen las ventas absolutas a partir de las variaciones interanuales predichas por el Random Forest seleccionado mediante TimeSeriesSplit de la siguiente forma:

Ilustración 101: Reconstrucción de las ventas mediante las variaciones predichas

```
In [ ]: test_var_resultados = test_var[
        ["Year", "Ventas_BMW_Unidades", "Ventas_BMW_Variacion"]
    ].copy()

test_var_resultados["Prediccion_Variacion"] = pred_test_cv_var

ventas_anterior = (
    data[data["Year"].isin(test_var_resultados["Year"] - 1)]
    [["Year", "Ventas_BMW_Unidades"]]
    .copy()
)

ventas_anterior["Year"] = ventas_anterior["Year"] + 1

ventas_anterior = ventas_anterior.rename(
    columns={
        "Ventas_BMW_Unidades":
        "Ventas_BMW_Año_Anterior"
    }
)

test_var_resultados = test_var_resultados.merge(
    ventas_anterior,
    on="Year",
    how="left"
)

test_var_resultados["Prediccion_Ventas"] = (
    test_var_resultados["Ventas_BMW_Año_Anterior"]
    * (1 + test_var_resultados["Prediccion_Variacion"])
)

display(test_var_resultados)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

De donde se puede obtener la siguiente tabla:

Ilustración 102: Reconstrucción de las ventas mediante las variaciones predichas

Año	Ventas reales	Variación real	Variación predicha	Ventas año anterior	Ventas predichas
2022	2.100.692	-5,11%	4,31%	2.213.795	2.309.211
2023	2.253.835	7,29%	4,31%	2.100.692	2.191.234
2024	2.200.177	-2,38%	4,31%	2.253.835	2.350.977
2025	2.169.761	-1,38%	4,31%	2.200.177	2.295.006

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

En dicha tabla se puede observar que el modelo genera una variación predicha constante del 4,31% para los cuatro años del conjunto de test. Lo que tiene como consecuencia que las ventas reconstruidas presenten diferencias apreciables respecto a los valores reales, especialmente en aquellos años en los que las ventas experimentaron una variación negativa.

En concreto, el modelo sobreestima las ventas en 2022, 2024 y 2025, mientras que las subestima en 2023. Este comportamiento muestra que una predicción adecuada de la variación media no implica necesariamente una predicción precisa de las ventas absolutas.

4.2.5 Evaluación de las ventas reconstruidas

Una vez transformadas las variaciones predichas en ventas absolutas, se evalúa el error cometido por el modelo sobre el conjunto de test.

Y los resultados obtenidos fueron:

- **MSE:** 21.456.592.625,46
- **RMSE:** 146.480,69 matriculaciones

Como se puede observar, el RMSE es de 146.480,69 matriculaciones y representa el error de predicción obtenido cuando el Enfoque B se expresa finalmente en la misma unidad que el Enfoque A.

Este resultado es superior al obtenido por el mejor modelo del Enfoque A, el Random Forest seleccionado mediante TimeSeriesSplit. Por tanto, aunque el Enfoque B consigue mejorar claramente al baseline cuando se evalúa directamente sobre la variación porcentual, esta ventaja no se mantiene cuando las variaciones predichas se transforman nuevamente en ventas absolutas.

En consecuencia, para el objetivo final de predecir el número de matriculaciones de BMW, el Enfoque A presenta un rendimiento superior al Enfoque B en el conjunto de test.

4.2.6 Evaluación del Baseline reconstruido

Para completar la comparación del Enfoque B en términos de ventas absolutas, se reconstruyen también las predicciones correspondientes al baseline naive.

Ilustración 103: Evaluación del Baseline Reconstruido

```

>> [ ]: test_baseline_var = test_var[
    ["Year", "Ventas_BMW_Unidades"]
].copy()

test_baseline_var["Prediccion_Variacion"] = {
    data_baseline_var.loc[
        data_baseline_var["Year"] >= 2022,
        "Prediccion_Naive_Variacion"
    ].values
}

test_baseline_var["Ventas_Año_Anterior"] = {
    test_baseline_var["Year"] - 1
}.map(
    data.set_index("Year")["Ventas_BMW_Unidades"]
)

test_baseline_var["Prediccion_Ventas"] = {
    test_baseline_var["Ventas_Año_Anterior"]
    * (1 + test_baseline_var["Prediccion_Variacion"])
}

>> [ ]: mse_baseline_var_ventas = mean_squared_error(
    test_baseline_var["Ventas_BMW_Unidades"],
    test_baseline_var["Prediccion_Ventas"]
)

rmse_baseline_var_ventas = np.sqrt(
    mse_baseline_var_ventas
)

print(
    f"RMSE baseline reconstruido: "
    f"{mse_baseline_var_ventas:.2f}"
)

print(
    f"RMSE baseline reconstruido: "
    f"{rmse_baseline_var_ventas:.2f}"
)

RMSE baseline reconstruido: 53,786,114,888.29
RMSE baseline reconstruido: 231.918,34
    
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

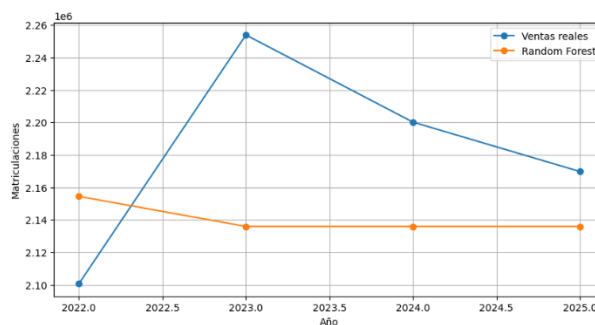
Como se puede observar, el baseline reconstruido presenta un RMSE de 231.918,34 matriculaciones, considerablemente superior al obtenido por el Random Forest, cuyo RMSE es de 146.480,69 matriculaciones. Por tanto, el Random Forest mantiene una mejora respecto al baseline también cuando las predicciones de variación se transforman nuevamente en valores absolutos.

Este resultado confirma que el modelo basado en Random Forest aporta capacidad predictiva adicional respecto al baseline dentro del Enfoque B. Sin embargo, al comparar las ventas reconstruidas con las predicciones directas del Enfoque A, el modelo de variaciones obtiene un error absoluto mayor.

4.2.7 Comparación gráfica de las predicciones

Para visualizar mejor las diferencias detectadas, se representará gráficamente las ventas reales y las predicciones obtenidas mediante los dos enfoques analizados.

Ilustración 104: Comparación gráfica de las predicciones (Enfoque A)

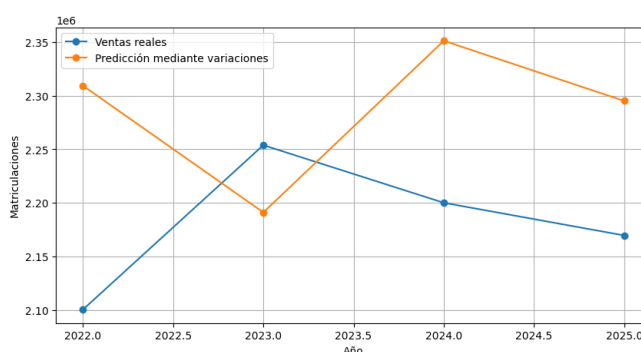


Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar, el gráfico superior muestra que el Random Forest optimizado mediante TimeSeriesSplit consigue aproximarse razonablemente al nivel real de ventas, obteniendo un RMSE de 74.254,86 unidades, mejor que el baseline de 100.061,71. Sin embargo, las predicciones de 2023, 2024 y 2025 son prácticamente iguales, lo que evidencia que el modelo tiene dificultades para reproducir la variabilidad y los cambios interanuales de las ventas.

Dicho comportamiento podría explicarse por la naturaleza del Random Forest y por el reducido número de observaciones disponibles. Por tanto, el modelo resulta útil para estimar el nivel general de ventas, pero presenta limitaciones para capturar la dinámica temporal y los puntos de inflexión de la serie.

Ilustración 105: Comparación gráfica de las predicciones (Enfoque B)



Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar, el enfoque de variaciones relativas consigue generar predicciones más variables que el modelo de niveles absolutos, evitando que las estimaciones queden prácticamente planas. Sin embargo, presenta dificultades para anticipar los cambios de tendencia de las ventas. Ya que en algunos años mantiene la dinámica observada previamente, lo que provoca sobreestimaciones cuando las ventas comienzan a disminuir.

Además, al reconstruir las ventas a partir de las variaciones predichas, los errores pueden acumularse y afectar a los años posteriores. Por tanto, aunque este enfoque representa mejor la variabilidad de la serie, los resultados muestran que la reducida cantidad de observaciones disponibles limita su capacidad para aprender y anticipar correctamente los cambios de tendencia.

4.2.8 MAE y R^2

Con el objetivo de comparar los modelos posteriormente, se han calculado e imprimido los siguientes parámetros:

Ilustración 106: MAE, RMSE y R^2

```
from sklearn.metrics import mean_absolute_error, r2_score

# MAE del Random Forest
mae_rf = mean_absolute_error(
    y_test,
    pred_test_cv
)

# R2 del Random Forest
r2_rf = r2_score(
    y_test,
    pred_test_cv
)
```

```
In [52]: mae_rf = mean_absolute_error(y_test, pred_test_cv)
rmse_rf = np.sqrt(mean_squared_error(y_test, pred_test_cv))
r2_rf = r2_score(y_test, pred_test_cv)

print(f"MAE: {mae_rf:.2f}")
print(f"RMSE: {rmse_rf:.2f}")
print(f"R2: {r2_rf:.4f}")

MAE: 67,421.47
RMSE: 74,254.86
R2: -0.8007
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Dichos parámetros se discutirán y compararán en apartados posteriores.

4.2.9 Conclusiones

En conclusión, los resultados muestran que el Random Forest basado en valores absolutos y optimizado mediante TimeSeriesSplit obtiene el mejor rendimiento cuando el objetivo es predecir directamente las ventas de BMW. Por otro lado, el enfoque de variaciones relativas (Enfoque B) también consigue mejorar respecto a su baseline cuando se evalúa la predicción de las variaciones. Sin embargo, al transformar posteriormente esas predicciones en ventas absolutas, su rendimiento es inferior al obtenido mediante el enfoque A. Por este motivo, para el conjunto de datos analizado, la predicción directa de las ventas absolutas resulta más adecuada que la predicción de sus variaciones interanuales.

4.3 Ridge Regression

A continuación, se desarrollará el modelo Ridge Regression. El cual, es un método de regresión lineal que incorpora una penalización sobre el tamaño de los coeficientes del modelo.

A lo largo de todo este apartado, este modelo se aplicará sobre las matriculaciones de BMW en valores absolutos, utilizando como variables explicativas las mismas

variables macroeconómicas empleadas en los modelos anteriores. De ahí la necesidad de plantear en los apartados anteriores dos enfoques A y B.

4.3.1 Preparación de los datos

El paso inicial será cargar y preparar los conjuntos de datos utilizados en el estudio. Para ello, se emplearán el dataset correspondiente a las matriculaciones de vehículos BMW y el dataset que contiene las diferentes variables macroeconómicas.

A continuación, se seleccionarán las variables macroeconómicas que se utilizarán posteriormente como variables explicativas del modelo y se agregan junto a las ventas de BMW utilizando la función auxiliar de añadir_variabes_macro. De donde se obtiene una matriz de 21 x 6, al igual que se puede observar en la imagen inferior.

Ilustración 107: Selección y unión de todas las variables del modelo

```
def añadir_variabes_macro(df_bmw, df_macro):
    df_final = df_bmw.merge(
        df_macro[variables_modelo],
        on="Year",
        how="left"
    )
    return df_final

df_bmw_filtrado = df_bmw.rename(
    columns={"Año": "Year"}
)

data = añadir_variabes_macro(
    df_bmw_filtrado,
    df_macro_reducido
)

display(data.head())

print("Shape del dataset definitivo:", data.shape)
```

	Year	Ventas_BMW_Unidades	Compensation_Per_Employee	Industrial_Production	Population_Total	EUR_USD
0	2005	1126768	31.22	153.2	434585887	1.2441
1	2006	1185088	31.95	147.7	436041018	1.2556
2	2007	1276793	32.75	145.5	437514477	1.3705
3	2008	1202239	33.86	148.0	439074280	1.4708
4	2009	1068770	34.41	144.5	440467898	1.3948

Shape del dataset definitivo: (21, 6)

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.3.2 Creación de la partición temporal

Una vez preparado el dataset definitivo, se procederá a realizar la partición temporal de los datos. Al igual que en el modelo Random Forest, se mantendrá el orden cronológico de las observaciones para evitar que información correspondiente a años posteriores pueda utilizarse durante el entrenamiento del modelo.

Esta división se realizará en tres conjuntos:

- **Train (2005–2018):** formado por 14 observaciones y destinado al entrenamiento del modelo.
- **Dev (2019–2021):** formado por 3 observaciones y utilizado para la evaluación durante el proceso de selección del modelo.
- **Test (2022–2025):** formado por 4 observaciones y reservado para la evaluación final del modelo.

Esta partición permite mantener unas condiciones de evaluación homogéneas con respecto a los modelos desarrollados anteriormente.

Ilustración 108: Partición temporal

```
In [6]: train = data[data["Year"] <= 2018].copy()

dev = data[
    (data["Year"] >= 2019) &
    (data["Year"] <= 2021)
].copy()

test = data[
    data["Year"] >= 2022
].copy()

print("Observaciones Train:", len(train))
print("Observaciones Dev:", len(dev))
print("Observaciones Test:", len(test))
```

Observaciones Train: 14
Observaciones Dev: 3
Observaciones Test: 4

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.3.3 Creación de X e y

Al igual que en Random Forest, se definirán las variables que utilizará el modelo Ridge Regression. Para ello, se crearán las matrices X e y.

Las cuales estarán formadas por las cuatro variables macroeconómicas seleccionadas (X) y la variable objetivo (y).

4.3.4 Preparar Ridge Regression

Como se comentó con anterioridad, en Ridge Regression es importante estandarizar las variables, porque las variables macroeconómicas están expresadas en escalas muy diferentes. Y para evitar errores y fugas de información, utilizaremos un Pipeline para encadenar varios pasos del procesamiento de los datos y del modelo en un único objeto de la siguiente forma:

Ilustración 109: Preparar Ridge Regression

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge

pipeline_ridge = Pipeline([
    ("scaler", StandardScaler()),
    ("ridge", Ridge())
])
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.3.5 Búsqueda del mejor valor de Alpha (α)

Como se comentó en el marco teórico del Ridge Regression, éste cuenta con un parámetro muy importante que se conoce como alpha (α). Y para seleccionarlo se probarán diferentes valores mediante una búsqueda sistemática utilizando GridSearchCV. Esta búsqueda se realizará junto con la validación temporal mediante TimeSeriesSplit, de manera que cada valor de alpha sea evaluado bajo las mismas particiones temporales.

Ilustración 110: Buscando el mejor valor de alpha

```
param_grid_ridge = {
    "ridge__alpha": [
        0.01,
        0.1,
        1,
        10,
        100,
        1000
    ]
}

grid_ridge = GridSearchCV(
    estimator=pipeline_ridge,
    param_grid=param_grid_ridge,
    cv=tscv,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1
)

grid_ridge.fit(
    X_train_dev,
    y_train_dev
)

GridSearchCV(cv=TimeSeriesSplit(gap=0, max_train_size=None, n_splits=4, test_size=None),
    estimator=Pipeline(steps=[('scaler', StandardScaler()),
                              ('ridge', Ridge())]),
    n_jobs=-1,
    param_grid={'ridge__alpha': [0.01, 0.1, 1, 10, 100, 1000]},
    scoring='neg_root_mean_squared_error')
```

```
print("Mejores parámetros Ridge:")
print(grid_ridge.best_params_)

print(
    "\nMejor RMSE medio de validación:",
    -grid_ridge.best_score_
)

Mejores parámetros Ridge:
{'ridge__alpha': 1}

Mejor RMSE medio de validación: 172726.51765651908
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Y una vez se haya finalizado la búsqueda, se consultará el valor de alpha que ha obtenido el mejor rendimiento medio durante las validaciones que en este caso ha

sido $\alpha = 1$, ya que es el que obtiene el menor RMSE medio (172.726,52 unidades) durante las diferentes particiones temporales utilizadas en la validación. Este valor de RMSE indica que, de media, las predicciones realizadas durante las distintas validaciones presentan un error de aproximadamente 172.727 matriculaciones respecto a los valores reales. Este resultado se utilizará como referencia para seleccionar el modelo final, que posteriormente será evaluado sobre el conjunto de test correspondiente al periodo 2022–2025.

4.3.6 Entrenar el Ridge Regression definitivo

Una vez seleccionado el mejor valor del parámetro de regularización, se procederá a entrenar el modelo Ridge definitivo. Para el cual, el valor seleccionado durante la validación mediante TimeSeriesSplit ha sido $\alpha = 1$.

Para el entrenamiento del modelo definitivo se utilizará el mejor estimador obtenido mediante GridSearchCV y se emplearán todas las observaciones disponibles hasta el año 2021, correspondientes a los conjuntos Train y Dev.

Ilustración 111: Entrenando el Ridge Regression definitivo

```
modelo_ridge = grid_ridge.best_estimator_  
  
modelo_ridge.fit(  
    X_train_dev,  
    y_train_dev  
)  
  
Pipeline(steps=[('scaler', StandardScaler()), ('ridge', Ridge(alpha=1))])
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.3.7 Realizar las predicciones sobre test

A continuación, se procederá a realizar las predicciones sobre el conjunto de test, correspondiente al periodo 2022–2025. Para ello, se utilizarán las variables macroeconómicas de estos cuatro años como entrada del modelo.

Ilustración 112: Realizar las predicciones sobre test

```
pred_test_ridge = modelo_ridge.predict(  
    X_test  
)  
predicciones_ridge = test[  
    ["Year", "Ventas_BMW_Unidades"]  
> ].copy()  
predicciones_ridge["Prediccion_Ridge"] = pred_test_ridge  
display(predicciones_ridge)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Y los resultados obtenidos son los recogidos en la tabla siguiente:

Tabla 23: Predicciones en test

Año	Ventas BMW reales	Predicción Ridge
2022	2.100.692	2.445.910
2023	2.253.835	2.542.766
2024	2.200.177	2.688.099
2025	2.169.761	2.803.533

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar, el modelo Ridge Regression sobreestima las ventas de en los cuatro años del periodo de test. La diferencia entre las predicciones y los valores reales aumenta progresivamente a lo largo del periodo, alcanzando la mayor desviación en 2025, donde el modelo predice aproximadamente 2,80 millones de matriculaciones frente a las 2,17 millones realmente observadas.

Estos resultados se utilizarán en el siguiente apartado para calcular las métricas de evaluación del modelo (MAE, RMSE y R^2).

4.3.8 Cálculo de MSE y RMSE

Una vez llevadas a cabo las predicciones sobre el conjunto de test, se procederá a evaluar el rendimiento del modelo Ridge Regression mediante las métricas MSE (Mean Squared Error) y RMSE (Root Mean Squared Error).

Y como se puede observar, los resultados son los siguientes:

Ilustración 113: Cálculo MSE y RMSE

```
from sklearn.metrics import mean_squared_error

mse_test_ridge = mean_squared_error(
    y_test,
    pred_test_ridge
)

rmse_test_ridge = np.sqrt(
    mse_test_ridge
)

print(f"MSE Ridge en test: {mse_test_ridge:.2f}")
print(f"RMSE Ridge en test: {rmse_test_ridge:.2f}")
```

MSE Ridge en test: 210,597,858,918.49
RMSE Ridge en test: 458,909.42

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.3.9 Cálculo de MAE y R^2

Para completar la evaluación del modelo Ridge Regression, se calcularán las métricas MAE (Mean Absolute Error) y R^2 (coeficiente de determinación). Las cuales

permitirán complementar los resultados obtenidos anteriormente mediante el MSE y el RMSE. Y posteriormente almacenar los resultados en una tabla.

Ilustración 114: Cálculo MAE y R^2

```
from sklearn.metrics import mean_absolute_error, r2_score

mae_test_ridge = mean_absolute_error(
    y_test,
    pred_test_ridge
)

r2_test_ridge = r2_score(
    y_test,
    pred_test_ridge
)

print(f"MAE Ridge en test: {mae_test_ridge:.2f}")
print(f"RMSE Ridge en test: {rmse_test_ridge:.2f}")
print(f"R² Ridge en test: {r2_test_ridge:.4f}")
```

MAE Ridge en test: 438,960.73
RMSE Ridge en test: 458,909.42
R² Ridge en test: -67.7760

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.3.10 Tabla comparativa de MAE, RMSE y R^2

Los resultados obtenidos anteriormente, se han recogido en una tabla:

Tabla 24: Tabla comparativa MAE, RMSE y R^2

Modelo	MAE	RMSE	R^2
Ridge Regression	438.960,73	458.909,42	-67,776

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar, el modelo Ridge Regression presenta un MAE de 438.960,73 y un RMSE de 458.909,42 matriculaciones. La proximidad entre estas dos métricas indica que los errores son elevados y que no existe una diferencia especialmente grande entre el error medio absoluto y el error penalizado por el RMSE.

Por otro lado, el R^2 de -67,7760, al tratarse de un valor negativo y considerablemente inferior a cero, indica que el modelo presenta un rendimiento muy inferior al de utilizar como referencia la media de las ventas del conjunto de test. Esto es coherente con los resultados de las predicciones observadas anteriormente, donde Ridge Regression sobreestima las ventas en los cuatro años del periodo 2022–2025 y la diferencia respecto a los valores reales aumenta progresivamente hasta 2025.

Por tanto, para este conjunto de datos, Ridge Regression no consigue capturar adecuadamente la evolución de las matriculaciones de BMW durante el periodo de test.

No obstante, estos resultados pueden deberse a que la evaluación final se realiza únicamente sobre cuatro observaciones, lo que puede provocar una elevada variabilidad en métricas como el R^2 .

4.4 Gradient Boosting

Una vez desarrollados Random Forest y Ridge Regression, se procederá a desarrollar el modelo de Gradient Boosting, con el objetivo de evaluar su capacidad predictiva sobre las ventas de BMW y comparar posteriormente sus resultados con los obtenidos mediante los modelos anteriores.

4.4.1 Pasos iniciales

El primer paso es cargar los datos de las ventas de BMW y las variables macroeconómicas y después agregarlos todos en un único dataset mediante una función auxiliar añadir_variables_macro.

A continuación, se crean las particiones temporales de la misma forma que en Random Forest y en Ridge Regression. Y posteriormente se crean X e y que contienen las variables macroeconómicas y la variable objetivo, respectivamente.

4.4.2 Preparación de los datos para TimeSeriesSplit y creación del mismo

Una vez llevados a cabo todos los pasos anteriores, se procederá con la selección de los hiperparámetros. Y para ello, se utilizarán todas las observaciones disponibles hasta el año 2021, manteniendo el conjunto de test (2022–2025) por separado. De esta forma, los datos correspondientes al periodo de evaluación final no intervienen ni en la selección de los hiperparámetros ni en el entrenamiento del modelo.

Ilustración 115: Preparación de los datos para TimeSeriesSplit

```
RANDOM_STATE = 42

data_train_dev = data[
    data["Year"] <= 2021
].copy()

X_train_dev = data_train_dev.drop(
    columns=["Year", "Ventas_BMW_Unidades"]
)

y_train_dev = data_train_dev["Ventas_BMW_Unidades"]
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

A continuación, se utilizará TimeSeriesSplit para realizar la validación cruzada respetando el orden temporal de las observaciones.

En este caso, se establecen 4 particiones ($n_splits=4$). Como resultado, se obtienen cuatro iteraciones en las que el conjunto de entrenamiento aumenta progresivamente, mientras que el conjunto de validación mantiene 3 observaciones en cada iteración al igual que se puede observar en la ilustración inferior:

Ilustración 116: Creación del TimeSeriesSplit

```
from sklearn.model_selection import TimeSeriesSplit

tscv = TimeSeriesSplit(
    n_splits=4
)

for i, (train_index, val_index) in enumerate(
    tscv.split(X_train_dev),
    start=1
):
    print(
        f"Fold {i}: "
        f"Train = {len(train_index)} observaciones, "
        f"Validación = {len(val_index)} observaciones"
    )
```

Fold 1: Train = 5 observaciones, Validación = 3 observaciones
Fold 2: Train = 8 observaciones, Validación = 3 observaciones
Fold 3: Train = 11 observaciones, Validación = 3 observaciones
Fold 4: Train = 14 observaciones, Validación = 3 observaciones

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.4.3 Creación del modelo Gradient Boosting

Una vez preparados los datos y definido el esquema de validación temporal, se procederá a crear el modelo Gradient Boosting Regressor.

Ilustración 117: Creación del modelo Gradient Boosting

```
from sklearn.ensemble import GradientBoostingRegressor

gb = GradientBoostingRegressor(
    random_state=RANDOM_STATE
)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.4.4 Definición y búsqueda de los mejores hiperparámetros

En primer lugar, para determinar la configuración más adecuada del modelo, se realizará una búsqueda de diferentes combinaciones de tres hiperparámetros ($n_estimators$, $learning_rate$ y max_depth) de la siguiente forma:

Ilustración 118: Definición de los hiperparámetros

```
param_grid_gb = {
    "n_estimators": [
        10,
        25,
        50,
        100
    ],
    "learning_rate": [
        0.01,
        0.05,
        0.1
    ],
    "max_depth": [
        1,
        2,
        3
    ]
}
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

A continuación, se utilizará GridSearchCV para evaluar todas las combinaciones de hiperparámetros definidas anteriormente. Y para la búsqueda se utilizará el esquema TimeSeriesSplit establecido previamente, de manera que cada combinación sea evaluada respetando el orden cronológico de los datos.

Ilustración 119: Búsqueda de los mejores hiperparámetros

```
from sklearn.model_selection import GridSearchCV

grid_gb = GridSearchCV(
    estimator=gb,
    param_grid=param_grid_gb,
    cv=tscv,
    scoring="neg_root_mean_squared_error",
    n_jobs=-1
)

grid_gb.fit(
    X_train_dev,
    y_train_dev
)

GridSearchCV(cv=TimeSeriesSplit(gap=0, max_train_size=None, n_splits=4, test_size=None),
    estimator=GradientBoostingRegressor(random_state=42), n_jobs=-1,
    param_grid={'learning_rate': [0.01, 0.05, 0.1],
                'max_depth': [1, 2, 3],
                'n_estimators': [10, 25, 50, 100]},
    scoring='neg_root_mean_squared_error')
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Y una vez se ha finalizado la búsqueda, se ha obtenido que la combinación de hiperparámetros que presenta el mejor comportamiento medio durante las distintas particiones temporales. Los resultados obtenidos son:

- learning_rate = 0.1
- max_depth = 3
- n_estimators = 100

Esta combinación será la seleccionada para construir el modelo definitivo. Además, es importante señalar que este valor corresponde al rendimiento medio

obtenido durante la validación cruzada sobre los datos disponibles hasta 2021, y no al resultado final sobre el conjunto de test.

La evaluación definitiva se realizará posteriormente utilizando exclusivamente las observaciones correspondientes a los años 2022–2025.

4.4.5 Obtención del modelo definitivo

Una vez seleccionada la mejor combinación de hiperparámetros mediante GridSearchCV, se procederá a obtener el modelo definitivo de Gradient Boosting. Y para ello, se utilizará `best_estimator_` que contiene el estimador configurado con los hiperparámetros que han obtenido el mejor resultado durante la validación mediante TimeSeriesSplit.

Ilustración 120: Obtención del modelo definitivo de Gradient Boosting

```
modelo_gb = grid_gb.best_estimator_  
  
modelo_gb.fit(  
    X_train_dev,  
    y_train_dev  
)  
  
GradientBoostingRegressor(random_state=42)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Posteriormente, el modelo se volverá a entrenar utilizando todas las observaciones disponibles hasta el año 2021, correspondientes a los conjuntos de Train y Dev.

4.4.6 Realizar las predicciones sobre el conjunto de test

Una vez entrenado el modelo definitivo, se procederá a realizar las predicciones sobre el conjunto de test, correspondiente al periodo 2022–2025.

Ilustración 121: Predicciones sobre el conjunto de test

```
pred_test_gb = modelo_gb.predict(  
    X_test  
)  
  
predicciones_gb = test[  
    ["Year", "Ventas_BMW_Unidades"]  
].copy()  
  
predicciones_gb["Prediccion_GB"] = pred_test_gb  
  
display(predicciones_gb)
```

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Y los resultados obtenidos son los recogidos en la tabla siguiente:

Tabla 25: Predicciones sobre el conjunto de test

Año	Ventas reales	Predicción Gradient Boosting
2022	2.100.692	2.170.307
2023	2.253.835	2.104.102
2024	2.200.177	2.104.102
2025	2.169.761	2.104.102

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Como se puede observar, el modelo sobreestima ligeramente las ventas en 2022, mientras que en 2023, 2024 y 2025 proporciona una predicción inferior al valor real. Además, las predicciones para los tres últimos años son prácticamente idénticas, situándose alrededor de 2,10 millones de unidades.

4.4.7 Calcular MSE y RMSE

Para evaluar el rendimiento del modelo sobre el conjunto de test, se calcularán el Error Cuadrático Medio (MSE) y el RMSE (Root Mean Squared Error) que se calcula a partir del MSE. Y los resultados obtenidos son los siguientes:

Ilustración 122: Cálculo MSE y RMSE

```
from sklearn.metrics import mean_squared_error

mse_test_gb = mean_squared_error(
    y_test,
    pred_test_gb
)

print(
    f"MSE Gradient Boosting en test: "
    f"{mse_test_gb:.2f}"
)
```

MSE Gradient Boosting en test: 10,201,965,929.32

```
rmse_test_gb = np.sqrt(
    mse_test_gb
)

print(
    f"RMSE Gradient Boosting en test: "
    f"{rmse_test_gb:.2f}"
)
```

RMSE Gradient Boosting en test: 101,004.78

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

4.4.8 Calcular MAE y R²

También se calcularán el Error Absoluto Medio (MAE) y el coeficiente de determinación (R²) de la siguiente forma:

Ilustración 123: Cálculo MAE y R^2

```
from sklearn.metrics import mean_absolute_error

mae_test_gb = mean_absolute_error(
    y_test,
    pred_test_gb
)

print(
    f"MAE Gradient Boosting en test: "
    f"{mae_test_gb:,.2f}"
)
```

MAE Gradient Boosting en test: 95,270.64

```
from sklearn.metrics import r2_score

r2_test_gb = r2_score(
    y_test,
    pred_test_gb
)

print(
    f"R2 Gradient Boosting en test: "
    f"{r2_test_gb:.4f}"
)
```

R² Gradient Boosting en test: -2.3317

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

4.4.9 Tabla comparativa de MAE, RMSE y R^2

Por último, se recogerán todos estos datos en una tabla de la siguiente forma:

Tabla 26: Tabla comparativa de MAE, RMSE y R^2

Modelo	MAE	RMSE	R^2
Gradient Boosting	95.270,64	101.004,78	-2,3317

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

Como se puede observar, se han obtenido un MAE de 95.270,64 unidades y un RMSE de 101.004,78 unidades. Lo que indica que, en términos absolutos, indica que, en promedio las predicciones del modelo se alejan aproximadamente 95.271 unidades de las ventas reales de BMW durante el periodo 2022–2025.

Respecto al R^2 , tiene un valor de -2,3317. El cual, al tratarse de un valor negativo, indica que el modelo presenta un rendimiento inferior al que se obtendría utilizando como predicción de referencia la media de las ventas del conjunto de test. Por tanto, aunque el modelo consigue aproximarse razonablemente a los valores reales en algunos años, no logra explicar adecuadamente la variabilidad de las ventas durante el periodo evaluado.

4.5 Baseline Naive

Por último, se desarrollará a cabo el Baseline Naive. El cual no es un modelo en sí mismo, sino un método de referencia que permitirá establecer un punto de comparación con los modelos de aprendizaje automático desarrollados anteriormente.

Su funcionamiento consiste en utilizar el valor observado en el periodo anterior como predicción para el periodo siguiente. De esta forma, se podrá evaluar hasta qué punto los modelos de Random Forest, Ridge Regression y Gradient Boosting son capaces de mejorar una predicción sencilla basada únicamente en la información del periodo anterior.

4.5.1 Pasos iniciales

Al igual que en los modelos anteriores, el primer paso será cargar los datos de ventas de BMW y las variables macroeconómicas y agregarlas en un único dataset. Para posteriormente, llevar a cabo una partición temporal. Para la cual, se utilizará la misma división que en los demás modelos.

4.5.2 Predicciones Naive

A continuación, se crea la predicción Naive mediante un desplazamiento de un periodo de las ventas de BMW. De esta forma, para cada año, la predicción corresponde al número de vehículos vendidos en el año inmediatamente anterior.

Ilustración 124: Predicciones Naive

```
data_baseline = data.copy()

data_baseline["Prediccion_Naive"] = (
    data_baseline["Ventas_BMW_Unidades"].shift(1)
)

display(
    data_baseline[
        [
            "Year",
            "Ventas_BMW_Unidades",
            "Prediccion_Naive"
        ]
    ]
)
```

Fuente: *Elaboración Propia a partir de los Cuadernos Jupyter*

Como consecuencia de este desplazamiento, el año 2005 no dispondrá de una predicción, ya que no existe en el conjunto de datos un año anterior al periodo de estudio.

4.5.3 Predicciones sobre Dev

Aunque el Baseline Naive no requiere de una fase de entrenamiento ni la estimación de hiperparámetros, se realizarán predicciones sobre el conjunto de Dev (2019–2021) para mantener la misma estructura de evaluación utilizada en los modelos de aprendizaje automático.

En los modelos anteriores, el conjunto de Dev se utilizaba durante el proceso de selección y validación de los modelos, mientras que el conjunto de Test se mantenía separado para realizar la evaluación final. En el caso del Baseline Naive, al no existir un proceso de entrenamiento, el conjunto de Dev no interviene en la selección de ningún parámetro. Sin embargo, su evaluación permite conocer cómo se comporta este método de referencia antes del periodo reservado para la evaluación final.

De esta forma, se podrá comprobar el rendimiento del modelo en un periodo intermedio y compararlo con su rendimiento sobre el conjunto de Test. Además, mantener esta división temporal permite que todos los modelos sean evaluados siguiendo una metodología homogénea.

Para esta parte de la predicción, se utilizarán las ventas del año anterior como predicción para cada observación del periodo 2019–2021 y se calcularán las métricas MAE, MSE, RMSE y R^2 , para las cuales se han obtenido los siguientes resultados:

- **MAE:** 122.827,67 unidades.
- **MSE:** 18.575.566.348,33.
- **RMSE:** 136.292,21 unidades.
- **R^2 :** -1,9915.

4.5.4 Predicciones sobre test

Una vez evaluado el comportamiento del Baseline Naive sobre Dev, se procederá a realizar las predicciones sobre el conjunto de Test (2022–2025). El cual se ha mantenido separado de los periodos anteriores y constituye la evaluación final del método de referencia.

La utilización de este conjunto permitirá comprobar el rendimiento del método sobre observaciones que pertenecen a un periodo posterior al utilizado para el desarrollo y validación de los modelos.

Para esta parte de la predicción, se utilizarán las ventas del año anterior como predicción para cada observación del periodo 2019–2021 y se calcularán las

métricas MAE, MSE, RMSE y R^2 , para las cuales se han obtenido los siguientes resultados:

- **MAE:** 87.580,00 unidades.
- **MSE:** 10.012.345.269,50.
- **RMSE:** 100.061,71 unidades.
- **R^2 :** -2,2698.

Estos resultados serán los que se utilicen posteriormente en la comparación final de los diferentes modelos.

Capítulo 5:

Comparación y evaluación de los modelos

5. Comparación y evaluación de los modelos

A lo largo de este apartado, se analizarán y compararán los resultados obtenidos por los diferentes modelos predictivos desarrollados en el estudio. Para ello, se utilizarán las métricas MAE, RMSE y R^2 , calculadas sobre el mismo conjunto de test (2022–2025), lo que permitirá realizar una comparación bajo condiciones homogéneas.

5.1 Comparación de los diferentes modelos

Una vez desarrollados los diferentes modelos predictivos, se procederá a comparar su rendimiento sobre el conjunto de test, correspondiente al periodo 2022–2025. Lo que permitirá determinar qué modelo presenta un mejor comportamiento para predecir las matriculaciones de BMW en Europa.

Para garantizar una comparación homogénea, se utilizarán las mismas métricas para todos los modelos (MAE, RMSE y R^2).

Los resultados obtenidos sobre el conjunto de test en el capítulo anterior, se muestran en la siguiente tabla:

Tabla 27: Comparación de los diferentes modelos mediante las métricas MAE, RMSE y R^2

Modelo	MAE	RMSE	R^2
Baseline Naive	87.580,00	100.061,71	-2,2698
Gradient Boosting	95.270,64	101.004,78	-2,3317
Ridge Regression	438.960,73	458.909,42	-67,776
Random Forest	67.421,47	74.254,86	-0,8007

Fuente: Elaboración Propia a partir de los Cuadernos Jupyter

A partir de los resultados obtenidos, se observa que Random Forest es el modelo que obtiene el mejor rendimiento global sobre el conjunto de test. Ya que presenta el menor MAE y RMSE con un valor de 67.421,47 y 74.254,86 unidades respectivamente. Lo que quiere decir que ,sus predicciones son en promedio más próximas a las ventas reales que las obtenidas mediante el resto de modelos.

En el caso del Baseline Naive, éste obtiene un MAE de 87.580 unidades y un RMSE de 100.061,71 unidades. Esto constituye una referencia para valorar la mejora aportada por los modelos de aprendizaje automático. Y en comparación con este método, Random Forest consigue reducir tanto el MAE como el RMSE.

Por otro lado, Gradient Boosting presenta unos resultados similares a los del Baseline Naive, aunque ligeramente peores. Su MAE asciende a 95.270,64

unidades y su RMSE a 101.004,78 unidades. Por tanto, en este caso concreto, Gradient Boosting no consigue mejorar el rendimiento del método de referencia.

En cuanto al Ridge Regression, este es el modelo que presenta el peor comportamiento observado. Ya que su MAE y RMSE alcanzan 438.960,73 y 458.909,42 unidades, respectivamente. Los cuales, son superiores a los obtenidos por los demás modelos. Además, presenta un R^2 de -67,7760, muy inferior al resto. Lo que indica una capacidad predictiva especialmente limitada sobre el periodo de test.

Por otro lado, en cuanto al R^2 , los cuatro modelos presentan valores negativos. Lo que indica que ninguno de ellos supera a una predicción basada en la media de las ventas del conjunto de test. No obstante, el R^2 debe interpretarse con especial cautela en este estudio, ya que el conjunto de test está compuesto únicamente por cuatro observaciones. Por este motivo, los errores de un único año pueden tener una influencia considerable sobre el valor final de esta métrica.

En consecuencia, tomando como referencia principalmente el MAE y el RMSE, que permiten interpretar directamente la magnitud de los errores en unidades de vehículos, Random Forest se posiciona como el modelo con mejor comportamiento predictivo entre los cuatro modelos evaluados. Ya que sus resultados muestran una mejora respecto al Baseline Naive y al resto de modelos desarrollados. Por lo que será el modelo que se tomará como referencia para las conclusiones del estudio.

Capítulo 6:

Conclusiones y Trabajo Futuro

6. Conclusiones y trabajo futuro

6.1 Conclusiones

En conclusión, a lo largo de todo este Trabajo Fin de Grado ha sido posible analizar la capacidad de diferentes modelos de aprendizaje automático para predecir las matriculaciones de vehículos BMW en Europa a partir de un conjunto de variables macroeconómicas con un número muy reducido de observaciones (21 observaciones). Lo que supone la particularidad principal de este trabajo y que dará resultados que deberán interpretarse con cierta cautela. Sobre todo en el desarrollo de los modelos predictivos. Ya que los conjuntos de dev, train y test son muy reducidos.

Para comenzar con el trabajo, se ha construido un dataset que combina la evolución histórica de las ventas de BMW con diferentes indicadores económicos y se han desarrollado y comparado varios modelos predictivos.

A lo largo del trabajo se han estudiado diferentes alternativas de modelización, incluyendo un Baseline Naive, Random Forest, Ridge Regression y Gradient Boosting. La utilización de varios modelos ha permitido establecer una comparación bajo unas condiciones homogéneas y analizar las ventajas y limitaciones de cada uno de ellos. Si bien es cierto que el Baseline Naive no es un modelo, permite tener una referencia contra la cual comparar los diferentes modelos.

Antes de proceder con la construcción y evaluación de los modelos predictivos, se realizó un análisis exploratorio de la serie histórica de matriculaciones de BMW en Europa, con el objetivo de conocer el comportamiento de la variable objetivo y detectar los principales cambios producidos durante el periodo analizado.

En primer lugar, se estudió la evolución de las ventas entre 2005 y 2025, observándose una tendencia general creciente durante buena parte del periodo. Donde las matriculaciones pasaron de 1.126.768 unidades en 2005 a 2.253.835 unidades en 2023. Llegando a alcanzar en este último año, el valor máximo de toda la serie.

Y para analizar con mayor detalle estos cambios, se calcularon tanto la variación absoluta como la variación porcentual interanual de las matriculaciones. Esto permitía identificar los años en los que las ventas experimentaron los incrementos o descensos más significativos. Además de analizar la evolución temporal, se calcularon los principales estadísticos descriptivos de las matriculaciones de BMW.

Este análisis se amplió posteriormente mediante el estudio de las variables macroeconómicas disponibles analizando su evolución temporal, distribución y posible relación con el comportamiento de las matriculaciones. Todo ello con el objetivo de seleccionar aquellas variables que pudieran aportar información relevante al modelo. Y tras utilizar Random Forest, se determinó que Compensation_Per_Employee, Industrial_Production, Population_Total y EUR_USD presentaban una mayor importancia para explicar la evolución de las ventas. Y estas fueron las que se utilizaron para todos los modelos desarrollados.

A continuación, se plantearon dos enfoques diferentes. El primero utilizó las ventas de BMW en valores absolutos como variable objetivo, mientras que el segundo trabajó con las variaciones interanuales de las ventas y de las variables macroeconómicas. Esta segunda alternativa permitió estudiar el problema desde una perspectiva centrada en los cambios relativos entre un año y el siguiente mediante Random Forest. Y se concluyó que el primer enfoque era el más adecuado para trabajar en los diferentes modelos desarrollados.

Otro de los aspectos importantes del trabajo ha sido la utilización de una estrategia de validación temporal. Dado que los datos corresponden a una serie cronológica, no resulta adecuado realizar una división aleatoria de las observaciones al igual que se hizo en el capítulo 3. Por este motivo, se utilizó TimeSeriesSplit para la selección de los hiperparámetros, manteniendo posteriormente los años 2022–2025 completamente separados para realizar la evaluación final.

Tras comparar todos modelos en una tabla, comparando el MAE, RMSE y el R^2 , se obtuvo que Random Forest fue el modelo que presentó el mejor comportamiento sobre el conjunto de test cuando se utilizaron las ventas en valores absolutos. Obteniendo, obtuvo un MAE de 67.421,47 unidades y un RMSE de 74.254,86 unidades, siendo ambas las métricas más bajas de los modelos comparados. Además, consiguió mejorar los resultados obtenidos por el Baseline Naive. Lo que quiere decir que fue capaz de realizar predicciones más próximas a las ventas reales que una predicción basada únicamente en las ventas del año anterior. No obstante, los resultados deben interpretarse con cierta cautela, ya que el conjunto de test está formado únicamente por cuatro observaciones, correspondientes al periodo 2022–2025.

Respecto a los demás modelos, Gradient Boosting obtuvo un MAE de 95.270,64 unidades y un RMSE de 101.004,78 unidades. No obstante, no consiguió superar al Baseline Naive en el periodo de evaluación. Lo que demuestra que a veces, una mayor complejidad del modelo no implica necesariamente una mejora de las predicciones cuando se trabaja con un conjunto de datos reducido. Y respecto a Ridge Regression, éste obtuvo los peores resultados con un MAE y un RMSE de

438.960,73 y 458.909,42 unidades, respectivamente. Además, mostró una tendencia a sobreestimar las ventas durante todo el periodo de test. Este comportamiento sugiere que una relación lineal entre las variables macroeconómicas y las ventas de BMW no resulta suficiente para representar adecuadamente la evolución de la variable objetivo en el periodo analizado. Lo cual es razonable, ya que la predicción de ventas debería de ser más compleja y tener más interacciones entre sus variables de entrada y tener relaciones no lineales entre ellas.

Además, respecto al coeficiente R^2 , todos los modelos obtuvieron valores negativos sobre el conjunto de test. Aunque este resultado indica que los modelos presentan margen de mejora en términos de capacidad explicativa, debe tenerse en cuenta que el conjunto de evaluación está formado únicamente por cuatro observaciones, correspondientes a los años 2022, 2023, 2024 y 2025. Por este motivo, el R^2 puede verse fuertemente condicionado por el comportamiento de un único año y no debe interpretarse de forma aislada. Aunque de todos los modelos, Random Forest presentó el comportamiento más favorable.

Por tanto, los resultados permiten concluir que Random Forest es el modelo que mejor se adapta al problema planteado dentro de las alternativas estudiadas debido a su capacidad para modelar relaciones no lineales y combinar diferentes variables explicativas. Aunque de nuevo, es importante recalcar que la calidad y cantidad de los datos disponibles condicionan significativamente los resultados de cualquier modelo de aprendizaje automático. Por lo que los modelos han estado muy limitados en este estudio y no deben interpretarse estos resultados como definitivos ante un conjunto mayor de datos

Por último, se considera que los objetivos planteados inicialmente han sido alcanzados, ya que se ha construido el conjunto de datos, se han seleccionado variables macroeconómicas, se han desarrollado diferentes modelos predictivos, se han aplicado estrategias de validación temporal y se ha realizado una comparación objetiva de sus resultados. De los cuales, Random Forest ha sido finalmente seleccionado como la alternativa con mejor rendimiento predictivo sobre el periodo de evaluación.

6.2 Trabajo futuro

Como líneas de trabajo futuro, sería interesante ampliar el conjunto de datos mediante la incorporación de un periodo histórico más amplio de ventas y los resultados serían más fiables de esta forma. O mediante las ventas mensuales o trimestrales de BMW.

Asimismo, podría mejorarse este mismo modelo incorporando nuevas variables macroeconómicas y factores específicos de BMW, así como explorando otros algoritmos de aprendizaje automático y modelos de series temporales.

También sería conveniente profundizar en la selección e interpretación de las variables, incorporar información temporal como valores retardados y realizar evaluaciones mediante ventanas temporales móviles que permitan analizar con mayor robustez la estabilidad de los modelos. Y poder desarrollar de esta forma, una herramienta capaz de estimar las matriculaciones bajo diferentes escenarios económicos.

Bibliografía

Bibliografía

Disponibilidad del código utilizado

NataliiaFakas. (2026). *TFG_BMW*. GitHub. https://github.com/NataliiaFakas/TFG_BMW

Bibliografía del proyecto

Armstrong, J. S. (2001). *Principles of forecasting: A handbook for researchers and practitioners*. Kluwer Academic Publishers.

Asenjo, R. (2025, 3 de febrero). *Qué es la Eurozona y qué países usan el euro*. LISA News. <https://www.lisanews.org/actualidad/que-es-la-eurozona-y-que-paises-usan-el-euro/>

Bhande, A. (2018, 18 de marzo). *What is underfitting and overfitting in machine learning and how to deal with it*. GreyAtom. <https://medium.com/greyatom/what-is-underfitting-and-overfitting-in-machine-learning-and-how-to-deal-with-it-6803a989c76>

Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

Boros, G. (2023, 25 de julio). *A simple introduction into supervised learning*. Medium. <https://medium.com/@gerzson.boros/a-simple-introduction-into-supervised-learning-dcce83ee3ada>

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

Breiman, L., Friedman, J. H., Olshen, R. A., & Stone, C. J. (1984). *Classification and regression trees*. Chapman & Hall/CRC.

Calvo, D. (2018, 8 de diciembre). *Perceptrón multicapa - red neuronal*. Diego Calvo. <https://www.diegocalvo.es/perceptron-multicapa/>

Cawley, G. C., & Talbot, N. L. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11, 2079–2107.

Chauhan, A. (2024, 8 de marzo). *Random forest classifier and its hyperparameters*. Analytics Vidhya. <https://medium.com/analytics-vidhya/random-forest-classifier-and-its-hyperparameters-8467bec755f6>

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

Choudhury, K. (2020, 31 de agosto). *Deep neural multilayer perceptron (MLP) with Scikit-Learn*. Towards Data Science. <https://towardsdatascience.com/deep-neural-multilayer-perceptron-mlp-with-scikit-learn-2698e77155e/>

Client challenge. (s. f.). Scribd. <https://es.scribd.com/document/654297772/9-15RN-CON-SKLEARN-1>

¿Cómo pueden las empresas sobrellevar la inflación? (2025, 5 de febrero). Channel Partner. <https://www.channelpartner.es/negocio/la-inflacion-como-afecta-la-economia-y-que-pueden-hacer-las-empresas/>

Comprobando tu navegador - reCAPTCHA. (s. f.). Kaggle. <https://www.kaggle.com/datasets/y0ussefkandil/bmw-sales2010-2024>

Consejo Europeo - lo que la gente piensa de Europa. (2020, 29 de junio). Deutschland.de. <https://www.deutschland.de/es/topic/politica/consejo-europeo-lo-que-la-gente-piensa-de-europa>

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5), 1189–1232. <https://doi.org/10.1214/aos/1013203451>

Friedman, J. H., Hastie, T., & Tibshirani, R. (2000). Additive logistic regression: A fresh look at boosting. *The Annals of Statistics*, 28(2), 337–407.

Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.

GeeksforGeeks. (s. f.). *Decision tree in machine learning*. <https://www.geeksforgeeks.org/>

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org/>

Google Developers. (s. f.). *Decision trees*.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2.^a ed.). Springer. <https://web.stanford.edu/~hastie/ElemStatLearn/>

Hernández, F. (s. f.). *3 árboles de regresión | Introducción a Machine Learning*. fheranb.github.io. https://fheranb.github.io/libro_mod_pred/arb-de-regre.html

Hoerl, A. E., & Kennard, R. W. (1970a). Ridge regression: Applications to nonorthogonal problems. *Technometrics*, 12(1), 69–82. <https://doi.org/10.1080/00401706.1970.10488635>

Hoerl, A. E., & Kennard, R. W. (1970b). Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.
<https://doi.org/10.1080/00401706.1970.10488634>

Hughes, C. M. L., et al. (2025). A comparative analysis of binary and multi-class classification machine learning algorithms to detect current frailty status using the English Longitudinal Study of Ageing (ELSA). *Frontiers in Aging*, 6, 1501168.
<https://doi.org/10.3389/fragi.2025.1501168>

Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3.^a ed.). OTexts. <https://otexts.com/fpp3/>

IBM. (s. f.). *What is a decision tree?*

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning with applications in R* (2.^a ed.). Springer. <https://www.statlearning.com/>

juliensuze. (2022, 7 de marzo). *Perceptrón: ¿Qué es y para qué sirve?* Liora. <https://liora.io/es/perceptron-que-es-y-para-que-sirve>

Ke, G., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30, 3146–3154.

Makridakis, S., & Hibon, M. (2000). The M3-Competition: Results, conclusions and implications. *International Journal of Forecasting*, 16(4), 451–476.

Makridakis, S., Smirlis, A., & Petropoulos, F. (2022). Forecasting: Theory and practice. *International Journal of Forecasting*, 38(3), 705–871.

Multiclass classification. (s. f.). UC San Diego, CSE 250B. <https://cseweb.ucsd.edu/classes/wi20/cse250B-a/lectures/multiclass.pdf>

Multiclass logistic regression. (2025, 23 de agosto). GeeksforGeeks. <https://www.geeksforgeeks.org/artificial-intelligence/multiclass-logistic-regression/>

Murphy, K. P. (2012). *Machine learning: A probabilistic perspective*. MIT Press.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Petropoulos, F., et al. (2022). The M5 Accuracy Competition: Results, findings and conclusions. *International Journal of Forecasting*, 38(4), 1346–1364.

Po, L. M. (2025, 17 de septiembre). *Backpropagation: The backbone of neural network training*. Medium. <https://medium.com/@lmpo/backpropagation-the-backbone-of-neural-network-training-64946d6c3ae5>

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.

Quinlan, J. R. (1993). *C4.5: Programs for machine learning*. Morgan Kaufmann.

Regression evaluation metrics. (s. f.). APXML. <https://apxml.com/courses/getting-started-with-scikit-learn/chapter-2-supervised-learning-regression/regression-evaluation-metrics>

Regresión logística — Aprendizaje de máquinas. (s. f.). Machine Learning Book. https://phuijse.github.io/MachineLearningBook/contents/supervised_learning/logistic.html

Schapire, R. E. (1990). The strength of weak learnability. *Machine Learning*, 5(2), 197–227.

Scikit-learn. (s. f.-a). 1. *Supervised learning*. https://scikit-learn.org/stable/supervised_learning.html

Scikit-learn. (s. f.-b). 1.1. *Linear models*. https://scikit-learn.org/stable/modules/linear_model.html

Scikit-learn. (s. f.-c). *LinearRegression*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

Scikit-learn. (s. f.-d). *LogisticRegression*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Scikit-learn. (s. f.-e). *Ordinary least squares and ridge regression*. https://scikit-learn.org/stable/auto_examples/linear_model/plot_ols_ridge.html

Scikit-learn. (s. f.-f). *Probability calibration curves*. https://scikit-learn.org/stable/auto_examples/calibration/plot_calibration_curve.html

Scikit-learn. (s. f.-g). *Scikit-learn metrics*. <https://scikit-learn.org/stable/api/sklearn.metrics.html>

Scikit-learn. (s. f.-h). *User guide*. https://scikit-learn.org/stable/user_guide.html

Scikit-learn. (s. f.-i). *A note about documentation on the logistic regression · Issue #24611 · scikit-learn/scikit-learn*. GitHub. <https://github.com/scikit-learn/scikit-learn/issues/24611>

Scikit-learn. (s. f.-j). *Scikit-learn/sklearn/linear_model/_logistic.py at main · scikit-learn/scikit-learn*. GitHub. https://github.com/scikit-learn/scikit-learn/blob/main/sklearn/linear_model/_logistic.py

Seseri, R. (2023, 6 de julio). *AI atlas: Reinforcement learning (RL)*. Glasswing Ventures. <https://glasswing.vc/blog/ai-atlas-19-reinforcement-learning-rl/>

Shalev-Shwartz, S., & Ben-David, S. (2014). *Understanding machine learning: From theory to algorithms*. Cambridge University Press.

The euro. (s. f.). Hablamos de Europa.
<https://www.hablamosdeeuropa.es/es/Paginas/El-euro.aspx>

Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288.

Understanding basic salary: The complete guide for employers - Shiftbase. (s. f.). Shiftbase. <https://www.shiftbase.com/glossary/basic-salary>

Varma, S., & Simon, R. (2006). Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7, 91.

Union Europea. (s. f.). Economía Simple. <https://economiasimple.net/union-europea>

Producto interno bruto (PIB). (s. f.). Universidad Internacional de La Rioja.
<https://ecuador.unir.net/actualidad-unir/producto-interno-bruto-pib/>

Regression overfitting. (s. f.). Crunching the Data.
<https://crunchingthedata.com/regression-overfitting/>

Why ridge regression only decreases slope and not increases it? (s. f.). Cross Validated.
<https://stats.stackexchange.com/questions/402889/why-ridge-regression-only-decreases-slope-and-not-increases-it>

Estrategias para hacer trading EUR/USD. (s. f.). TopBrokers.
<https://topbrokers.com/es/forex-strategies/estrategias-para-hacer-trading-eur-usd/>

¿Qué es Euríbor?. (s. f.). Bankinter Consumer Finance.
<https://www.bankinterconsumerfinance.com/blog/educacion-financiera/que-es-euribor>

La population du monde : croissance et vieillissement. (s. f.). Futuribles.
<https://www.futuribles.com/en/la-population-du-monde-croissance-et-vieillissement/>

Zivkovic, S. (2022, 30 de agosto). #012 *Machine learning - Introduction to random forest*. Master Data Science. <https://datahacker.rs/012-machine-learning-introduction-to-random-forest/>

Zou, H., & Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society: Series B*, 67(2), 301–320.

Anexos

Anexos

Tabla 28: Dataset de ventas de BMW en Europa

Año	Ventas BMW (unidades)	Fuente oficial BMW
2005	1.126.768	BMW Group Annual Report 2005
2006	1.185.088	BMW Group Annual Report 2006
2007	1.276.793	BMW Group Annual Report 2007
2008	1.202.239	BMW Group Annual Report 2008
2009	1.068.770	BMW Group Annual Report 2009
2010	1.224.280	BMW Group Annual Report 2010
2011	1.380.384	BMW Group Annual Report 2011
2012	1.540.085	BMW Group Annual Report 2012
2013	1.655.138	BMW Group Annual Report 2013
2014	1.811.719	BMW Group Annual Report 2014
2015	1.905.234	BMW Group Annual Report 2015
2016	2.003.359	BMW Group Annual Report 2016
2017	2.088.283	BMW Group Annual Report 2017
2018	2.125.026	BMW Group Annual Report 2018
2019	2.168.516	BMW Group Annual Report 2019
2020	2.028.659	BMW Group Report 2020
2021	2.213.795	BMW Group Report 2021
2022	2.100.692	BMW Group Report 2022
2023	2.253.835	BMW Group Report 2023
2024	2.200.177	BMW Group Report 2024
2025	2.169.761	BMW Group Report 2025

Fuente: Reportes BMW

Tabla 29: Variables Macroeconómicas

Year	Unemployment Rate	GDP Growth	Deposit facility	HICP	Industrial Production	Retail Sales Growth	Consumer Confidence	Compensation Per Employee	Brent Oil Price	EUR USD	Euribor	Population Total
2005	8.9	1.7	1.25	2.18	153.2	0.03	-4.2	31.22	54.57	1,2441	2.19	434585887
2006	8.2	3.2	2.50	2.19	147.7	0.04	-3.5	31.95	65.16	1,2556	3.08	436041018
2007	7.2	2.9	3.00	2.13	145.5	0.03	-2.1	32.75	72.44	1,3705	4.24	437514477
2008	7.1	0.4	2.00	3.30	148.0	0.01	-10.8	33.86	96.94	1,4708	4.63	439074280
2009	9.0	-4.4	0.25	0.29	144.5	-0.03	-21.7	34.41	61.74	1,3948	1.31	440467896
2010	9.7	2.1	0.25	1.62	127.2	0.01	-11.2	35.20	79.61	1,3257	0.81	441160594
2011	9.7	1.7	0.25	2.70	128.9	0.01	-15.8	35.97	111.26	1,3920	1.39	440526112
2012	10.5	-1.0	0.00	2.50	125.5	0.00	-20.7	36.55	111.63	1,2848	0.57	441280191
2013	10.9	-0.2	0.00	1.35	120.7	0.00	-13.7	37.13	108.56	1,3281	0.22	441739935
2014	10.2	1.4	-0.20	0.44	116.5	0.01	-10.5	37.64	98.97	1,3285	0.21	442235565
2015	9.4	2.1	-0.30	0.18	109.0	0.02	-6.1	38.22	52.32	1,1095	0.00	442883289
2016	8.6	1.8	-0.40	0.24	100.0	0.02	-5.3	38.71	43.73	1,1069	-0.08	443954935
2017	7.7	2.6	-0.40	1.53	94.6	0.04	-2.5	39.44	54.19	1,1297	-0.19	444609939
2018	6.8	1.8	-0.40	1.76	92.4	0.03	-1.8	40.32	71.31	1,1810	-0.17	445227547
2019	6.3	1.6	-0.50	1.19	88.1	0.03	-6.4	41.23	64.21	1,1195	-0.22	446060538
2020	7.1	-6.0	-0.50	0.25	82.9	-0.00	-15.9	41.08	41.84	1,1413	-0.30	446923963
2021	7.0	6.4	-0.50	2.60	76.0	0.07	-7.0	42.88	70.68	1,1833	-0.26	445767723
2022	6.2	3.6	2.00	8.37	77.0	0.06	-22.9	44.84	100.94	1,0530	0.34	445999571
2023	5.9	0.4	4.00	5.46	77.6	0.01	-16.3	47.23	82.41	1,0813	3.86	447805685
2024	5.9	1.0	3.00	2.37	71.5	-0.07	-12.1	49.36	80.53	1,0810	3.68	449489862
2025	6.0	1.2	2.00	2.10	69.0	0.03	-13.0	51.27	68.89	1,0800	2.45	451284558

Fuente: Reportes BMW